Python para Matemática

Material de apoio para 10 aulas

MAT 172 – Atividades computacionais com JupyterLab



Baseado no roteiro das aulas da disciplina, no livro
Python para Matemáticos (A. L. L. Souza, SBM, 2023, cap. 3 a 6)
e no *Tutorial rápido de Python para Matemáticos* (R. M. Martins, 2022).

Versão do professor

(contém notas ao professor e soluções dos exercícios)

3 de novembro de 2026

Apresentação

Este material organiza a parte final da disciplina MAT 172 em **dez aulas de 60 a 90 minutos**, usando a linguagem Python no **JupyterLab**. Ele substitui as atividades que antes eram feitas com o software Maxima e segue o roteiro abaixo:

Aula	Tema	Conteúdo principal
1	Revisão de Python I	JupyterLab; `print`, `input`, `int`, `float`, `type`; variáveis; operações aritméticas; comparação; `and`, `or`, `not`; `if`/`elif`/`else`; `for`; `while`; indentação.
2	Revisão de Python II	Listas, tuplas (e dicionários); fatiamento; percorrer listas; definição de funções com `def`, `return`, parâmetros e `lambda`.
3	O módulo `math`	Importação de módulos; piso e teto; raiz, exponencial, logaritmo; trigonometria; fatorial e combinações; raiz quadrada pelo método babilônico; teste de primalidade; bissecção.
4	NumPy I: vetores	`array`, `arange`, `linspace`; operações elemento a elemento; funções universais; indexação e máscaras; somas, médias; regra dos trapézios.
5	NumPy II: matrizes	Matrizes 2D, produto `@`, transposta, `det`, `inv`, `solve`, autovalores; eliminação de Gauss; polinômios (`roots`, `polyval`).
6	Gráficos (Matplotlib)	`plot`, estilos, legendas e eixos; vários gráficos; curvas paramétricas e polares; curvas de nível; superfícies 3D; salvar figuras.
7	SymPy I: álgebra	Símbolos e expressões; `expand`, `factor`, `simplify`; `subs`, `evalf`; `solve`, `nsolve`, `linsolve`; gráficos do SymPy.
8	SymPy II: limites e derivadas	`limit` (laterais, no infinito); derivada pela definição; `diff`; reta tangente; máximos e mínimos; série de Taylor.
9	SymPy III: integrais	`integrate`: indefinidas, definidas, impróprias; áreas; volumes de revolução; somas de Riemann; integrais duplas.
10	SymPy IV: matrizes e EDOs	`Matrix`, determinante, inversa, `rref`, autovalores; equações diferenciais com `dsolve`; atividade de encerramento.

As aulas 1 e 2 seguem o capítulo 3 do livro *Python para Matemáticos* (SBM); a aula 3 segue o capítulo 4; as aulas 4 e 5, o capítulo 5; a aula 6, o capítulo 6. As aulas 7 a 10 usam o *Tutorial rápido de Python para Matemáticos* de R. M. Martins e reproduzem, com o SymPy, os tipos de cálculo que eram feitos no Maxima. O Apêndice A traz uma tabela de correspondência entre comandos do Maxima e do SymPy.

Como ler este material

Os códigos aparecem como **células do JupyterLab**. O número entre colchetes à esquerda é a ordem de execução dentro da aula, como no Jupyter. Logo abaixo de cada célula, marcada por uma barra vermelha, aparece a **saída** que o Jupyter mostra:

[1]:
```
x = 7
print("O dobro de x é", 2*x)
```

```
O dobro de x é 14
```

Quando a última linha de uma célula é uma expressão (sem `print`), o Jupyter mostra o seu valor automaticamente. No caso do SymPy, o resultado aparece formatado como fórmula matemática:

[2]:
```
import sympy as sp
x = sp.symbols('x')
sp.expand((x + 1)**3)
```

[2]:
$$x^3 + 3x^2 + 3x + 1$$

Ao longo do texto também aparecem caixas com finalidades específicas:

- **Sintaxe**: a forma geral de um comando;
- **Observação** e **Dica**: detalhes e atalhos úteis;
- **Atenção**: erros comuns dos estudantes;
- **Para quem vem do Maxima**: o comando equivalente no Maxima;
- **Exercício** e **Desafio**: atividades para a aula ou para casa.

Cada aula começa com os objetivos e um **roteiro sugerido** com a divisão do tempo. Os tempos somam cerca de 80 minutos; os exercícios marcados como "extra" podem ser usados quando a aula tiver 90 minutos, ou deixados para casa.

Nota ao professor

Esta é a versão do professor. Ela contém notas como esta (sugestões de condução da aula, erros frequentes, perguntas para a turma) e, no Apêndice C, as soluções de todos os exercícios. A versão do estudante (`curso-aluno.pdf`) é idêntica, mas sem as notas e sem as soluções.
Todos os códigos deste material foram executados com Python 3.13, NumPy 2.5, SymPy 1.14 e Matplotlib 3.11; as saídas mostradas são as reais. Os notebooks de cada aula, prontos para abrir no JupyterLab, estão na pasta `notebooks/`.

Antes da primeira aula

- Verifique se o JupyterLab abre nas máquinas do laboratório (no terminal: `jupyter lab`).
- Teste se as bibliotecas estão instaladas, executando em uma célula:

```
import numpy, sympy, matplotlib
print(numpy.__version__, sympy.__version__, matplotlib.__version__)
```

- Se faltar alguma, instale com `pip install numpy sympy matplotlib jupyterlab` (ou, dentro do próprio notebook, `%pip install sympy`).
- Alternativas sem instalação: `https://jupyter.org/try-jupyter` (JupyterLite, roda no navegador) ou o Google Colab (`https://colab.research.google.com`).

Sumário

Apresentação **ii**

1 Revisão de Python I: comandos básicos, decisões e repetições **1**
1.1 O ambiente JupyterLab . . . 1
1.2 Primeiros comandos: `print` e comentários . . . 2
1.3 Variáveis e atribuição . . . 3
1.4 Tipos de dados e conversões . . . 4
1.5 Operações aritméticas . . . 6
1.6 Entrada de dados com `input` e saída formatada . . . 7
1.7 Comparações e operadores lógicos . . . 7
1.8 Decisões: `if`, `elif` e `else` . . . 8
1.9 Repetições com `for` . . . 10
1.10 Repetições com `while` . . . 12
1.11 Exercícios . . . 13

2 Revisão de Python II: listas, tuplas e funções **15**
2.1 Listas . . . 15
2.1.1 Índices . . . 15
2.1.2 Fatiamento . . . 16
2.1.3 Operações e métodos . . . 17
2.1.4 Cópia ou novo nome? . . . 17
2.2 Percorrendo e construindo listas . . . 18
2.3 Tuplas e dicionários . . . 19
2.4 Funções . . . 20
2.5 Exercícios . . . 23

3 O módulo `math` **25**
3.1 Módulos e importação . . . 25
3.2 Constantes e funções do módulo `math` . . . 26
3.2.1 Piso, teto e arredondamentos . . . 27
3.2.2 Raízes, exponenciais e logaritmos . . . 28
3.2.3 Trigonometria . . . 28
3.2.4 Funções para inteiros . . . 29
3.3 Aplicações em geometria . . . 29
3.4 Atividade: a raiz quadrada pelo método babilônico . . . 30
3.5 Aplicação: teste de primalidade . . . 32
3.6 Aplicação: o método da bissecção . . . 33
3.7 Exercícios . . . 34

4 NumPy I: vetores (*arrays*) e cálculo numérico **35**
4.1 Por que NumPy? . . . 35
4.2 Criando *arrays* . . . 36
4.3 Operações elemento a elemento . . . 37
4.3.1 Funções universais . . . 37
4.4 Índices, fatias e máscaras . . . 38
4.5 Somas, médias e outras reduções . . . 39
4.6 Aplicações . . . 39
4.6.1 Somas de séries sem laços . . . 39
4.6.2 Integrais pela regra dos trapézios . . . 40
4.6.3 Números aleatórios: estimando π por Monte Carlo . . . 41
4.6.4 Notação científica . . . 42
4.7 Exercícios . . . 42

5 NumPy II: matrizes, sistemas lineares e polinômios **44**
5.1 Matrizes como *arrays* bidimensionais . . . 44
5.2 Operações com matrizes . . . 46
5.3 Sistemas lineares . . . 47
5.4 Programando a eliminação de Gauss . . . 48
5.5 Autovalores e autovetores . . . 50
5.6 Polinômios . . . 51
5.7 Exercícios . . . 51

6 Gráficos com Matplotlib **53**
6.1 O primeiro gráfico . . . 53
6.2 Personalizando o gráfico . . . 54
6.3 Vários gráficos em uma figura . . . 56
6.4 Descontinuidades e assíntotas . . . 58
6.5 Curvas paramétricas e polares . . . 58
6.6 Funções de duas variáveis: curvas de nível . . . 60
6.7 Gráficos em três dimensões . . . 61
6.8 Salvando figuras . . . 63
6.9 Exercícios . . . 63

7 SymPy I: cálculo simbólico, álgebra e equações **65**
7.1 Cálculo simbólico . . . 65
7.2 Símbolos e expressões . . . 66
7.3 Manipulando expressões . . . 67
7.4 Equações . . . 68
7.4.1 `solve` . . . 68
7.4.2 `solveset`: soluções em um conjunto . . . 69
7.4.3 Sistemas de equações . . . 70
7.4.4 Quando não há fórmula: `nsolve` . . . 70

7.5 Somatórios e números inteiros . . . 71
7.6 Gráficos com o SymPy . . . 72
7.7 Exercícios . . . 73

8 SymPy II: limites, derivadas e séries de Taylor 75
8.1 Limites . . . 75
8.2 Derivadas . . . 77
8.2.1 Pela definição . . . 77
8.2.2 O comando `diff` . . . 77
8.3 Reta tangente . . . 79
8.4 Máximos e mínimos . . . 79
8.5 Polinômios de Taylor . . . 81
8.6 Exercícios . . . 83

9 SymPy III: integrais e aplicações 84
9.1 Integrais indefinidas e definidas . . . 84
9.2 Integrais impróprias e valores numéricos . . . 86
9.3 Somas de Riemann . . . 86
9.4 Área entre curvas . . . 87
9.5 Volumes de sólidos de revolução . . . 89
9.6 Integrais duplas . . . 89
9.7 Exercícios . . . 90

10 SymPy IV: matrizes simbólicas e equações diferenciais 92
10.1 Matrizes no SymPy . . . 92
10.2 Equações diferenciais ordinárias . . . 95
10.2.1 Primeira ordem . . . 95
10.2.2 Segunda ordem . . . 96
10.2.3 Campo de direções . . . 97
10.2.4 Extra: sistemas e soluções numéricas . . . 98
10.3 Atividade de encerramento . . . 99
10.4 Exercícios . . . 99

A Do Maxima para o Python 101

B Referência rápida 104

C Soluções dos exercícios 106

Referências 122

CAPÍTULO 1

Revisão de Python I: comandos básicos, decisões e repetições

Objetivos da aula

Ao final desta aula o estudante deve ser capaz de:

- usar o JupyterLab: criar um notebook, executar células e reiniciar o *kernel*;
- usar os comandos `print()`, `input()`, `int()`, `float()`, `str()` e `type()`;
- atribuir valores a variáveis e usar as operações aritméticas e os operadores de atribuição;
- escrever condições com operadores de comparação e com `and`, `or`, `not`;
- usar `if`, `elif`, `else`, `for` e `while`, respeitando a indentação.

Roteiro sugerido

Tempo	Atividade
0–10 min	O JupyterLab: notebook, células, `Shift+Enter`, células de texto.
10–25 min	`print`, comentários, variáveis, tipos de dados e conversões.
25–40 min	Operações aritméticas, precedência, `input()` e f-strings.
40–50 min	Comparações e operadores lógicos.
50–65 min	`if/elif/else` e a indentação.
65–80 min	Repetições com `for` e `while`. Exercícios.

1.1 O ambiente JupyterLab

O **JupyterLab** é um ambiente que roda no navegador e permite misturar, em um mesmo documento (o *notebook*, arquivo `.ipynb`), trechos de código, os resultados desse código, textos e fórmulas. Para abri-lo, digite no terminal

```
jupyter lab
```

ou procure "JupyterLab" no menu do sistema. Na tela inicial (*Launcher*), clique em **Notebook** → **Python 3**. A Figura 1.1 mostra as partes principais da janela.

Células. Um notebook é uma sequência de *células*. Há dois tipos principais:

- **Code** (código): contém comandos Python. Ao executá-la, o resultado aparece logo abaixo;
- **Markdown** (texto): contém títulos, explicações e fórmulas em LaTeX, como `$x^2+1$`.

Para executar a célula selecionada, use **Shift+Enter** (executa e passa para a próxima) ou **Ctrl+Enter** (executa e permanece na mesma). O número que aparece em `[ ]:` indica a *ordem* em que as células foram executadas; uma célula com `[*]:` ainda está em execução.

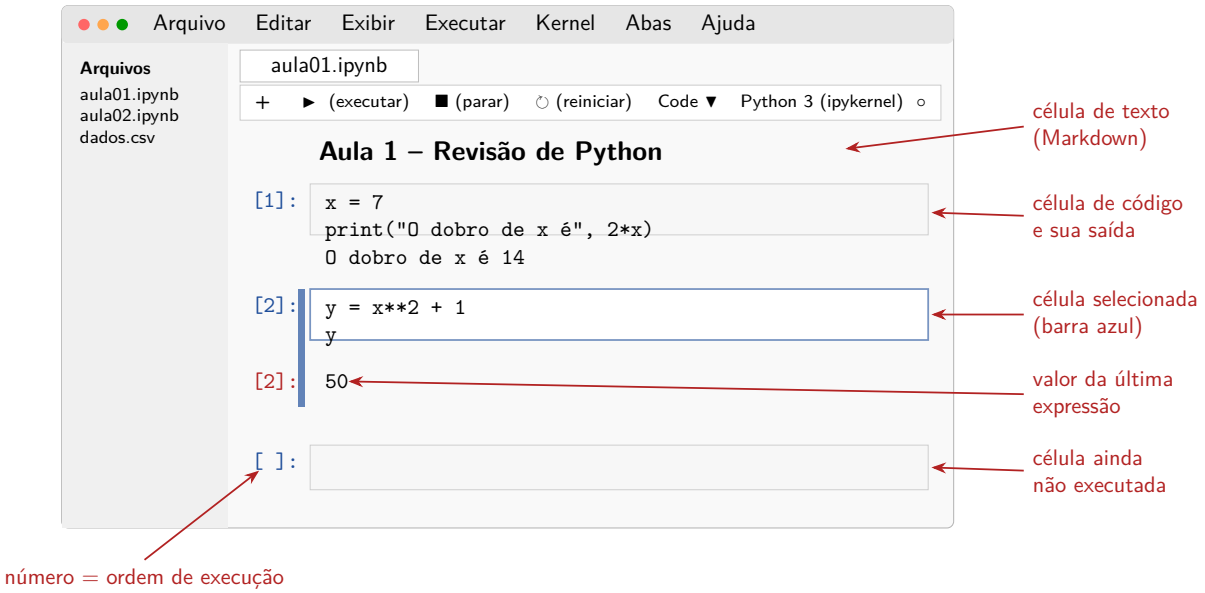


Figura 1.1: Esquema da janela do JupyterLab com um notebook aberto.

Atalho	Efeito
`Shift+Enter`	executa a célula e seleciona a próxima
`Ctrl+Enter`	executa a célula e continua nela
`Esc` / `Enter`	sai do modo de edição / entra no modo de edição
`A` / `B` (fora da edição)	insere célula acima / abaixo
`D`, `D` (fora da edição)	apaga a célula
`M` / `Y` (fora da edição)	transforma a célula em texto / em código
`Ctrl+S`	salva o notebook
`Tab`	completa o nome de um comando ou variável
`Shift+Tab`	mostra a ajuda do comando onde está o cursor

Atenção: a ordem de execução importa

O Python "lembra" tudo o que já foi executado, *na ordem em que foi executado*, e não na ordem em que as células aparecem na tela. Se algo estranho acontecer, use **Kernel → Restart Kernel and Run All Cells** para recomeçar do zero e executar tudo de cima para baixo.

Para quem vem do Maxima

No wxMaxima as entradas aparecem como `(%i1)` e as saídas como `(%o1)`; no Jupyter são `[1]:`. No Maxima cada comando termina com `;` (ou `$` para não mostrar o resultado) e se executa com `Shift+Enter`, como aqui. Em Python **não** se usa ponto e vírgula no fim da linha.

1.2 Primeiros comandos: `print` e comentários

O comando `print()` mostra na tela um ou mais valores, separados por vírgula. Tudo que vem depois do símbolo `#` é um **comentário**: o Python ignora, mas serve para explicar o código a quem lê.

[1]:
```
# meu primeiro programa
print("Olá, MAT 172!")
print("2 + 3 =", 2 + 3)        # texto e número na mesma linha
print(1, 2, 3, sep=" < ")      # sep: o que vai entre os valores
print("sem pular linha", end=" ... ")
print("continuação")
```

```
Olá, MAT 172!
2 + 3 = 5
1 < 2 < 3
sem pular linha ... continuação
```

Sintaxe

```
print(valor1, valor2, ..., sep=" ", end="\n")
```

Por padrão os valores são separados por um espaço (`sep=" "`) e, no final, o `print` pula uma linha (`end="\n"`). Textos (*strings*) ficam entre aspas simples `'...'` ou duplas `"..."`.

Dica

Para ver a ajuda de qualquer comando, use `help(print)` ou digite `print?` em uma célula.

1.3 Variáveis e atribuição

Uma **variável** é um nome associado a um valor. A associação é feita com o sinal `=`, que em Python **não** significa igualdade matemática, mas sim *atribuição*: "calcule o lado direito e guarde o resultado com o nome do lado esquerdo".

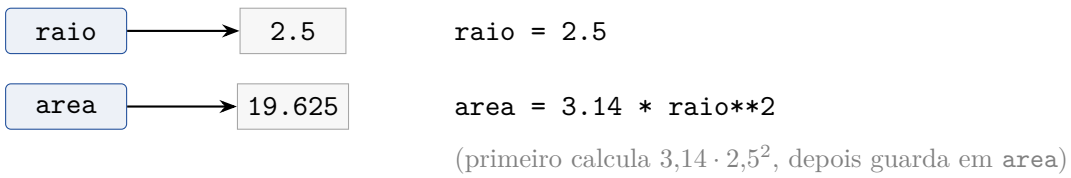


Figura 1.2: Uma variável é um nome (etiqueta) que aponta para um valor guardado na memória.

[2]:
```
raio = 2.5
area = 3.14 * raio**2
print("A área do círculo é", area)
raio = 10          # a variável pode receber outro valor
print("Novo raio:", raio, "  área antiga:", area)
```

```
A área do círculo é 19.625
Novo raio: 10   área antiga: 19.625
```

Observe que mudar `raio` *não* recalcula `area`: a conta foi feita no momento da atribuição.

Regras para nomes. Os nomes podem ter letras, dígitos e o sublinhado `_`, mas **não podem começar com dígito** nem conter espaços ou hífen. Letras maiúsculas e minúsculas são diferentes (`x` e `X` são variáveis distintas). Também não se pode usar as *palavras reservadas*:

```
False   None    True   and    as      assert    break  class   continue
def     del     elif   else   except  finally   for    from    global
if      import  in     is     lambda  nonlocal  not    or      pass
raise   return  try    while  with    yield
```

Válidos	Inválidos
`x`, `x1`, `soma_total`, `Area`, `_aux`	`1x`, `soma total`, `soma-total`, `if`, `for`

Atribuição múltipla. Podemos atribuir vários valores de uma só vez, o que permite, por exemplo, *trocar* os valores de duas variáveis sem variável auxiliar:

[3]:
```
a, b = 3, 8
print("antes:  a =", a, " b =", b)
a, b = b, a
print("depois: a =", a, " b =", b)
```

```
antes:  a = 3  b = 8
depois: a = 8  b = 3
```

Operadores de atribuição. É muito comum atualizar uma variável a partir do seu próprio valor, como em `n = n + 1`. Para isso existem atalhos:

Operador	Exemplo	Equivale a
`+=`	`n += 2`	`n = n + 2`
`-=`	`n -= 2`	`n = n - 2`
`*=`	`n *= 2`	`n = n * 2`
`/=`	`n /= 2`	`n = n / 2`
`//=`	`n //= 2`	`n = n // 2`
`%=`	`n %= 2`	`n = n % 2`
`**=`	`n **= 2`	`n = n ** 2`

[4]:
```
n = 13
n -= 15;  print(n)
n **= 4;  print(n)
n /= 4;   print(n)
n %= 3;   print(n)
```

```
-2
16
4.0
1.0
```

Observação

O ponto e vírgula `;` permite escrever dois comandos na mesma linha. Isso é raro em Python e só foi feito aqui para economizar espaço; o normal é um comando por linha.

1.4 Tipos de dados e conversões

Todo valor em Python tem um **tipo**, que pode ser consultado com `type()`. Os tipos básicos são:

Tipo	Nome	Exemplos	Em matemática
`int`	inteiro	`7`, `-3`, `0`, `10**30`	$\mathbb{Z}$ (sem limite de tamanho)
`float`	ponto flutuante	`2.5`, `-0.001`, `1.23e8`, `7.0`	aproximação de $\mathbb{R}$
`complex`	complexo	`3+2j`, `1j`	$\mathbb{C}$ (a unidade imaginária é `j`)
`bool`	lógico	`True`, `False`	verdadeiro, falso
`str`	texto (*string*)	`"MAT 172"`, `'x'`, `"3"`	—

[5]:
```
print(type(7), type(7.0), type(3+2j), type(True), type("7"))
print(1.23e8)        # notação científica: 1,23 x 10^8
print(2**100)        # inteiros não têm limite de tamanho
```

```
<class 'int'> <class 'float'> <class 'complex'> <class 'bool'> <class 'str'>
123000000.0
1267650600228229401496703205376
```

Conversões. Os comandos `int()`, `float()`, `str()` e `complex()` convertem um valor de um tipo para outro. Atenção: `int()` *corta* a parte decimal (não arredonda); para arredondar use `round()`.

[6]:
```
print(int(3.99), int(-3.99), round(3.99), round(3.14159, 2))
print(float(5), float("2.5"), int("42") + 1)
print(str(10) + str(20))       # junção de textos, não soma!
```

```
3 -3 4 3.14
5.0 2.5 43
1020
```

[7]:
```
int("3.5")
```

```
ValueError: invalid literal for int() with base 10: '3.5'
```

Atenção: `float` é uma aproximação

Os números `float` são guardados em base 2 com cerca de 16 algarismos significativos. Por isso, alguns resultados "simples" não são exatos:

[8]:
```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
print(abs((0.1 + 0.2) - 0.3) < 1e-12)   # forma correta de comparar floats
```

```
0.30000000000000004
False
True
```

Esse fenômeno será retomado na Aula 7: o SymPy trabalha com frações e raízes *exatas*, como o Maxima.

1.5 Operações aritméticas

Operação	Símbolo	Exemplo	Resultado
adição	`+`	`7 + 2`	`9`
subtração	`-`	`7 - 2`	`5`
multiplicação	`*`	`7 * 2`	`14`
divisão	`/`	`7 / 2`	`3.5` (sempre `float`)
quociente inteiro	`//`	`7 // 2`	`3`
resto da divisão	`%`	`7 % 2`	`1`
potência	`**`	`7 ** 2`	`49`
raiz n-ésima	`** (1/n)`	`8 ** (1/3)`	`2.0`

Atenção

Em Python a potência é `**` e **não** `^`. O símbolo `^` existe, mas é outra operação ("ou exclusivo" bit a bit): `2^3` dá `1`, e não `8`.

Precedência. Vale a convenção da matemática: primeiro parênteses, depois potências, depois `*`, `/`, `//`, `%` (da esquerda para a direita) e por último `+` e `-`.

[9]:
```
print(3 + 2 * 4, (3 + 2) * 4)
print(2 ** 3 ** 2)        # potência associa da direita: 2**(3**2)
print(-2 ** 2, (-2) ** 2)  # cuidado com o sinal!
print(1 / 2 * 4)          # (1/2)*4, e não 1/(2*4)
```

```
11 20
512
-4 4
2.0
```

Quociente e resto. Para inteiros a e $b > 0$, o algoritmo da divisão garante $a = b \cdot q + r$ com $0 \leq r < b$. Em Python, $q =$ `a // b` e $r =$ `a % b`; o comando `divmod(a, b)` devolve os dois de uma vez. Exemplo (do livro da SBM): converter minutos em horas e minutos.

[10]:
```
minutos = 700
horas = minutos // 60
resto = minutos % 60
print(minutos, "minutos correspondem a", horas, "horas e", resto, "minutos")
print(divmod(700, 60))
print(17 % 2, 18 % 2)     # o resto por 2 diz se o número é par
```

```
700 minutos correspondem a 11 horas e 40 minutos
(11, 40)
1 0
```

Números complexos. A unidade imaginária é escrita `j` (colada ao número):

[11]:
```
z1 = 3 + 4j
z2 = 2 - 2j
print(z1 + z2, z1 * z2, z1 / z2)
print(z1.real, z1.imag, z1.conjugate(), abs(z1))
```

```
(5+2j) (14+2j) (-0.25+1.75j)
3.0 4.0 (3-4j) 5.0
```

1.6 Entrada de dados com `input` e saída formatada

O comando `input("mensagem")` mostra a mensagem, espera o usuário digitar algo e apertar **Enter**, e devolve o que foi digitado **sempre como texto** (`str`). Para fazer contas, é preciso converter:

[12]:
```
n = int(input("Digite um inteiro: "))
x = float(input("Digite um real: "))
print("n + x =", n + x)
```

```
Digite um inteiro: 3
Digite um real: 4.5
n + x = 7.5
```

Atenção

Sem a conversão, `input()` devolve texto e o "+" junta textos: se o usuário digitar 3 e 4, `input() + input()` dá `"34"`. Outro erro comum é digitar vírgula decimal: `float("2,5")` gera erro; em Python o separador decimal é o **ponto**.

f-strings. Para montar mensagens com valores, a forma mais prática é a *f-string*: coloca-se um `f` antes das aspas e as expressões entre chaves `{}`. Depois de dois-pontos, pode-se indicar o formato:

[13]:
```
x = 2 / 3
print(f"x = {x}")
print(f"x com 3 casas: {x:.3f}")
print(f"notação científica: {12345.678:.2e}")
print(f"{7} ao quadrado é {7**2}")
```

```
x = 0.6666666666666666
x com 3 casas: 0.667
notação científica: 1.23e+04
7 ao quadrado é 49
```

1.7 Comparações e operadores lógicos

Comparações produzem valores do tipo `bool`: `True` (verdadeiro) ou `False` (falso).

Matemática	Python	Exemplo	Resultado
$=$	`==`	`3 == 3.0`	`True`
$\neq$	`!=`	`3 != 4`	`True`
$<$, $\leq$	`<`, `<=`	`5 <= 5`	`True`
$>$, $\geq$	`>`, `>=`	`2 > 7`	`False`
$0 < x < 1$	`0 < x < 1`	`0 < 0.5 < 1`	`True`

Atenção

Um sinal de igual (=) *atribui*; dois sinais (==) *comparam*.

Condições podem ser combinadas com os **operadores lógicos**:

- `A and B`: verdadeiro só quando `A` e `B` são ambos verdadeiros (conectivo "e", $\wedge$);
- `A or B`: verdadeiro quando pelo menos um deles é verdadeiro ("ou", $\vee$);
- `not A`: inverte o valor lógico (negação, $\neg$).

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

[14]:
```
x = 7
print(x > 0 and x % 2 == 0)      # positivo E par?
print(x > 0 or x % 2 == 0)       # positivo OU par?
print(not x > 10)
print(2 <= x < 10)               # x pertence a [2, 10)?
```

```
False
True
True
True
```

A ordem de prioridade é: comparações, depois `not`, depois `and` e por último `or`. Na dúvida, use parênteses.

1.8 Decisões: `if`, `elif` e `else`

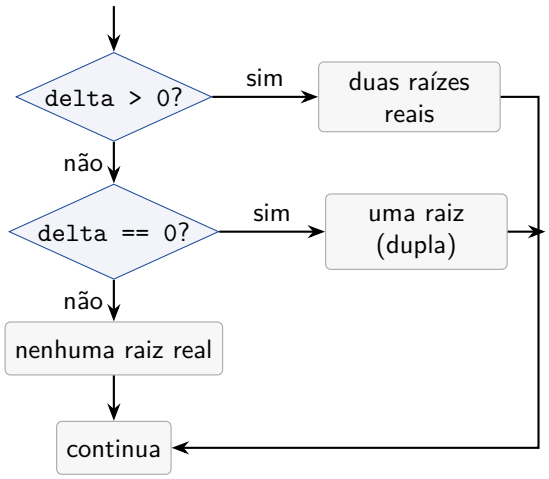
Sintaxe

```
if condição_1:
    comandos executados se a condição_1 for verdadeira
elif condição_2:
    comandos executados se a condição_1 for falsa e a condição_2 verdadeira
else:
    comandos executados se nenhuma condição anterior for verdadeira
```

As partes `elif` ("senão, se") e `else` ("senão") são opcionais, e pode haver quantos `elif` forem necessários. Note os **dois-pontos** no fim de cada linha de `if`, `elif` e `else`.

Indentação. Em Python, os comandos que pertencem ao `if` são identificados pelo **recuo** (indentação) em relação à margem: o padrão são **4 espaços** (o Jupyter insere automaticamente após os dois-pontos). Em muitas linguagens usam-se chaves `{ }`; no Python o recuo *faz parte da sintaxe*.

Exemplo: raízes de uma equação do segundo grau. Para $ax^2 + bx + c = 0$ com $a \neq 0$, o número de raízes reais depende do sinal de $\Delta = b^2 - 4ac$ (Figura 1.3).




```
if delta > 0:
    print("duas raízes reais")
elif delta == 0:
    print("uma raiz (dupla)")
else:
    print("nenhuma raiz real")
print("continua")
```

4 espaços de recuo:
o comando pertence ao bloco

Figura 1.3: Fluxograma de um `if`/`elif`/`else` e o código correspondente.

[15]:
```
a, b, c = 1, -5, 6
delta = b**2 - 4*a*c
if delta > 0:
    x1 = (-b + delta**0.5) / (2*a)
    x2 = (-b - delta**0.5) / (2*a)
    print(f"Duas raízes reais: {x1} e {x2}")
elif delta == 0:
    print("Uma raiz real dupla:", -b / (2*a))
else:
    print("Não há raízes reais (delta < 0)")
print("delta =", delta)
```

```
Duas raízes reais: 3.0 e 2.0
delta = 1
```

O último `print` não está recuado, portanto **não** faz parte do `if`: ele sempre é executado.

[16]:
```
n = 15
if n % 2 == 0:
    print(n, "é par")
else:
    print(n, "é ímpar")

if n % 3 == 0 and n % 5 == 0:
    print(n, "é múltiplo de 3 e de 5")
```

```
15 é ímpar
15 é múltiplo de 3 e de 5
```

Erros de indentação. Esquecer o recuo (ou os dois-pontos) gera um erro de sintaxe:

[17]:
```
exec("""
if 2 > 1:
print("dois é maior que um")
""")
```

```
IndentationError: expected an indented block after 'if' statement on line 2
    ↪ (<string>, line 3)
```

Observação

No Jupyter, a célula com erro seria simplesmente `if 2 > 1:` seguido de `print(...)` sem recuo; usamos `exec` aqui só para poder mostrar a mensagem. Leia sempre a **última linha** da mensagem de erro: ela diz o tipo do erro e onde ocorreu.

Exemplo: classificação de triângulos. Este exemplo, adaptado do livro da SBM, lê os lados a, b, c e verifica primeiro se eles formam um triângulo (desigualdade triangular), corrigindo os problemas apontados no livro.

[18]:
```
a = float(input("Lado a: "))
b = float(input("Lado b: "))
c = float(input("Lado c: "))
if a <= 0 or b <= 0 or c <= 0 or a >= b + c or b >= a + c or c >= a + b:
    print("Esses valores não formam um triângulo.")
else:
    if a == b == c:
        print("Triângulo equilátero")
    elif a == b or b == c or a == c:
        print("Triângulo isósceles")
    else:
        print("Triângulo escaleno")
    if a**2 == b**2 + c**2 or b**2 == a**2 + c**2 or c**2 == a**2 + b**2:
        print("e também é retângulo")
```

```
Lado a: 3
Lado b: 4
Lado c: 5
Triângulo escaleno
e também é retângulo
```

Repare nos dois níveis de recuo: o segundo `if` está *dentro* do `else`.

1.9 Repetições com `for`

O `for` repete um bloco de comandos para cada elemento de uma sequência. A sequência mais usada é `range`, que gera inteiros:

Sintaxe

```
for variável in range(início, fim, passo):
    comandos (recuados)
```

- `range(n)` gera $0, 1, \ldots, n-1$ (são n números; o `n` **não** entra);
- `range(a, b)` gera $a, a+1, \ldots, b-1$;
- `range(a, b, p)` gera $a, a+p, a+2p, \ldots$ enquanto for menor que b (se $p < 0$, enquanto for maior que b).

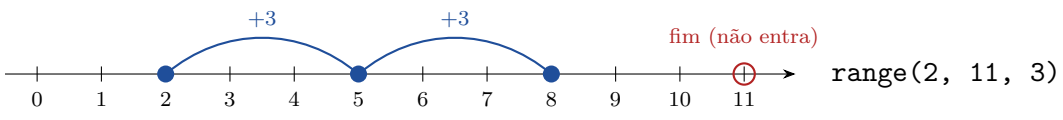


Figura 1.4: `range(2, 11, 3)` gera 2, 5, 8: começa no início, anda de 3 em 3 e para *antes* do fim.

[19]:
```
print(list(range(5)))
print(list(range(2, 11, 3)))
print(list(range(10, 0, -2)))
```

```
[0, 1, 2, 3, 4]
[2, 5, 8]
[10, 8, 6, 4, 2]
```

(O `list()` serve só para mostrar todos os números gerados; no `for` ele não é necessário.)

[20]:
```
for i in range(1, 6):
    print(i, i**2, i**3)
```

```
1 1 1
2 4 8
3 9 27
4 16 64
5 25 125
```

Acumuladores. Para calcular uma soma $\sum_{k=1}^{n} a_k$, usa-se uma variável que começa em zero e recebe cada parcela. Abaixo, verificamos a fórmula $1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$.

[21]:
```
n = 10
soma = 0                    # acumulador
for k in range(1, n + 1):
    soma = soma + k**2      # ou: soma += k**2
print("soma =", soma, "  fórmula =", n*(n + 1)*(2*n + 1)//6)
```

```
soma = 385   fórmula = 385
```

Para produtos, o acumulador começa em 1. Exemplo: o fatorial $n! = 1 \cdot 2 \cdots n$.

[22]:
```
n = 6
fat = 1
for k in range(2, n + 1):
    fat *= k
print(f"{n}! = {fat}")
```

```
6! = 720
```

O `for` também percorre os caracteres de um texto (e, na próxima aula, os elementos de uma lista):

[23]:
```
for letra in "MAT":
    print(letra, end=" ")
```

```
M A T
```

[24]:
```
# repetições encaixadas: tabuada de 1 a 3
for i in range(1, 4):
    for j in range(1, 6):
        print(f"{i*j:3d}", end="")
    print()   # pula linha ao fim de cada i
```

```
  1  2  3  4  5
  2  4  6  8 10
```

```
  3  6  9 12 15
```

1.10 Repetições com `while`

O `while` ("enquanto") repete um bloco *enquanto* uma condição for verdadeira. Usa-se quando não sabemos de antemão quantas repetições serão necessárias.

Sintaxe

```
while condição:
    comandos (recuados)
```

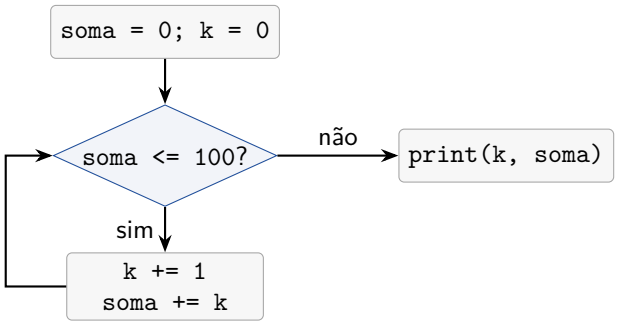


Figura 1.5: O `while` testa a condição antes de cada repetição.

Exemplo. Qual o menor k tal que $1 + 2 + \cdots + k > 100$?

[25]:
```
soma = 0
k = 0
while soma <= 100:
    k += 1
    soma += k
print("k =", k, "  soma =", soma)
```

```
k = 14   soma = 105
```

Exemplo: algoritmo de Euclides. O máximo divisor comum (mdc) satisfaz a relação $\text{mdc}(a, b) = \text{mdc}(b, a \bmod b)$ e $\text{mdc}(a, 0) = a$. Repetimos a troca enquanto $b \neq 0$:

[26]:
```
a, b = 252, 198
while b != 0:
    print(f"mdc({a}, {b})")
    a, b = b, a % b
print("mdc =", a)
```

```
mdc(252, 198)
mdc(198, 54)
mdc(54, 36)
mdc(36, 18)
mdc = 18
```

`break` e `continue`. Dentro de `for` ou `while`, `break` interrompe a repetição imediatamente e `continue` pula para a próxima volta.

[27]:
```python
# primeiro múltiplo de 7 maior que 50
n = 51
while True:           # repetição "infinita"...
    if n % 7 == 0:
        break         # ...interrompida pelo break
    n += 1
print(n)

for i in range(1, 11):
    if i % 3 == 0:
        continue      # pula os múltiplos de 3
    print(i, end=" ")
```

```
56
1 2 4 5 7 8 10
```

Atenção: laço infinito

Se a condição do `while` nunca ficar falsa, o programa não termina (a célula fica com `[*]`). Isso acontece, por exemplo, quando se esquece de atualizar a variável (`k += 1`). Para interromper, use o botão ■ ou **Kernel → Interrupt Kernel**.

Nota ao professor

Erros mais frequentes nesta aula: (i) usar vírgula decimal no `input`; (ii) esquecer a conversão `int()`/`float()`; (iii) confundir `=` com `==`; (iv) esquecer os dois-pontos; (v) misturar tabulação e espaços no recuo (raro no Jupyter); (vi) achar que `range(1,n)` inclui n. Vale pedir que a turma "preveja" a saída de algumas células antes de executá-las, por exemplo `-2**2`, `7/2`, `7//2` e `list(range(2,11,3))`.

1.11 Exercícios

Exercício 1.1 (conversão de temperatura)

Leia uma temperatura em graus Celsius com `input` e mostre o valor em Fahrenheit, $F = \frac{9}{5}C + 32$, com uma casa decimal.

Exercício 1.2 (ano bissexto)

Um ano é bissexto se for divisível por 4 e não por 100, ou se for divisível por 400. Leia um ano e diga se ele é bissexto. Teste com 1900, 2000, 2024 e 2025.

Exercício 1.3 (somatórios)

Use `for` para calcular: (a) a soma dos ímpares de 1 a 99; (b) $\sum_{k=1}^{20} \frac{1}{k^2}$ e compare com $\pi^2/6 \approx 1{,}6449$; (c) $\sum_{k=1}^{n} k^3$ para $n = 10$ e verifique que é igual a $\left(\frac{n(n+1)}{2}\right)^2$.

Exercício 1.4 (livro da SBM)

Some os números inteiros a partir de 1 até que a soma ultrapasse 5000. Imprima o último número somado e a soma final.

Exercício 1.5 (progressão aritmética)

Leia a_1, a razão r e n, e imprima os n primeiros termos da PA e a sua soma. Confira com $S_n = \frac{n(a_1+a_n)}{2}$.

Exercício 1.6 (extra: sequência de Collatz)

Comece com um inteiro $n > 0$. Se n for par, troque por $n/2$; se for ímpar, por $3n + 1$. Repita até chegar em 1. Escreva um programa que mostre a sequência e quantos passos foram necessários. Teste com $n = 6$ e $n = 27$.

Exercício 1.7 (extra: divisores)

Leia um inteiro $n > 1$, imprima todos os seus divisores e diga se ele é primo (um número é primo se tem exatamente dois divisores).

CAPÍTULO 2

Revisão de Python II: listas, tuplas e funções

Objetivos da aula

- criar e manipular listas: índices, fatiamento, `append`, `insert`, `len`, `sum`, `sorted`;
- entender a diferença entre *copiar* uma lista e dar a ela um *segundo nome*;
- percorrer listas com `for` e construí-las com *compreensão de listas*;
- usar tuplas (sequências imutáveis) e, brevemente, dicionários;
- definir funções com `def`, `return`, parâmetros com valor padrão e funções `lambda`.

Roteiro sugerido

Tempo	Atividade
0–5 min	Revisão rápida da aula anterior (`for` e `if`).
5–30 min	Listas: criação, índices, fatiamento, métodos, cópia *versus* referência.
30–40 min	Percorrendo listas; compreensão de listas.
40–50 min	Tuplas e dicionários.
50–75 min	Funções: `def`, `return`, parâmetros, escopo, `lambda`.
75–90 min	Exercícios (primos, sequências, estatística simples).

2.1 Listas

Uma **lista** é uma sequência ordenada de valores, escrita entre colchetes e separada por vírgulas. Os elementos podem ser de qualquer tipo, inclusive outras listas.

[1]:
```
notas = [9.5, 7.5, 10, 8.0]
alunos = ["Davi", "Felipe", "Henrique", "Maria"]
mista = [1, "dois", 3.0, [4, 5]]
vazia = []
print(notas)
print(len(alunos), type(alunos), len(vazia))
```

```
[9.5, 7.5, 10, 8.0]
4 <class 'list'> 0
```

2.1.1 Índices

Cada elemento ocupa uma posição (*índice*). **A contagem começa em 0**. Índices negativos contam a partir do fim: `-1` é o último, `-2` o penúltimo etc.

[2]:
```
print(alunos[0], alunos[2], alunos[-1])
alunos[1] = "Laura"        # listas podem ser alteradas
```

índice	0	1	2	3
alunos	"Davi"	"Felipe"	"Henrique"	"Maria"
índice negativo	-4	-3	-2	-1

Figura 2.1: Índices positivos e negativos da lista `alunos`.

```
print(alunos)
```

```
Davi Henrique Maria
['Davi', 'Laura', 'Henrique', 'Maria']
```

[3]:
```
alunos[4]
```

```
IndexError: list index out of range
```

Uma lista com 4 elementos tem índices de 0 a 3; pedir o índice 4 é um erro muito comum.

2.1.2 Fatiamento

A expressão `lista[i:j]` devolve a *fatia* com os elementos de índice $i, i+1, \ldots, j-1$ (como no `range`, o fim não entra). Também é possível indicar um passo: `lista[i:j:p]`. Se `i` for omitido, começa do início; se `j` for omitido, vai até o fim.

[4]:
```
v = [10, 20, 30, 40, 50, 60]
print(v[1:4])      # índices 1, 2, 3
print(v[:3])       # os 3 primeiros
print(v[3:])       # do índice 3 até o fim
print(v[-2:])      # os 2 últimos
print(v[::2])      # de 2 em 2
print(v[::-1])     # invertida
```

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
```

2.1.3 Operações e métodos

Comando	O que faz
`len(L)`, `sum(L)`, `min(L)`, `max(L)`	tamanho, soma, menor e maior elemento
`x in L`	`True` se `x` está na lista
`L + M`, `3 * L`	concatena listas; repete a lista 3 vezes
`L.append(x)`	acrescenta `x` no **fim** da lista
`L.insert(i, x)`	insere `x` na posição `i`
`L.remove(x)`	remove a primeira ocorrência de `x`
`L.pop()` / `L.pop(i)`	remove e devolve o último elemento / o de índice `i`
`L.index(x)`, `L.count(x)`	posição da primeira ocorrência de `x`; quantas vezes aparece
`L.sort()` / `sorted(L)`	ordena a própria lista / devolve uma *nova* lista ordenada
`L.reverse()`	inverte a própria lista
`L.copy()` ou `L[:]`	cria uma cópia da lista

[5]:
```
notas = [9.5, 7.5, 10, 8.0]
notas.append(7.0)
notas.insert(1, 6.5)
print(notas)
print("média:", sum(notas) / len(notas))
print("maior:", max(notas), " menor:", min(notas))
print(sorted(notas))        # nova lista ordenada
print(10 in notas, 5 in notas)
print([1, 2] + [3], 3 * [0])
```

```
[9.5, 6.5, 7.5, 10, 8.0, 7.0]
média: 8.083333333333334
maior: 10  menor: 6.5
[6.5, 7.0, 7.5, 8.0, 9.5, 10]
True False
[1, 2, 3] [0, 0, 0]
```

2.1.4 Cópia ou novo nome?

Este é um ponto que confunde muitos estudantes (e que aparece no livro da SBM, Exemplo 3.6). Ao escrever `B = A`, **não** se cria uma nova lista: `B` passa a ser apenas outro nome para a *mesma* lista. Alterar `B` altera `A`. Para obter uma lista independente, use `A.copy()` ou `A[:]`.

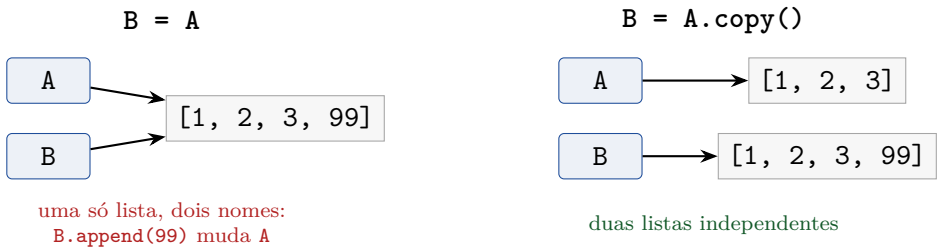


Figura 2.2: Atribuir uma lista a outro nome não a copia.

[6]:
```
A = [1, 2, 3]
B = A              # B é outro nome para a MESMA lista
B.append(99)
print("A =", A)
```

```
A = [1, 2, 3]
C = A.copy()       # C é uma lista nova, independente
C.append(99)
print("A =", A, "  C =", C)
```

```
A = [1, 2, 3, 99]
A = [1, 2, 3]   C = [1, 2, 3, 99]
```

2.2 Percorrendo e construindo listas

O `for` percorre diretamente os elementos de uma lista. Se também precisarmos do índice, usamos `enumerate`; para percorrer duas listas lado a lado, `zip`.

[7]:
```
alunos = ["Davi", "Felipe", "Henrique", "Maria"]
notas = [9.5, 7.5, 10, 8.0]
for nome in alunos:
    print(nome, end="; ")
print()
for i, nome in enumerate(alunos):
    print(i, nome)
for nome, nota in zip(alunos, notas):
    situacao = "aprovado" if nota >= 6 else "reprovado"
    print(f"{nome:10s} {nota:5.1f}  {situacao}")
```

```
Davi; Felipe; Henrique; Maria;
0 Davi
1 Felipe
2 Henrique
3 Maria
Davi         9.5  aprovado
Felipe       7.5  aprovado
Henrique    10.0  aprovado
Maria        8.0  aprovado
```

Observação

A linha `situacao = "aprovado" if nota >= 6 else "reprovado"` é um `if` "em uma linha" (expressão condicional): vale o primeiro valor se a condição for verdadeira e o segundo caso contrário.

Construindo uma lista com `append`. Começa-se com a lista vazia e acrescenta-se um elemento a cada volta. Exemplo: os 10 primeiros termos da sequência de Fibonacci, $F_1 = F_2 = 1$, $F_n = F_{n-1} + F_{n-2}$.

[8]:
```
fib = [1, 1]
for n in range(8):
    fib.append(fib[-1] + fib[-2])
print(fib)
razoes = []
for k in range(1, len(fib)):
    razoes.append(fib[k] / fib[k-1])
print(razoes[-1], (1 + 5**0.5)/2)    # converge para o número de ouro
```

```
[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
1.6176470588235294 1.618033988749895
```

Compreensão de listas. É uma forma compacta, muito parecida com a notação de conjuntos da matemática. Compare

$$\{k^2 : k \in \{1, \ldots, 10\}\}, \qquad \{k \in \{1, \ldots, 30\} : k \text{ é múltiplo de } 7\}$$

com

[9]:
```
quadrados = [k**2 for k in range(1, 11)]
multiplos7 = [k for k in range(1, 31) if k % 7 == 0]
print(quadrados)
print(multiplos7)
print(sum([1/k**2 for k in range(1, 1001)]))    # aproxima pi^2/6
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[7, 14, 21, 28]
1.6439345666815597
```

Sintaxe

```
[expressão for variável in sequência]
[expressão for variável in sequência if condição]
```

Listas de listas. Uma matriz pode ser representada como uma lista de linhas. O elemento da linha i e coluna j é `M[i][j]` (lembrando que a contagem começa em 0). Na Aula 5 veremos uma forma muito melhor de trabalhar com matrizes (NumPy).

[10]:
```
M = [[1, 2, 3],
     [4, 5, 6]]
print(M[1][2], len(M), len(M[0]))
T = [[M[i][j] for i in range(2)] for j in range(3)]   # transposta
print(T)
```

```
6 2 3
[[1, 4], [2, 5], [3, 6]]
```

2.3 Tuplas e dicionários

Uma **tupla** é parecida com uma lista, mas escrita com parênteses e **imutável**: depois de criada, não pode ser alterada. É usada para dados que "andam juntos", como as coordenadas de um ponto.

[11]:
```
P = (3, 4)
Q = (0, 1, 2)
print(P[0], P[1], len(Q), type(P))
x, y = P                    # desempacotamento
print("x =", x, " y =", y)
```

```
3 4 3 <class 'tuple'>
x = 3  y = 4
```

[12]:
```
P[0] = 10
```

```
TypeError: 'tuple' object does not support item assignment
```

Para "alterar" uma tupla, converte-se em lista com `list()` (e de volta com `tuple()`, se necessário).

[13]:
```
L = list(P)
L.append(5)
print(L, tuple(L))
```

```
[3, 4, 5] (3, 4, 5)
```

Um **dicionário** associa *chaves* a *valores*, como uma tabela. É escrito com chaves `{}` e pares `chave: valor`.

[14]:
```
notas = {"Davi": 9.5, "Felipe": 8.5, "Maria": 8.0}
print(notas["Felipe"])
notas["Laura"] = 7.0          # nova chave
for nome, nota in notas.items():
    print(nome, nota)
```

```
8.5
Davi 9.5
Felipe 8.5
Maria 8.0
Laura 7.0
```

	Lista	**Tupla**	**Dicionário**
Delimitador	`[ ]`	`( )`	`{ }`
Acesso	por índice `L[0]`	por índice `T[0]`	por chave `D["Davi"]`
Pode ser alterado?	sim	não	sim
Uso típico	coleções que mudam	pontos, pares, retornos	tabelas, cadastros

Para quem vem do Maxima

No Maxima, listas também são escritas com colchetes, `[1, 2, 3]`, mas o primeiro elemento é `L[1]`. Em Python é `L[0]`. O `makelist(k^2, k, 1, 10)` do Maxima corresponde a `[k**2 for k in range(1, 11)]`.

2.4 Funções

Uma função agrupa uma sequência de comandos sob um nome, para que possam ser reutilizados. Em matemática escrevemos $f(x) = x^2 + 1$; em Python:

[15]:
```
def f(x):
    return x**2 + 1

print(f(2), f(-1.5), f(10))
```

```
5 3.25 101
```

Sintaxe

```
def nome_da_funcao(parametro1, parametro2, ...):
    """Texto opcional explicando a função (docstring)."""
    comandos
    return resultado
```

- a primeira linha começa com `def`, tem os parâmetros entre parênteses e termina com dois-pontos;
- o corpo da função é **recuado**;
- `return` encerra a função e devolve o resultado para quem a chamou;
- a função só é executada quando é *chamada*, como em `f(2)`.

`return` não é `print`. Uma função que apenas *imprime* um valor não o devolve: o resultado não pode ser usado em outras contas.

[16]:
```
def dobro_imprime(x):
    print(2*x)

def dobro(x):
    return 2*x

a = dobro(5) + 1
b = dobro_imprime(5)
print("a =", a, "  b =", b)
```

```
10
a = 11   b = None
```

(`None` é o valor "nada", devolvido por funções sem `return`.)

Vários parâmetros e vários resultados. Uma função pode receber vários valores e devolver vários resultados (que chegam como uma tupla). Exemplo: área e perímetro de um círculo de raio r e as raízes de $ax^2 + bx + c = 0$.

[17]:
```
import math

def circulo(r):
    """Devolve a área e o perímetro de um círculo de raio r."""
    return math.pi * r**2, 2 * math.pi * r

area, perimetro = circulo(3)
print(f"área = {area:.4f}, perímetro = {perimetro:.4f}")

def raizes(a, b, c):
    delta = b**2 - 4*a*c
    if delta < 0:
        return ()                  # tupla vazia: sem raízes reais
    r1 = (-b + math.sqrt(delta)) / (2*a)
    r2 = (-b - math.sqrt(delta)) / (2*a)
    return r1, r2

print(raizes(1, -5, 6), raizes(1, 0, 1), raizes(2, 4, 2))
```

```
área = 28.2743, perímetro = 18.8496
(3.0, 2.0) () (-1.0, -1.0)
```

(O módulo `math`, que fornece π e a raiz quadrada, é o tema da próxima aula.)

Parâmetros com valor padrão. Um parâmetro pode ter um valor usado quando ele não é informado. Os argumentos também podem ser passados pelo nome.

[18]:
```
def soma_pa(a1, r, n=10):
    """Soma dos n primeiros termos da PA de primeiro termo a1 e razão r."""
    return sum([a1 + k*r for k in range(n)])

print(soma_pa(1, 1))          # n = 10 (padrão): 1 + 2 + ... + 10
print(soma_pa(1, 1, 100))     # 1 + 2 + ... + 100
print(soma_pa(r=2, a1=1, n=5))
```

```
55
5050
25
```

Variáveis locais. Variáveis criadas dentro de uma função só existem dentro dela (são *locais*). Isso evita que uma função "bagunce" as variáveis do resto do notebook.

[19]:
```
def quadrado(x):
    resultado = x * x
    return resultado

print(quadrado(7))
print("resultado" in dir())    # 'resultado' não existe fora da função
```

```
49
False
```

Funções que usam funções. Funções podem chamar outras funções e podem até ser passadas como argumento. Abaixo, `tabela` recebe uma função `g` qualquer e mostra seus valores em pontos igualmente espaçados de $[a, b]$.

[20]:
```
def tabela(g, a, b, n):
    h = (b - a) / n
    for k in range(n + 1):
        x = a + k*h
        print(f"{x:6.2f} {g(x):10.4f}")

tabela(f, 0, 1, 4)
```

```
  0.00     1.0000
  0.25     1.0625
  0.50     1.2500
  0.75     1.5625
  1.00     2.0000
```

Funções `lambda`. Para funções curtas, de uma expressão só, pode-se usar `lambda`: `g = lambda x: x**3 - x` é o mesmo que definir `g` com `def` e `return x**3 - x`.

[21]:
```
g = lambda x: x**3 - x
print(g(2))
tabela(lambda t: 1/(1 + t**2), 0, 2, 2)
```

```
6
  0.00     1.0000
  1.00     0.5000
  2.00     0.2000
```

Exemplo completo: números primos. Um inteiro $n \geq 2$ é primo se não tem divisores entre 2 e $n-1$. Na verdade basta testar divisores d com $d^2 \leq n$: se $n = d \cdot e$ com $d \leq e$, então $d^2 \leq n$.

[22]:
```
def eh_primo(n):
    if n < 2:
        return False
    d = 2
    while d * d <= n:
        if n % d == 0:
            return False      # achou um divisor: não é primo
        d += 1
    return True

print([n for n in range(2, 50) if eh_primo(n)])
print(eh_primo(97), eh_primo(2**31 - 1))
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
True True
```

Atenção

- Esquecer os parênteses na chamada: `f` é a função; `f(2)` é o valor dela em 2.
- Usar a função antes de executar a célula onde ela foi definida gera `NameError`.
- Não use como nome de variável ou função nomes já existentes, como `sum`, `list`, `max` ou `print`: o comando original deixa de funcionar até reiniciar o kernel.

Nota ao professor

O exemplo da soma dos quadrados no livro da SBM (Exemplo 3.25) tem um erro de digitação (`i + i` em vez de `i * i`) e usa `sum` como nome de variável, o que "apaga" a função `sum` do Python dentro da função. São bons pontos para discutir com a turma. A diferença entre `print` e `return` costuma gerar muitas dúvidas: vale insistir no exemplo `dobro` *versus* `dobro_imprime`.

2.5 Exercícios

Exercício 2.1 (manipulando listas)

Dada `L = [3, 8, -2, 7, 0, 5, -6, 1]`: (a) mostre o terceiro elemento e o último; (b) crie a lista dos elementos positivos; (c) crie a lista dos quadrados dos elementos; (d) calcule a média e mostre a lista ordenada em ordem decrescente.

Exercício 2.2 (soma de quadrados – livro da SBM)

Escreva uma função `soma_quadrados(n)` que devolva $1^2+2^2+\cdots+n^2$. Use-a para verificar a fórmula $\frac{n(n+1)(2n+1)}{6}$ para $n = 1, \ldots, 20$.

Exercício 2.3 (média e desvio padrão)

Escreva funções `media(L)` e `desvio(L)` que calculem a média $\bar{x}$ e o desvio padrão $s = \sqrt{\frac{1}{n}\sum(x_i - \bar{x})^2}$ de uma lista. Teste com as notas `[9.5, 7.5, 10, 8.0, 6.5]`.

Exercício 2.4 (primos gêmeos)

Usando `eh_primo`, liste todos os pares de primos gêmeos $(p, p+2)$ com $p < 100$.

Exercício 2.5 (MDC e MMC)

Escreva uma função `mdc(a, b)` usando o algoritmo de Euclides (Aula 1) e uma função `mmc(a, b)` usando $\text{mmc}(a,b) = \frac{ab}{\text{mdc}(a,b)}$. Calcule o MMC de $1, 2, \ldots, 10$.

Exercício 2.6 (extra: polinômio como lista)

Represente o polinômio $p(x) = a_0 + a_1 x + \cdots + a_n x^n$ pela lista `[a0, a1, ..., an]`. Escreva uma função `avalia(coef, x)` que calcule $p(x)$ e uma função `deriva(coef)` que devolva a lista de coeficientes de $p'(x)$. Teste com $p(x) = 1 - 3x + 2x^3$.

CAPÍTULO 3

O módulo `math`

Objetivos da aula

- entender o que é um módulo e as formas de importá-lo;
- usar as constantes e funções do módulo `math`: piso, teto, raízes, exponencial, logaritmos, trigonométricas, fatorial e combinações;
- programar a raiz quadrada pelo **método babilônico** e comparar com `math.sqrt`;
- usar `math` em aplicações: teorema de Pitágoras, lei dos cossenos, teste de primalidade e o método da bissecção.

Roteiro sugerido

Tempo	Atividade
0–10 min	Módulos: `import`, `as`, `from ... import`; ajuda com `help` e `dir`.
10–30 min	Funções do `math`: piso/teto, raízes, exp/log, trigonometria, funções inteiras.
30–40 min	Aplicação: Pitágoras e lei dos cossenos.
40–60 min	Atividade principal: raiz quadrada pelo método babilônico.
60–75 min	Teste de primalidade e método da bissecção.
75–90 min	Exercícios.

3.1 Módulos e importação

O Python "puro" só sabe fazer as operações básicas. Funções como raiz quadrada, seno ou logaritmo estão em **módulos** (bibliotecas), que precisam ser *importados*. O módulo `math` faz parte da instalação padrão do Python e contém as funções matemáticas elementares para números reais (`int` e `float`).

Há três formas usuais de importar:

[1]:
```
import math                  # 1) importa o módulo inteiro
print(math.sqrt(2), math.pi)

import math as m             # 2) importa com um apelido
print(m.sqrt(2))

from math import sqrt, pi    # 3) importa só alguns nomes
print(sqrt(2) * pi)
```

```
1.4142135623730951 3.141592653589793
1.4142135623730951
4.442882938158366
```

- Na forma 1, os nomes são usados com o prefixo `math.` (*math ponto*).
- Na forma 2, o prefixo é o apelido (`m.`). É a forma usada com NumPy (`np`) e SymPy (`sp`).
- Na forma 3, os nomes importados são usados sem prefixo.

Atenção

Existe também `from math import *`, que importa *todos* os nomes. Evite: fica difícil saber de onde veio cada função e, mais adiante, o `sin` do `math`, o do NumPy e o do SymPy vão "brigar" entre si.

Dica

Para ver todas as funções do módulo, execute `dir(math)`. Para ler a ajuda de uma função, `help(math.log)`. No Jupyter, digite `math.` e aperte **Tab** para ver a lista de opções.

[2]:
```
help(math.log)
```

```
Help on built-in function log in module math:

log(...)
    log(x, [base=math.e])
    Return the logarithm of x to the given base.

    If the base is not specified, returns the natural logarithm (base e) of
    ↪ x.
```

3.2 Constantes e funções do módulo `math`

Matemática	Python	Observação
π, e	`math.pi`, `math.e`	aproximações `float`
∞	`math.inf`	"infinito" (`float`)
$\lfloor x \rfloor$, $\lceil x \rceil$	`math.floor(x)`, `math.ceil(x)`	piso e teto (devolvem `int`)
$\lvert x \rvert$	`abs(x)` ou `math.fabs(x)`	`abs` não precisa do `math`
$\sqrt{x}$	`math.sqrt(x)`	$x \geq 0$
$\lfloor \sqrt{n} \rfloor$	`math.isqrt(n)`	raiz inteira
$\sqrt[3]{x}$	`math.cbrt(x)`	aceita $x < 0$
x^y	`x**y` ou `math.pow(x, y)`	
e^x	`math.exp(x)`	
$\ln x$	`math.log(x)`	logaritmo natural
$\log_b x$	`math.log(x, b)`	
$\log_{10} x$, $\log_2 x$	`math.log10(x)`, `math.log2(x)`	
$\sin x$, $\cos x$, $\tan x$	`math.sin(x)`, `math.cos(x)`, `math.tan(x)`	em **radianos**
$\arcsin x$, $\arccos x$, $\arctan x$	`math.asin(x)`, `math.acos(x)`,	
	`math.atan(x)`	em radianos
graus $\leftrightarrow$ radianos	`math.radians(g)`, `math.degrees(r)`	
$\sinh x$, $\cosh x$, $\tanh x$	`math.sinh(x)`, `math.cosh(x)`, `math.tanh(x)`	
$n!$	`math.factorial(n)`	n inteiro ≥ 0
$\binom{n}{k}$, $\frac{n!}{(n-k)!}$	`math.comb(n, k)`, `math.perm(n, k)`	combinações e arranjos
mdc, mmc	`math.gcd(a, b)`, `math.lcm(a, b)`	aceitam vários argumentos
$\sqrt{x^2 + y^2}$	`math.hypot(x, y)`	hipotenusa
$a \approx b$	`math.isclose(a, b)`	compara `floats`

Para quem vem do Maxima

No Maxima as funções estão sempre disponíveis: `sqrt(2)`, `log(x)`, `sin(x)`, `%pi`, `%e`. Além disso, o Maxima devolve `sqrt(2)` de forma *exata*, enquanto o `math.sqrt(2)` devolve um número aproximado. O comportamento "exato" do Maxima será obtido com o SymPy (Aulas 7 a 10). Note também que no Maxima `log` é o logaritmo natural, assim como no Python.

3.2.1 Piso, teto e arredondamentos

O **piso** $\lfloor x \rfloor$ é o maior inteiro menor ou igual a x, e o **teto** $\lceil x \rceil$ é o menor inteiro maior ou igual a x:

$$\lfloor x \rfloor = \max\{m \in \mathbb{Z} : m \le x\}, \qquad \lceil x \rceil = \min\{n \in \mathbb{Z} : n \ge x\}.$$

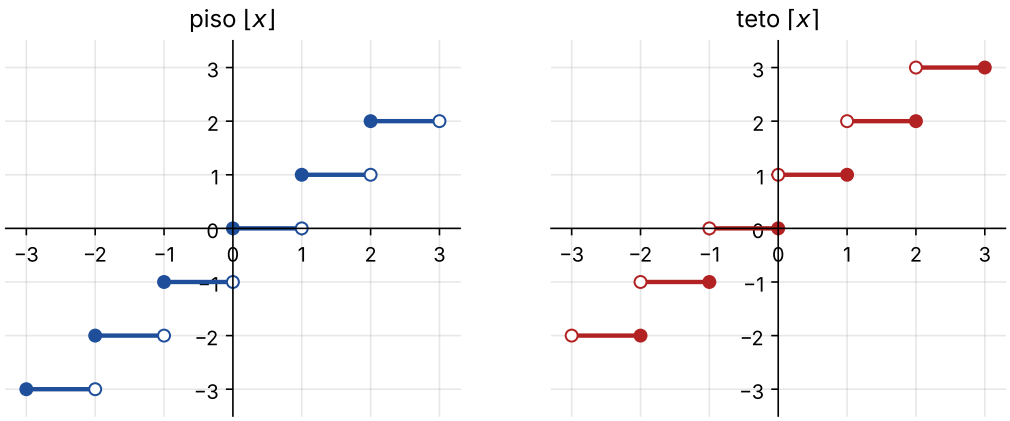


Figura 3.1: Gráficos das funções piso e teto. A bolinha cheia indica o valor da função no ponto de salto.

[3]:
```
for x in [2.1, 2.7, -2.1, -2.7, 3.0]:
    print(f"x = {x:5}  piso = {math.floor(x):3}  teto = {math.ceil(x):3}"
          f"  trunc = {math.trunc(x):3}  round = {round(x):3}")
```

```
x =   2.1  piso =   2  teto =   3  trunc =   2  round =   2
x =   2.7  piso =   2  teto =   3  trunc =   2  round =   3
x =  -2.1  piso =  -3  teto =  -2  trunc =  -2  round =  -2
x =  -2.7  piso =  -3  teto =  -2  trunc =  -2  round =  -3
x =   3.0  piso =   3  teto =   3  trunc =   3  round =   3
```

Observe que para números negativos o piso e o `trunc` (que apenas descarta a parte decimal, como `int()`) são diferentes.

Observação

O `round` do Python usa o "arredondamento do banqueiro": quando o número está exatamente no meio, arredonda para o *par* mais próximo. Assim `round(2.5)` dá `2` e `round(3.5)` dá `4`.

3.2.2 Raízes, exponenciais e logaritmos

[4]:
```
print(math.sqrt(16), math.sqrt(2), math.cbrt(-8))
print(math.isqrt(50))                  # maior inteiro k com k*k <= 50
print(math.exp(1), math.e)
print(math.log(math.e), math.log(100, 10), math.log10(0.001),
    ↪ math.log2(1024))
```

```
4.0 1.4142135623730951 -2.0
7
2.718281828459045 2.718281828459045
1.0 2.0 -3.0 10.0
```

Fora do domínio, as funções do `math` geram erro:

[5]:
```
math.sqrt(-4)
```

```
ValueError: math domain error
```

Para raízes de números negativos (resultados complexos) existe o módulo `cmath`, com as mesmas funções:

[6]:
```
import cmath
print(cmath.sqrt(-4), (-4)**0.5)
```

```
2j (1.2246467991473532e-16+2j)
```

Exemplo: juros compostos. Um capital aplicado a uma taxa de 1% ao mês dobra quando $(1{,}01)^n = 2$, isto é, $n = \dfrac{\ln 2}{\ln 1{,}01}$.

[7]:
```
n = math.log(2) / math.log(1.01)
print(f"n = {n:.2f} meses, ou seja, {math.ceil(n)} meses")
```

```
n = 69.66 meses, ou seja, 70 meses
```

3.2.3 Trigonometria

As funções trigonométricas usam o ângulo em **radianos**. Para trabalhar em graus, converta com `math.radians`.

[8]:
```
print(math.sin(math.pi/2), math.cos(math.pi))
print(math.sin(30))                    # 30 RADIANOS, não graus!
print(math.sin(math.radians(30)))      # sen(30°)
print(math.degrees(math.atan(1)))      # arctan(1) em graus
print(math.sin(math.pi))               # deveria ser 0...
```

```
1.0 -1.0
-0.9880316240928618
0.49999999999999994
45.0
1.2246467991473532e-16
```

O último valor, $1{,}22 \times 10^{-16}$, é "zero com erro de arredondamento": `math.pi` é apenas uma aproximação de π. Valores muito pequenos como esse devem ser lidos como zero.

[9]:
```
print(" graus    sen      cos      tg")
for g in [0, 30, 45, 60]:
    r = math.radians(g)
    print(f"{g:5d} {math.sin(r):8.4f} {math.cos(r):8.4f} {math.tan(r):8.4f}")
```

```
 graus    sen      cos      tg
    0   0.0000   1.0000   0.0000
   30   0.5000   0.8660   0.5774
   45   0.7071   0.7071   1.0000
   60   0.8660   0.5000   1.7321
```

3.2.4 Funções para inteiros

[10]:
```
print(math.factorial(10), math.factorial(25))
print(math.comb(60, 6))           # jogos possíveis na Mega-Sena
print(math.perm(5, 2))            # arranjos de 5 tomados 2 a 2
print(math.gcd(252, 198), math.lcm(4, 6, 10))
```

```
3628800 15511210043330985984000000
50063860
20
18 60
```

3.3 Aplicações em geometria

Teorema de Pitágoras. Em um triângulo retângulo de catetos a, b e hipotenusa c, vale $c^2 = a^2 + b^2$. O exemplo abaixo é o do livro da SBM.

[11]:
```
from math import sqrt
a = float(input("Digite o primeiro cateto: "))
b = float(input("Digite o segundo cateto: "))
c = sqrt(a**2 + b**2)          # ou math.hypot(a, b)
print("Hipotenusa:", c)
```

```
Digite o primeiro cateto: 10
Digite o segundo cateto: 11
Hipotenusa: 14.866068747318506
```

Lei dos cossenos e fórmula de Herão. Para um triângulo de lados a, b, c, o ângulo γ oposto a c satisfaz $c^2 = a^2 + b^2 - 2ab\cos\gamma$, e a área é $A = \sqrt{s(s-a)(s-b)(s-c)}$, com $s = \frac{a+b+c}{2}$.

[12]:
```
def angulos(a, b, c):
    """Ângulos (em graus) opostos aos lados a, b, c."""
    alfa = math.degrees(math.acos((b**2 + c**2 - a**2) / (2*b*c)))
    beta = math.degrees(math.acos((a**2 + c**2 - b**2) / (2*a*c)))
    gama = 180 - alfa - beta
    return alfa, beta, gama

def area_herao(a, b, c):
    s = (a + b + c) / 2
    return math.sqrt(s*(s - a)*(s - b)*(s - c))

print(angulos(3, 4, 5))
print(area_herao(3, 4, 5), area_herao(5, 5, 5))
```

```
(36.86989764584401, 53.13010235415599, 90.0)
6.0 10.825317547305483
```

3.4 Atividade: a raiz quadrada pelo método babilônico

Como uma calculadora (ou o `math.sqrt`) calcula $\sqrt{2}$? Uma ideia muito antiga, atribuída aos babilônios e descrita por Herão de Alexandria, é a seguinte.

Ideia geométrica. Queremos um *quadrado* de área x; seu lado é $\sqrt{x}$. Começamos com um *retângulo* de área x e lados y e x/y. Se y for menor que $\sqrt{x}$, então x/y é maior que $\sqrt{x}$, e vice-versa: a raiz está sempre entre os dois lados. Tomamos como novo lado a **média** dos dois, obtendo um retângulo "mais quadrado". Repetindo:

$$\boxed{y_{k+1} = \frac{1}{2}\left(y_k + \frac{x}{y_k}\right)}, \qquad y_0 > 0 \text{ qualquer.}$$

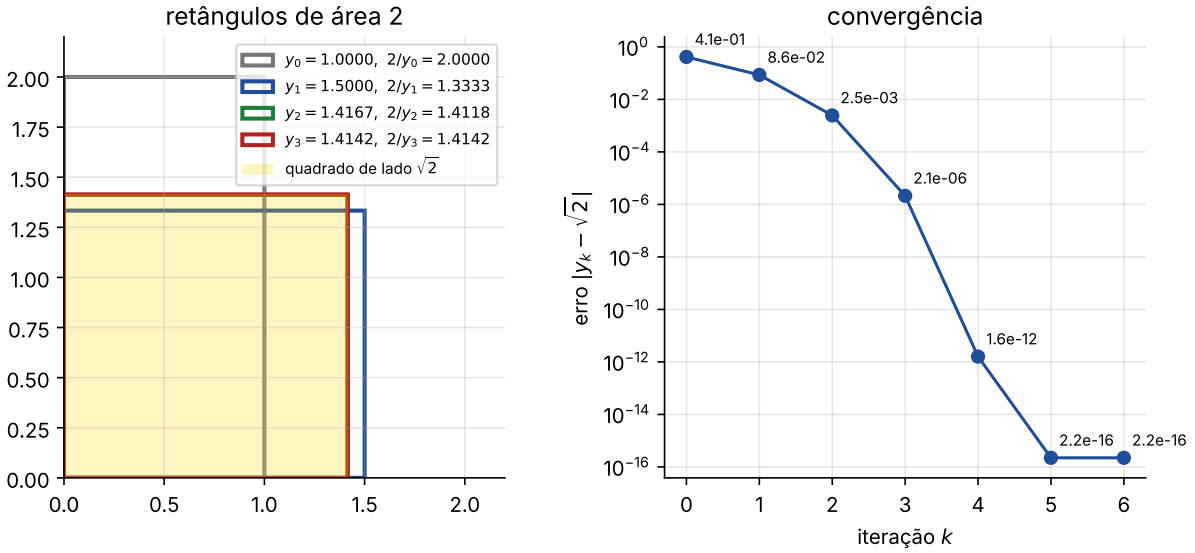


Figura 3.2: Método babilônico para $\sqrt{2}$, começando com $y_0 = 1$. À esquerda: os retângulos de área 2 e lados y_k e $2/y_k$ vão se aproximando de um quadrado. À direita: o erro $|y_k - \sqrt{2}|$ em escala logarítmica; o número de casas decimais corretas praticamente dobra a cada passo.

Primeira versão. Seguindo o tutorial do R. Miranda, fazemos um número fixo de repetições:

[13]:
```
def minharaizquadrada(x):
    y = 1
    for k in range(1, 10):       # 9 iterações
        y = (y + x/y) / 2
    return y

print(minharaizquadrada(2), math.sqrt(2))
```

```
1.414213562373095 1.4142135623730951
```

Acompanhando as iterações. Vamos imprimir cada aproximação e o seu erro.

[14]:
```
x = 2
y = 1.0
for k in range(1, 7):
    y = (y + x/y) / 2
    print(f"k = {k}   y = {y:.16f}   erro = {abs(y - math.sqrt(x)):.2e}")
```

```
k = 1   y = 1.5000000000000000   erro = 8.58e-02
k = 2   y = 1.4166666666666665   erro = 2.45e-03
k = 3   y = 1.4142156862745097   erro = 2.12e-06
k = 4   y = 1.4142135623746899   erro = 1.59e-12
k = 5   y = 1.4142135623730949   erro = 2.22e-16
k = 6   y = 1.4142135623730949   erro = 2.22e-16
```

Em apenas 5 passos chegamos ao limite de precisão do `float`. Esse método é um caso particular do **método de Newton** aplicado a $f(y) = y^2 - x$ (que será visto em Cálculo Numérico).

Segunda versão: com critério de parada. Em vez de um número fixo de passos, paramos quando duas aproximações consecutivas ficam muito próximas (diferença menor que uma tolerância). Também tratamos os casos $x < 0$ e $x = 0$.

[15]:
```
def raiz(x, tol=1e-12):
    """Raiz quadrada de x pelo método babilônico."""
    if x < 0:
        print("Erro: x deve ser não negativo")
        return None
    if x == 0:
        return 0.0
    y = x if x > 1 else 1.0          # chute inicial
    passos = 0
    while True:
        novo = (y + x/y) / 2
        passos += 1
        if abs(novo - y) < tol:
            break
        y = novo
    print(f"  ({passos} passos)")
    return novo

for valor in [2, 10, 1e6, 0.0001]:
    r = raiz(valor)
    print(f"raiz({valor}) = {r}   math.sqrt = {math.sqrt(valor)}")
```

```
  (6 passos)
raiz(2) = 1.414213562373095   math.sqrt = 1.4142135623730951
  (7 passos)
raiz(10) = 3.162277660168379   math.sqrt = 3.1622776601683795
  (16 passos)
raiz(1000000.0) = 1000.0   math.sqrt = 1000.0
  (12 passos)
raiz(0.0001) = 0.01   math.sqrt = 0.01
```

Desafio

(a) O que acontece se o chute inicial for negativo, por exemplo $y_0 = -1$?

(b) Adapte o método para a raiz cúbica, usando $y_{k+1} = \frac{1}{3}\left(2y_k + \frac{x}{y_k^2}\right)$.

(c) (Herão) Calcule $\sqrt{720}$ começando com $y_0 = 27$ e veja quantos passos são necessários.

Nota ao professor

Sugestão: deixar a turma escrever a primeira versão sozinha (10 min) a partir da fórmula no quadro, e só depois mostrar a tabela de erros. Pergunta para a turma: "por que o erro para de diminuir em $2{,}22 \times 10^{-16}$?" (resposta: é a precisão do `float`, cerca de 16 algarismos). Para o item (a) do desafio, a sequência converge para $-\sqrt{x}$.

3.5 Aplicação: teste de primalidade

Na Aula 2 vimos que, para testar se n é primo, basta procurar divisores d com $d \leq \sqrt{n}$. No tutorial do R. Miranda isso é feito com `range(3, int(math.sqrt(n)), 2)`. Vamos testar essa versão:

[16]:
```
def checaprimo2(n):
    if n == 2:
        return True
    if n % 2 == 0 or n <= 1:
        return False
    for divisor in range(3, int(math.sqrt(n)), 2):
        if n % divisor == 0:
            return False
    return True

print([n for n in range(2, 60) if checaprimo2(n)])
```

```
[2, 3, 5, 7, 9, 11, 13, 15, 17, 19, 23, 25, 29, 31, 35, 37, 41, 43, 47, 49,
    ↪ 53, 59]
```

Há um erro: 9, 15, 25, 35 e 49 aparecem como primos! Como o `range` **não inclui o fim**, o maior divisor candidato nunca é testado: para $n = 15$, `int(math.sqrt(15))` vale 3 e `range(3, 3, 2)` é vazio; para $n = 49$, o divisor 7 fica de fora. A correção é ir até `math.isqrt(n) + 1`:

[17]:
```
def eh_primo(n):
    if n == 2:
        return True
    if n % 2 == 0 or n <= 1:
        return False
    for divisor in range(3, math.isqrt(n) + 1, 2):
        if n % divisor == 0:
            return False
    return True

primos = [n for n in range(2, 60) if eh_primo(n)]
print(primos)
print(len([n for n in range(2, 10**5) if eh_primo(n)]), "primos menores que
    ↪ 100000")
```

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59]
9592 primos menores que 100000
```

Observação

O Teorema dos Números Primos diz que a quantidade de primos menores que N é aproximadamente $N/\ln N$. Para $N = 10^5$, `1e5 / math.log(1e5)` ≈ 8686; compare com 9592.

3.6 Aplicação: o método da bissecção

O Teorema do Valor Intermediário garante: se f é contínua em $[a, b]$ e $f(a)$ e $f(b)$ têm sinais opostos, então existe $c \in (a, b)$ com $f(c) = 0$. O **método da bissecção** usa isso repetidamente: calcula o ponto médio $m = \frac{a+b}{2}$ e fica com a metade do intervalo onde a troca de sinal acontece. A cada passo, o comprimento do intervalo cai pela metade.

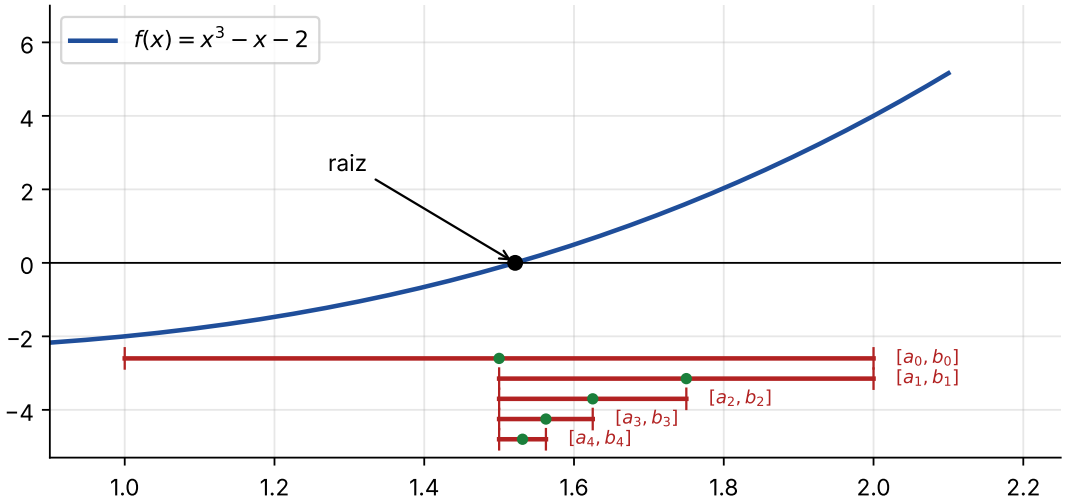


Figura 3.3: Bissecção para $f(x) = x^3 - x - 2$ em $[1, 2]$: os intervalos $[a_k, b_k]$ encolhem em torno da raiz.

[18]:

```
def bisseccao(f, a, b, tol=1e-10):
    if f(a) * f(b) > 0:
        print("f(a) e f(b) têm o mesmo sinal: escolha outro intervalo.")
        return None
    n = 0
    while (b - a) / 2 > tol:
        m = (a + b) / 2
        if f(m) == 0:
            return m
        if f(a) * f(m) < 0:       # a raiz está em [a, m]
            b = m
        else:                     # a raiz está em [m, b]
            a = m
        n += 1
    print(f"  {n} iterações")
    return (a + b) / 2

f = lambda x: x**3 - x - 2
r = bisseccao(f, 1, 2)
print("raiz ~", r, "  f(raiz) =", f(r))
```

```
g = lambda x: math.cos(x) - x
print("cos(x) = x  para x ~", bisseccao(g, 0, 1))
```

```
  33 iterações
raiz ~ 1.5213797068572603   f(raiz) = 3.131943593359665e-10
  33 iterações
cos(x) = x  para x ~ 0.7390851332456805
```

Como o intervalo é dividido por 2 a cada passo, depois de n passos o erro é no máximo $\frac{b-a}{2^{n+1}}$. Para garantir erro menor que 10^{-10} partindo de $[1,2]$, são necessários $n \geq \log_2(10^{10}) - 1 \approx 32{,}2$ passos, o que confere com o programa.

3.7 Exercícios

Exercício 3.1 (distância e ponto médio)

Escreva uma função `distancia(P, Q)` que receba dois pontos do plano como tuplas e devolva a distância entre eles. Use-a para calcular o perímetro do triângulo de vértices $(0,0)$, $(4,0)$ e $(1,3)$.

Exercício 3.2 (ângulo entre ponteiros)

Às h horas e m minutos, o ponteiro das horas está a $30h + 0{,}5m$ graus da posição das 12h e o dos minutos a $6m$ graus. Escreva uma função que devolva o menor ângulo entre os ponteiros. Teste com 3h00, 9h45 e 12h30.

Exercício 3.3 (aproximando e e π)

(a) Calcule $\sum_{k=0}^{n} \frac{1}{k!}$ para $n = 5, 10, 15$ e compare com `math.e`. (b) Calcule a série de Leibniz $4\sum_{k=0}^{n} \frac{(-1)^k}{2k+1}$ para $n = 10, 1000, 100000$ e compare com `math.pi`. Qual das séries converge mais rápido?

Exercício 3.4 (série de Taylor do seno)

Escreva uma função `seno(x, n)` que calcule $\displaystyle\sum_{k=0}^{n} \frac{(-1)^k x^{2k+1}}{(2k+1)!}$ e compare com `math.sin(x)` para $x = 1$ e $x = 10$, com $n = 5$ e $n = 20$.

Exercício 3.5 (raiz cúbica)

Adapte a função `raiz` para calcular $\sqrt[3]{x}$ com a iteração $y_{k+1} = \frac{1}{3}\left(2y_k + \frac{x}{y_k^2}\right)$ e compare com `math.cbrt`.

Exercício 3.6 (extra: bissecção)

Use a bissecção para encontrar: (a) a raiz positiva de $x^2 - 2$ (compare com $\sqrt{2}$); (b) a solução de $e^{-x} = x$; (c) a raiz de $x^7 + x - 1$ em $[0,1]$ (exemplo do tutorial do R. Miranda).

CAPÍTULO 4

NumPy I: vetores (*arrays*) e cálculo numérico

Objetivos da aula

- entender a diferença entre uma lista do Python e um *array* do NumPy;
- criar *arrays* com `np.array`, `np.zeros`, `np.ones`, `np.arange` e `np.linspace`;
- fazer operações elemento a elemento e aplicar funções (`np.sin`, `np.exp`, ...) a vetores inteiros;
- selecionar elementos com índices, fatias e máscaras lógicas;
- usar somas, médias, máximos e somas acumuladas; aplicar a regra dos trapézios para integrais.

Roteiro sugerido

Tempo	Atividade
0–10 min	Por que NumPy? Lista *versus* *array*.
10–25 min	Criando *arrays*: `array`, `zeros`, `ones`, `arange`, `linspace`.
25–40 min	Operações elemento a elemento e funções universais; tabela de valores de uma função.
40–55 min	Índices, fatias e máscaras; `sum`, `mean`, `max`, `argmax`, `cumsum`.
55–75 min	Aplicações: séries, regra dos trapézios, Monte Carlo.
75–90 min	Exercícios.

4.1 Por que NumPy?

O NumPy (*Numerical Python*) é a biblioteca básica de computação científica em Python. Seu objeto principal é o ***array*** (vetor n-dimensional): uma sequência de números *do mesmo tipo* com a qual se pode fazer contas como se fosse um vetor da matemática. Por convenção, importa-se com o apelido `np`:

[1]:
```
import numpy as np

L = [1, 2, 3, 4]
v = np.array([1, 2, 3, 4])
print(L * 2)          # lista: repete
print(v * 2)          # array: multiplica cada elemento
print(L + [10])       # lista: concatena
print(v + 10)         # array: soma 10 a cada elemento
```

```
[1, 2, 3, 4, 1, 2, 3, 4]
[2 4 6 8]
[1, 2, 3, 4, 10]
[11 12 13 14]
```

Repare que o `print` de um *array* não tem vírgulas. Além de mais natural para a matemática, o NumPy é muito mais rápido que as listas para grandes quantidades de números, pois as contas são feitas em código compilado.

Dica

Se `import numpy` gerar `ModuleNotFoundError`, instale com `%pip install numpy` em uma célula do Jupyter e reinicie o kernel.

4.2 Criando *arrays*

Comando	Resultado
`np.array([1, 2, 3])`	a partir de uma lista
`np.zeros(n)`, `np.ones(n)`	n zeros / n uns (`float`)
`np.full(n, c)`	n cópias do valor c
`np.arange(a, b, h)`	$a, a+h, a+2h, \ldots$ (menores que b), como o `range`
`np.linspace(a, b, n)`	n pontos igualmente espaçados de a até b (**inclui** b)

[2]:
```
print(np.zeros(3), np.ones(4), np.full(3, 7))
print(np.arange(5))
print(np.arange(2, 9, 2))
print(np.arange(0, 1, 0.25))
print(np.linspace(0, 1, 5))
print(np.linspace(0, 10, num=5))
```

```
[0. 0. 0.] [1. 1. 1. 1.] [7 7 7]
[0 1 2 3 4]
[2 4 6 8]
[0.   0.25 0.5  0.75]
[0.   0.25 0.5  0.75 1.  ]
[ 0.   2.5  5.   7.5 10. ]
```

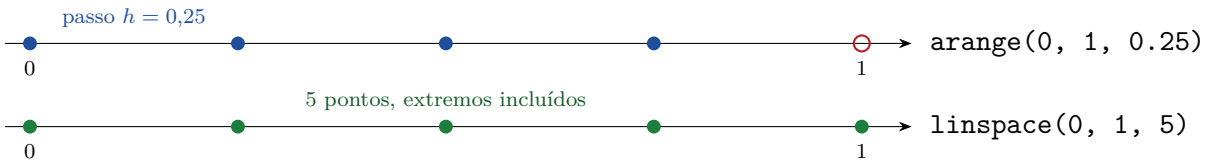


Figura 4.1: `arange` usa o *passo* e exclui o fim; `linspace` usa a *quantidade de pontos* e inclui o fim.

Dica

Para dividir um intervalo $[a, b]$ em pontos (para gráficos, tabelas e integrais), prefira `np.linspace`: com passos decimais o `arange` pode incluir ou não o último ponto por causa de arredondamentos.

Todo *array* tem um tipo (`dtype`), uma forma (`shape`) e um tamanho (`size`):

[3]:
```
v = np.array([1, 2, 3])
w = np.array([1, 2.5, 3])
print(v.dtype, w.dtype, v.shape, v.size, v.ndim)
```

```
print(np.array([1, 2, 3], dtype=float))
```

```
int64 float64 (3,) 3 1
[1. 2. 3.]
```

4.3 Operações elemento a elemento

As operações +, −, *, /, ** entre *arrays* do mesmo tamanho são feitas **posição por posição**. Entre um *array* e um número, o número é operado com cada elemento.

[4]:
```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
print(u + v, u - v, u * v, v / u, u ** 2, 2 ** u)
print(np.dot(u, v), u @ v)       # produto escalar: 1*4 + 2*5 + 3*6
print(np.linalg.norm(u))         # norma: sqrt(1 + 4 + 9)
```

```
[5 7 9] [-3 -3 -3] [ 4 10 18] [4.  2.5 2. ] [1 4 9] [2 4 8]
32 32
3.7416573867739413
```

Atenção

`u * v` **não** é o produto escalar: é o vetor dos produtos (u_1v_1, u_2v_2, u_3v_3). O produto escalar é `np.dot(u, v)` ou `u @ v`.

4.3.1 Funções universais

O NumPy tem suas próprias versões das funções matemáticas, que funcionam com *arrays* inteiros: `np.sqrt`, `np.exp`, `np.log`, `np.log10`, `np.sin`, `np.cos`, `np.tan`, `np.arcsin`, `np.arctan`, `np.abs`, `np.floor`, `np.ceil`, além das constantes `np.pi` e `np.e`. Isso permite escrever uma função matemática e aplicá-la a muitos pontos de uma vez.

[5]:
```
x = np.linspace(0, np.pi, 5)
print(x)
print(np.sin(x))
print(np.round(np.sin(x), 4))
```

```
[0.         0.78539816 1.57079633 2.35619449 3.14159265]
[0.00000000e+00 7.07106781e-01 1.00000000e+00 7.07106781e-01
 1.22464680e-16]
[0.     0.7071 1.     0.7071 0.    ]
```

[6]:
```
import math
math.sin(x)
```

```
TypeError: only 0-dimensional arrays can be converted to Python scalars
```

As funções do `math` só aceitam *um* número; para *arrays*, use as do NumPy.

Tabela de valores. Definindo uma função com operações do NumPy, ela pode receber um *array*:

[7]:
```
def f(x):
    return x**2 * np.exp(-x)

x = np.linspace(0, 4, 9)
y = f(x)
for xi, yi in zip(x, y):
    print(f"{xi:5.2f}  {yi:8.5f}")
```

```
 0.00   0.00000
 0.50   0.15163
 1.00   0.36788
 1.50   0.50204
 2.00   0.54134
 2.50   0.51303
 3.00   0.44808
 3.50   0.36992
 4.00   0.29305
```

Esse é exatamente o processo usado para fazer gráficos (Aula 6): calcula-se f em muitos pontos e ligam-se os pontos.

4.4 Índices, fatias e máscaras

Índices e fatias funcionam como nas listas (começando em 0). A novidade são as **máscaras**: uma condição aplicada a um *array* produz um *array* de `True`/`False`, que pode ser usado para selecionar elementos.

[8]:
```
v = np.array([3, -1, 4, -1, 5, -9, 2, 6])
print(v[0], v[-1], v[2:5], v[::2])
print(v > 0)
print(v[v > 0])                 # só os positivos
print(np.sum(v > 0))            # quantos são positivos (True conta 1)
v[v < 0] = 0                    # zera os negativos
print(v)
print(np.where(v > 3, 1, -1))   # 1 onde v > 3, -1 nos outros
```

```
3 6 [ 4 -1  5] [3 4 5 2]
[ True False  True False  True False  True  True]
[3 4 5 2 6]
5
[3 0 4 0 5 0 2 6]
[-1 -1  1 -1  1 -1 -1  1]
```

Condições podem ser combinadas com `&` (“e”) e `|` (“ou”), **sempre entre parênteses**:

[9]:
```
n = np.arange(1, 31)
print(n[(n % 2 == 0) & (n % 3 == 0)])    # múltiplos de 2 e de 3
print(n[(n < 4) | (n > 27)])
```

```
[ 6 12 18 24 30]
[ 1  2  3 28 29 30]
```

Atenção

Com *arrays* não se usa `and`/`or`, e sim `&`/`|` com parênteses em cada condição.

4.5 Somas, médias e outras reduções

Comando	Resultado
`np.sum(v)`, `np.prod(v)`	soma e produto de todos os elementos
`np.mean(v)`, `np.std(v)`, `np.median(v)`	média, desvio padrão, mediana
`np.min(v)`, `np.max(v)`	menor e maior valor
`np.argmin(v)`, `np.argmax(v)`	*índice* do menor e do maior valor
`np.cumsum(v)`	somas acumuladas $v_0,\ v_0+v_1,\ v_0+v_1+v_2, \ldots$
`np.diff(v)`	diferenças $v_1-v_0,\ v_2-v_1, \ldots$
`np.sort(v)`	cópia ordenada

Também se pode escrever `v.sum()`, `v.mean()`, `v.max()` etc.

[10]:
```
notas = np.array([9.5, 7.5, 10, 8.0, 6.5])
print(notas.mean(), notas.std(), np.median(notas))
print(notas.max(), notas.argmax())
k = np.arange(1, 11)
print(np.cumsum(k))
print(np.diff(k**2))           # diferenças entre quadrados consecutivos
```

```
8.3 1.2884098726725126 8.0
10.0 2
[ 1  3  6 10 15 21 28 36 45 55]
[ 3  5  7  9 11 13 15 17 19]
```

Máximo de uma função (aproximado). Com um *array* de muitos pontos, `argmax` dá o ponto onde f é maior. Para $f(x) = x^2 e^{-x}$ em $[0, 4]$, o máximo exato ocorre em $x = 2$ (Aula 8):

[11]:
```
x = np.linspace(0, 4, 4001)
y = f(x)
i = np.argmax(y)
print(f"máximo aproximado em x = {x[i]:.4f}, f(x) = {y[i]:.6f}")
```

```
máximo aproximado em x = 2.0000, f(x) = 0.541341
```

4.6 Aplicações

4.6.1 Somas de séries sem laços

Com *arrays*, uma soma $\sum_{k=1}^{n} a_k$ se escreve em uma linha, sem `for`:

[12]:
```
k = np.arange(1, 100001)
print(np.sum(1 / k**2), np.pi**2 / 6)              # série de Basileia
j = np.arange(0, 100000)
print(4 * np.sum((-1)**j / (2*j + 1)), np.pi)       # série de Leibniz
```

```
1.6449240668982266 1.6449340668482264
3.1415826535897926 3.141592653589793
```

As somas parciais (para ver a convergência) saem de `np.cumsum`:

[13]:
```
parciais = np.cumsum(1 / k**2)
for n in [10, 100, 1000, 10000, 100000]:
```

```
    print(n, parciais[n-1], np.pi**2/6 - parciais[n-1])
```

```
10 1.5497677311665408 0.09516633568168564
100 1.6349839001848923 0.009950166663334148
1000 1.6439345666815615 0.0009995001666649461
10000 1.6448340718480652 9.999500016122376e-05
100000 1.6449240668982423 9.999949984074163e-06
```

O erro se comporta como $1/n$, o que se prova comparando a série com a integral $\int_n^\infty \frac{dx}{x^2}$.

4.6.2 Integrais pela regra dos trapézios

Para aproximar $\int_a^b f(x)\,dx$, dividimos $[a,b]$ em n subintervalos de largura $h = \frac{b-a}{n}$, com pontos $x_0 = a, x_1, \ldots, x_n = b$, e somamos as áreas dos trapézios:

$$\int_a^b f(x)\,dx \approx h\left[\frac{f(x_0)}{2} + f(x_1) + \cdots + f(x_{n-1}) + \frac{f(x_n)}{2}\right].$$

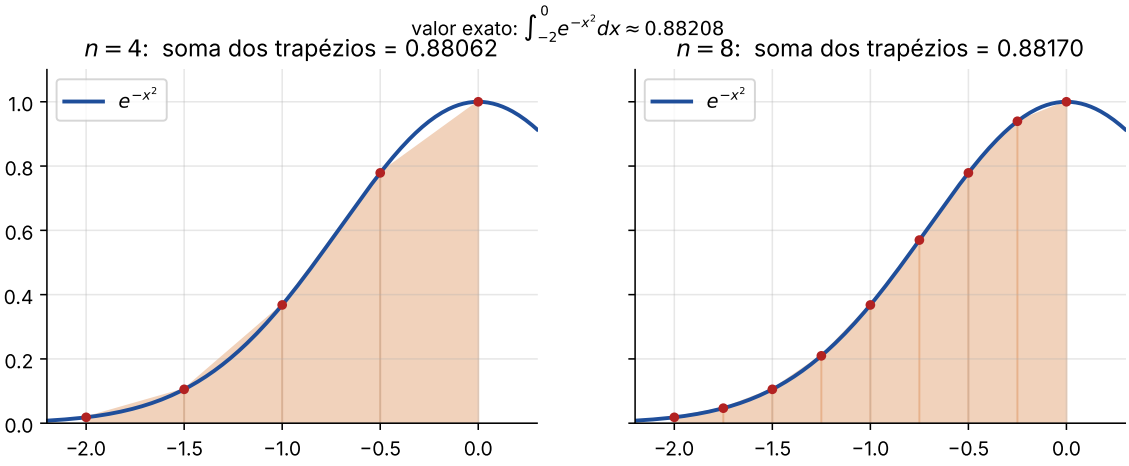


Figura 4.2: Regra dos trapézios para $f(x) = e^{-x^2}$ em $[-2, 0]$ com $n = 4$ e $n = 8$ subintervalos.

Seguindo o livro da SBM, calculamos $\int_{-\infty}^{0} e^{-x^2}\,dx = \frac{\sqrt{\pi}}{2}$, aproximando $-\infty$ por -5 (pois $e^{-25} \approx 10^{-11}$):

[14]:
```
def trapezios(f, a, b, n=100):
    x = np.linspace(a, b, n + 1)
    h = x[1] - x[0]
    y = f(x)
    return h * (np.sum(y) - (y[0] + y[-1]) / 2)

g = lambda x: np.exp(-x**2)
exato = np.sqrt(np.pi) / 2
for n in [10, 100, 1000]:
    I = trapezios(g, -5, 0, n)
    print(f"n = {n:5d}   I = {I:.12f}   erro = {abs(I - exato):.2e}")
```

```
n =    10   I = 0.886226925449   erro = 3.51e-12
n =   100   I = 0.886226925451   erro = 1.39e-12
n =  1000   I = 0.886226925451   erro = 1.38e-12
```

Já com $n = 10$ o resultado é excelente (isso acontece com funções muito "suaves" que se anulam nas pontas). O erro que sobra, da ordem de 10^{-12}, vem de termos trocado $-\infty$ por -5: a área desprezada é $\int_{-\infty}^{-5} e^{-x^2} dx \approx 1{,}4 \times 10^{-12}$. Um exemplo mais típico é $\int_0^1 \frac{4}{1+x^2}\, dx = \pi$:

[15]:
```
for n in [10, 20, 40, 80]:
    I = trapezios(lambda x: 4/(1 + x**2), 0, 1, n)
    print(f"n = {n:3d}   I = {I:.10f}   erro = {abs(I - np.pi):.2e}")
```

```
n =  10   I = 3.1399259889   erro = 1.67e-03
n =  20   I = 3.1411759870   erro = 4.17e-04
n =  40   I = 3.1414884869   erro = 1.04e-04
n =  80   I = 3.1415666119   erro = 2.60e-05
```

Cada vez que n dobra, o erro cai para $\frac{1}{4}$: o erro da regra dos trapézios é proporcional a h^2.

Observação

O NumPy já tem a regra dos trapézios pronta: `np.trapezoid(y, x)` (em versões antigas, `np.trapz`).

4.6.3 Números aleatórios: estimando π por Monte Carlo

Sorteando pontos (x, y) uniformemente no quadrado $[0, 1] \times [0, 1]$, a proporção dos que caem dentro do quarto de círculo $x^2 + y^2 \leq 1$ se aproxima da razão entre as áreas, $\frac{\pi/4}{1}$.

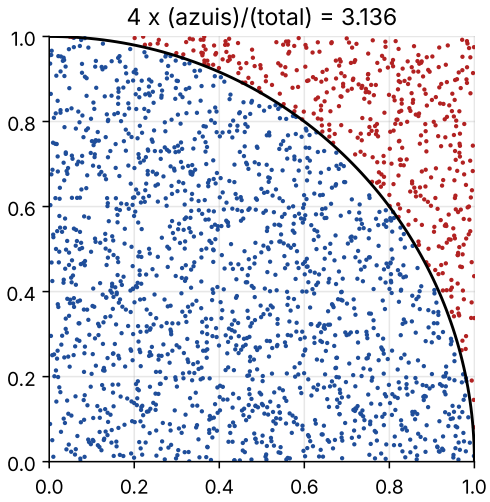


Figura 4.3: 2000 pontos sorteados: em azul, os que caem dentro do quarto de círculo.

[16]:
```
rng = np.random.default_rng(2024)      # gerador com "semente" fixa
for n in [100, 10_000, 1_000_000]:
    x = rng.random(n)
    y = rng.random(n)
    dentro = np.sum(x**2 + y**2 <= 1)
    print(f"n = {n:8d}   pi ~ {4 * dentro / n:.5f}")
```

```
n =      100   pi ~ 3.36000
n =    10000   pi ~ 3.13760
n =  1000000   pi ~ 3.14259
```

(Com outra semente os valores mudam; o erro típico é da ordem de $1/\sqrt{n}$.)

4.6.4 Notação científica

[17]:
```
c = (12 - 3)**100 * 100**100
print(np.format_float_scientific(c, precision=4))
print(f"{float(c):.4e}")
```

```
2.6561e+295
2.6561e+295
```

Nota ao professor

O exemplo de Monte Carlo é uma boa oportunidade para discutir o que é uma "semente" e por que resultados aleatórios variam. Se o tempo permitir, peça aos alunos que troquem a semente e comparem os resultados entre si.

4.7 Exercícios

Exercício 4.1 (criando arrays)

Crie, sem usar `for`: (a) os 20 primeiros ímpares; (b) 11 pontos igualmente espaçados em $[-1, 1]$; (c) as potências $2^0, 2^1, \ldots, 2^{10}$; (d) o vetor $\left(1, \frac{1}{2}, \frac{1}{3}, \ldots, \frac{1}{10}\right)$.

Exercício 4.2 (tabela)

Faça uma tabela de $\sin x$, $\cos x$ e $\sin^2 x + \cos^2 x$ para 7 pontos igualmente espaçados em $[0, \pi]$.

Exercício 4.3 (estatística)

Dadas as alturas (em m) `h = np.array([1.72, 1.65, 1.80, 1.58, 1.75, 1.69, 1.83, 1.61])`: calcule a média, o desvio padrão, quantas pessoas têm mais de 1,70 m e a média apenas dessas pessoas.

Exercício 4.4 (série harmônica)

Usando `np.cumsum`, encontre o menor n tal que $1 + \frac{1}{2} + \cdots + \frac{1}{n} > 10$. Compare $\sum_{k=1}^{n} \frac{1}{k} - \ln n$ para $n = 10^6$ com a constante de Euler–Mascheroni $\gamma \approx 0{,}5772$.

Exercício 4.5 (regra de Simpson)

A regra de Simpson (com n par) é

$$\int_a^b f \approx \frac{h}{3}\left[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n)\right].$$

Implemente-a usando fatias (`y[1:-1:2]` e `y[2:-1:2]`) e compare com os trapézios em $\int_0^1 \frac{4}{1+x^2}\,dx$ para $n = 10, 20, 40$.

Exercício 4.6 (extra: Monte Carlo para integrais)

Se $x_1, \ldots, x_n$ são sorteados uniformemente em $[0, 1]$, então $\frac{1}{n} \sum f(x_i) \approx \int_0^1 f$. Use isso para estimar $\int_0^1 e^{-x^2} dx$ e compare com a regra dos trapézios.

CAPÍTULO 5

NumPy II: matrizes, sistemas lineares e polinômios

Objetivos da aula

- criar matrizes como *arrays* bidimensionais e acessar linhas, colunas e submatrizes;
- distinguir o produto elemento a elemento (`*`) do produto de matrizes (`@`);
- calcular transposta, determinante, inversa, posto, autovalores e autovetores com `np.linalg`;
- resolver sistemas lineares com `np.linalg.solve` e programar a eliminação de Gauss;
- trabalhar com polinômios: raízes, avaliação e ajuste de curvas (`np.roots`, `np.polyval`, `np.polyfit`).

Roteiro sugerido

Tempo	Atividade
0–15 min	Matrizes no NumPy: criação, `shape`, índices, linhas, colunas, matrizes especiais.
15–30 min	Operações: soma, `*` *versus* `@`, transposta, potências.
30–45 min	`np.linalg`: `det`, `inv`, `solve`, `matrix_rank`; sistemas lineares.
45–60 min	Eliminação de Gauss programada (exercício do livro da SBM).
60–75 min	Autovalores e autovetores; polinômios e ajuste por mínimos quadrados.
75–90 min	Exercícios.

5.1 Matrizes como *arrays* bidimensionais

Uma matriz é criada a partir de uma lista de linhas:

[1]:
```
import numpy as np
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A)
print("forma:", A.shape, " elementos:", A.size, " dimensões:", A.ndim)
```

```
[[1 2 3]
 [4 5 6]]
forma: (2, 3)  elementos: 6  dimensões: 2
```

O elemento da linha i e coluna j é `A[i, j]` (contando a partir de 0). O símbolo `:` sozinho significa "todos": `A[i, :]` é a linha i e `A[:, j]` é a coluna j.

[2]:
```
M = np.arange(1, 10).reshape(3, 3)   # 1..9 arrumados em 3 x 3
print(M)
print(M[1, 1], M[0, 2], M[-1, -1])
print(M[1, :], M[:, 2])
```

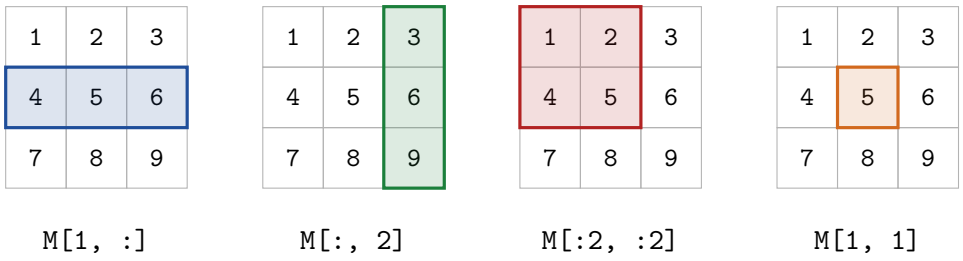


Figura 5.1: Seleção de linha, coluna, submatriz e elemento em `M = np.arange(1, 10).reshape(3, 3)`.

```
print(M[:2, :2])
```

```
 [[1 2 3]
  [4 5 6]
  [7 8 9]]
5 3 9
[4 5 6] [3 6 9]
[[1 2]
 [4 5]]
```

Para quem vem do Maxima

No Maxima: `M: matrix([1,2,3],[4,5,6],[7,8,9])`, o elemento é `M[2,2]` (contando a partir de 1), a linha é `row(M,2)` e a coluna `col(M,3)`. No NumPy a contagem começa em 0: `M[1, 1]`, `M[1, :]`, `M[:, 2]`.

Matrizes especiais.

[3]:
```
print(np.eye(3))                    # identidade
print(np.zeros((2, 3)))             # nula 2 x 3 (note os parênteses duplos)
print(np.diag([1, 2, 3]))           # diagonal
print(np.ones((2, 2), dtype=int))
```

```
 [[1. 0. 0.]
  [0. 1. 0.]
  [0. 0. 1.]]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1 0 0]
 [0 2 0]
 [0 0 3]]
[[1 1]
 [1 1]]
```

Também podemos construir uma matriz a partir de uma fórmula para a_{ij}. Por exemplo, a matriz de Hilbert $h_{ij} = \frac{1}{i+j-1}$ (com i, j começando em 1):

[4]:
```
n = 4
i = np.arange(1, n + 1).reshape(n, 1)     # coluna
j = np.arange(1, n + 1).reshape(1, n)     # linha
H = 1 / (i + j - 1)                       # "broadcasting": vira n x n
print(np.round(H, 4))
```

```
[[1.     0.5    0.3333 0.25  ]
 [0.5    0.3333 0.25   0.2   ]
 [0.3333 0.25   0.2    0.1667]
 [0.25   0.2    0.1667 0.1429]]
```

5.2 Operações com matrizes

Matemática	NumPy	Observação
$A+B$, $A-B$, kA	`A + B`, `A - B`, `k * A`	
AB (produto de matrizes)	`A @ B` ou `np.dot(A, B)`	**não** é `A * B`
A^T	`A.T` ou `np.transpose(A)`	
A^n	`np.linalg.matrix_power(A, n)`	**não** é `A**n`
$\det A$	`np.linalg.det(A)`	resultado aproximado
A^{-1}	`np.linalg.inv(A)`	
posto(A)	`np.linalg.matrix_rank(A)`	
$\mathrm{tr}(A)$	`np.trace(A)`	
solução de $Ax=b$	`np.linalg.solve(A, b)`	A quadrada e invertível
autovalores/autovetores	`np.linalg.eig(A)`	

[5]:
```
A = np.array([[1, 2], [-3, 1]])
B = np.array([[2, 5], [4, 10]])
print("A * B =\n", A * B)      # elemento a elemento
print("A @ B =\n", A @ B)      # produto de matrizes
print("B @ A =\n", B @ A)      # em geral AB != BA
print("A^T =\n", A.T)
```

```
A * B =
 [[  2  10]
 [-12  10]]
A @ B =
 [[10 25]
 [-2 -5]]
B @ A =
 [[-13   9]
 [-26  18]]
A^T =
 [[ 1 -3]
 [ 2  1]]
```

Atenção

`A * B` e `A ** 2` operam **elemento a elemento**. Para o produto matricial use `A @ B`; para potências, `A @ A` ou `np.linalg.matrix_power(A, n)`. Multiplicar matrizes de tamanhos incompatíveis gera `ValueError`.

[6]:
```
print(np.linalg.det(A), np.linalg.det(B))
Ainv = np.linalg.inv(A)
print(Ainv)
print(A @ Ainv)
print(np.allclose(A @ Ainv, np.eye(2)))
```

```
7.000000000000001 0.0
[[ 0.14285714 -0.28571429]
 [ 0.42857143  0.14285714]]
[[1. 0.]
 [0. 1.]]
True
```

Note o pequeno erro de arredondamento: $\det A = 7$, mas apareceu `7.000000000000001`. Erros desse tipo (da ordem de 10^{-16}) podem aparecer em qualquer conta com `float`; por isso, para comparar matrizes, usa-se `np.allclose`, que admite uma pequena tolerância. Os cálculos *exatos* (como no Maxima) serão feitos com o SymPy na Aula 10.

[7]:
```
np.linalg.inv(B)            # det(B) = 0
```

```
LinAlgError: Singular matrix
```

Exemplo: Fibonacci por potências de matriz. Como $\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$, temos $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$.

[8]:
```
Q = np.array([[1, 1], [1, 0]])
print(np.linalg.matrix_power(Q, 10))
```

```
[[89 55]
 [55 34]]
```

5.3 Sistemas lineares

O sistema

$$\begin{cases} x_0 + 2x_1 = 1 \\ 3x_0 + 5x_1 = 2 \end{cases} \iff \underbrace{\begin{pmatrix} 1 & 2 \\ 3 & 5 \end{pmatrix}}_{A} \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \underbrace{\begin{pmatrix} 1 \\ 2 \end{pmatrix}}_{b}$$

(exemplo do livro da SBM) se resolve com `np.linalg.solve(A, b)`. Geometricamente, a solução é o ponto de encontro das duas retas (Figura 5.2).

[9]:
```
A = np.array([[1, 2], [3, 5]])
b = np.array([1, 2])
x = np.linalg.solve(A, b)
print(x)
print(np.allclose(A @ x, b))        # verificação: Ax = b ?
```

```
[-1.  1.]
True
```

Um sistema 3×3:

$$\begin{cases} x + y - 2z = 1 \\ x + y - z = 3 \\ 3x + y - z = 3 \end{cases}$$

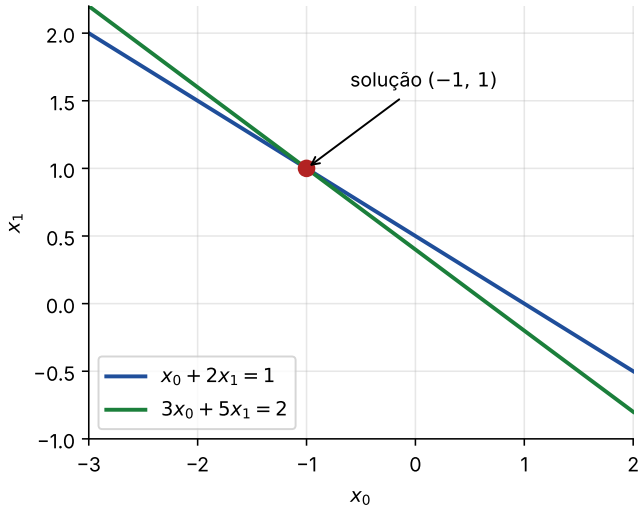


Figura 5.2: As retas $x_0 + 2x_1 = 1$ e $3x_0 + 5x_1 = 2$ se encontram em $(-1, 1)$.

[10]:
```
A = np.array([[1, 1, -2], [1, 1, -1], [3, 1, -1]])
b = np.array([1, 3, 3])
print(np.linalg.det(A))
print(np.linalg.solve(A, b))
```

```
2.0000000000000004
[0. 5. 2.]
```

Observação

`np.linalg.solve` só funciona para matrizes quadradas invertíveis. Se o sistema for impossível ou indeterminado ($\det A = 0$), ele gera `LinAlgError`. Para analisar esses casos, compare o posto da matriz dos coeficientes com o da matriz ampliada (Teorema de Rouché–Capelli), usando `np.linalg.matrix_rank`, ou use o SymPy (Aula 10).

[11]:
```
A = np.array([[1, 2], [2, 4]])
for b in [np.array([3, 6]), np.array([3, 7])]:
    Ab = np.column_stack([A, b])            # matriz ampliada [A | b]
    print(b, np.linalg.matrix_rank(A), np.linalg.matrix_rank(Ab))
```

```
[3 6] 1 1
[3 7] 1 2
```

No primeiro caso os postos são iguais (1) e menores que o número de incógnitas: sistema possível e indeterminado. No segundo, os postos diferem: sistema impossível.

5.4 Programando a eliminação de Gauss

O livro da SBM (Exemplo 3.35 e Exercício 3) propõe programar o método de eliminação de Gauss. Com o NumPy as operações com linhas ficam bem mais simples, pois `M[i]` é a linha i inteira e podemos fazer `M[j] = M[j] - fator * M[i]`.

O método tem duas etapas:

1. **Escalonamento**: para cada coluna i, escolhe-se como pivô o elemento de maior módulo na coluna (da linha i para baixo), troca-se a sua linha com a linha i, e zeram-se os elementos

abaixo do pivô com $L_j \leftarrow L_j - \frac{a_{ji}}{a_{ii}} L_i$.

2. **Retrossubstituição**: no sistema triangular $Ux = c$, calcula-se x_{n-1}, depois x_{n-2}, e assim por diante: $x_i = \dfrac{c_i - \sum_{j>i} u_{ij} x_j}{u_{ii}}$.

[12]:
```
def escalona(A, b):
    """Escalona a matriz ampliada [A | b] (com pivoteamento parcial)."""
    M = np.column_stack([A, b]).astype(float)
    n = len(b)
    for i in range(n - 1):
        p = i + np.argmax(np.abs(M[i:, i]))   # linha do maior pivô
        if M[p, i] == 0:
            raise ValueError("matriz singular")
        M[[i, p]] = M[[p, i]]                 # troca as linhas i e p
        for j in range(i + 1, n):
            fator = M[j, i] / M[i, i]
            M[j] = M[j] - fator * M[i]
    return M

def retrossubstituicao(M):
    n = M.shape[0]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):            # i = n-1, ..., 1, 0
        x[i] = (M[i, -1] - M[i, i+1:n] @ x[i+1:]) / M[i, i]
    return x

def gauss(A, b):
    return retrossubstituicao(escalona(A, b))
```

Vamos testar com o sistema do livro da SBM, cuja matriz ampliada é

$$\left(\begin{array}{cccc|c} 1 & -1 & 2 & -1 & -8 \\ 2 & -2 & 3 & -3 & -20 \\ 1 & 1 & 1 & 0 & -2 \\ 1 & -1 & 4 & -3 & 4 \end{array}\right).$$

Repare que, depois do primeiro passo, aparece um zero na posição do segundo pivô: sem a troca de linhas o método falharia.

[13]:
```
A = np.array([[1, -1, 2, -1], [2, -2, 3, -3], [1, 1, 1, 0], [1, -1, 4, -3]])
b = np.array([-8, -20, -2, 4])
np.set_printoptions(precision=3, suppress=True)
print(escalona(A, b))
x = gauss(A, b)
print("x =", x)
print("conferindo com solve:", np.linalg.solve(A, b))
```

```
[[  2.   -2.    3.   -3.  -20. ]
 [  0.    2.   -0.5   1.5   8. ]
 [  0.    0.    2.5  -1.5  14. ]
 [  0.    0.    0.    0.8  -0.8]]
x = [-13.   6.   5.  -1.]
conferindo com solve: [-13.   6.   5.  -1.]
```

Observação

O comando `np.set_printoptions(precision=3, suppress=True)` faz o NumPy mostrar 3 casas decimais e não usar notação científica para números muito pequenos. Ele vale para o resto do notebook; para voltar ao padrão, use `np.set_printoptions(precision=8, suppress=False)`.

5.5 Autovalores e autovetores

Um número λ é autovalor de A se existe $v \neq 0$ com $Av = \lambda v$. O comando `np.linalg.eig(A)` devolve os autovalores e uma matriz cujas **colunas** são autovetores (normalizados, com norma 1). Para matrizes **simétricas** há o `np.linalg.eigh`, que devolve os autovalores reais e em ordem crescente.

[14]:
```
A = np.array([[2, 1], [1, 2]])
autovalores, V = np.linalg.eigh(A)      # A é simétrica
print(autovalores)
print(V)
v = V[:, 1]                             # autovetor da coluna 1 (lambda = 3)
print(A @ v, autovalores[1] * v)        # A v = lambda v
```

```
[1. 3.]
[[-0.707  0.707]
 [ 0.707  0.707]]
[2.121 2.121] [2.121 2.121]
```

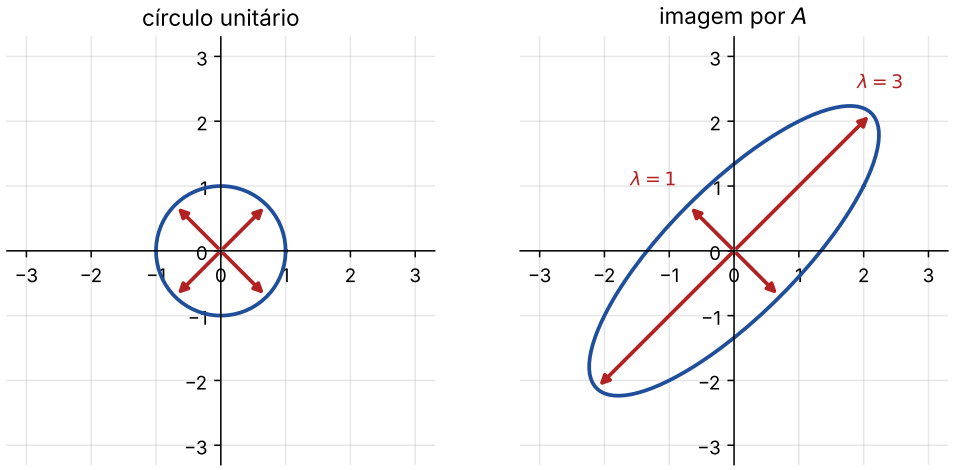


Figura 5.3: A matriz $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ transforma o círculo unitário em uma elipse. Os autovetores (em vermelho) são as direções que não giram: são apenas esticadas, pelos fatores $\lambda = 3$ e $\lambda = 1$.

Para uma matriz de rotação de 90° nenhuma direção é preservada, e os autovalores são complexos:

[15]:
```
R = np.array([[0, -1], [1, 0]])
print(np.linalg.eig(R)[0])
```

```
[0.+1.j 0.-1.j]
```

5.6 Polinômios

No NumPy, um polinômio é representado pela lista de coeficientes, **do maior grau para o menor**: $p(x) = x^3 - 6x^2 + 11x - 6$ é `[1, -6, 11, -6]`.

[16]:
```
p = [1, -6, 11, -6]
print(np.roots(p))                    # raízes
print(np.polyval(p, [0, 1, 4]))       # p(0), p(1), p(4)
print(np.roots([1, 0, 1]))            # x^2 + 1: raízes complexas
print(np.poly([1, 2, 3]))             # polinômio com raízes 1, 2, 3
```

```
[3. 2. 1.]
[-6  0  6]
[-0.+1.j  0.-1.j]
[ 1. -6. 11. -6.]
```

Ajuste de curvas (mínimos quadrados). Dados pontos (x_i, y_i) experimentais, `np.polyfit(x, y, grau)` encontra o polinômio do grau pedido que melhor se ajusta aos dados (minimizando $\sum (p(x_i) - y_i)^2$).

[17]:
```
x = np.array([0, 1, 2, 3, 4, 5])
y = np.array([1.1, 2.9, 5.2, 7.1, 8.8, 11.2])
a, b = np.polyfit(x, y, 1)            # reta y = a x + b
print(f"y = {a:.3f} x + {b:.3f}")
print(f"previsão para x = 6: {np.polyval([a, b], 6):.2f}")
```

```
y = 2.003 x + 1.043
previsão para x = 6: 13.06
```

O gráfico desses dados com a reta ajustada é feito na Aula 6.

Nota ao professor

Se a turma já viu escalonamento em Álgebra Linear, vale fazer o primeiro passo da eliminação no quadro e comparar com a saída de `escalona`. A função usa `M[[i, p]] = M[[p, i]]` para trocar linhas, um recurso ("indexação por lista") que pode precisar de explicação.

5.7 Exercícios

Exercício 5.1 (operações)

Sejam $A = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 4 & 0 & 1 \end{pmatrix}$ e $B = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 3 & 1 \end{pmatrix}$. Calcule $A + B$, AB, BA, $A^T B$, $\det(AB)$ e $\det A \cdot \det B$. Verifique que $(AB)^T = B^T A^T$.

Exercício 5.2 (sistema)

Resolva com `np.linalg.solve` e com a função `gauss`: $\begin{cases} 2x + y - z = 8 \\ -3x - y + 2z = -11 \\ -2x + y + 2z = -3 \end{cases}$.

Exercício 5.3 (matriz de Hilbert)

Para $n = 4, 8, 12$, crie a matriz de Hilbert H_n, tome $x = (1, 1, \ldots, 1)$, calcule $b = H_n x$ e resolva $H_n y = b$. Compare y com x. O que acontece? (Pesquise "matriz mal condicionada" e use `np.linalg.cond`.)

Exercício 5.4 (autovalores)

Calcule os autovalores e autovetores de $\begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$ e verifique $Av = \lambda v$ para cada par. Confira que a soma dos autovalores é o traço e o produto é o determinante.

Exercício 5.5 (polinômios)

Encontre as raízes de $x^4 - 5x^2 + 4$ e de $x^3 - 2x - 5$. Use `np.polyval` para conferir que são raízes.

Exercício 5.6 (extra: ajuste de parábola)

Um objeto é lançado e sua altura (m) é medida nos instantes $t = 0,0{,}5, 1, 1{,}5, 2, 2{,}5$ s: $h = 1{,}0;\ 7{,}9;\ 12{,}4;\ 14{,}6;\ 14{,}3;\ 11{,}7$. Ajuste uma parábola $h(t) = at^2 + bt + c$ e estime a aceleração da gravidade ($g \approx -2a$) e o instante de altura máxima.

CAPÍTULO 6

Gráficos com Matplotlib

Objetivos da aula

- entender como o computador desenha o gráfico de uma função (pontos ligados por segmentos);
- fazer gráficos com `plt.plot` e personalizar cores, estilos, legendas, títulos e eixos;
- colocar vários gráficos em uma mesma figura (`plt.subplots`);
- desenhar curvas paramétricas, curvas em coordenadas polares e curvas de nível;
- desenhar superfícies e curvas no espaço; salvar figuras em arquivo.

Roteiro sugerido

Tempo	Atividade
0–15 min	Primeiro gráfico; discretização com `linspace`; poucos *versus* muitos pontos.
15–30 min	Personalização: cores, estilos, legenda, título, eixos, grade; vários gráficos.
30–40 min	`subplots`; gráficos de dados (`scatter`) e reta ajustada.
40–55 min	Curvas paramétricas e polares; descontinuidades.
55–70 min	`meshgrid`, curvas de nível e curvas implícitas.
70–85 min	Superfícies e curvas em 3D. Salvando figuras.
85–90 min	Exercícios para casa.

6.1 O primeiro gráfico

O Matplotlib é a biblioteca de gráficos mais usada em Python. Usamos seu submódulo `pyplot`, importado com o apelido `plt`, junto com o NumPy:

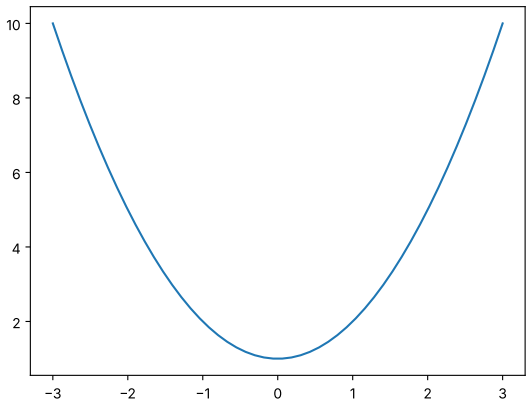
```
import numpy as np
import matplotlib.pyplot as plt
```

Para o computador, o gráfico de $y = f(x)$ é um conjunto de pontos $(x_i, f(x_i))$ ligados por segmentos de reta. Os passos são sempre os mesmos:

1. **discretizar** o intervalo: `x = np.linspace(a, b, n)`;
2. calcular `y = f(x)` (com funções do NumPy);
3. desenhar com `plt.plot(x, y)` e mostrar com `plt.show()`.

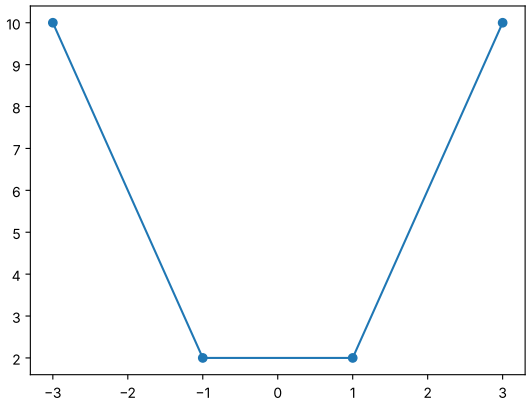
[1]:
```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 50)     # 50 pontos entre -3 e 3
y = x**2 + 1
plt.plot(x, y)
plt.show()
```



Com poucos pontos, o "gráfico" vira uma poligonal. É por isso que um esboço com 4 pontos não é um bom gráfico:

[2]:
```
x = np.linspace(-3, 3, 4)
plt.plot(x, x**2 + 1, "o-")    # "o-": bolinhas ligadas por linhas
plt.show()
```



Dica

No JupyterLab o gráfico aparece logo abaixo da célula. O `plt.show()` no fim evita que apareçam linhas de texto como `[<matplotlib.lines.Line2D at 0x...>]` antes da figura.

6.2 Personalizando o gráfico

O terceiro argumento de `plt.plot` é uma *string de formato* que combina cor, marcador e tipo de linha:

Cores		Linhas		Marcadores	
`"b"`	azul	`"-"`	contínua	`"o"`	círculo
`"r"`	vermelho	`"--"`	tracejada	`"s"`	quadrado
`"g"`	verde	`":"`	pontilhada	`"^"`	triângulo
`"k"`	preto	`"-."`	traço-ponto	`"*"`	estrela
`"m"`, `"c"`, `"y"`	magenta, ciano, amarelo			`"."`	ponto

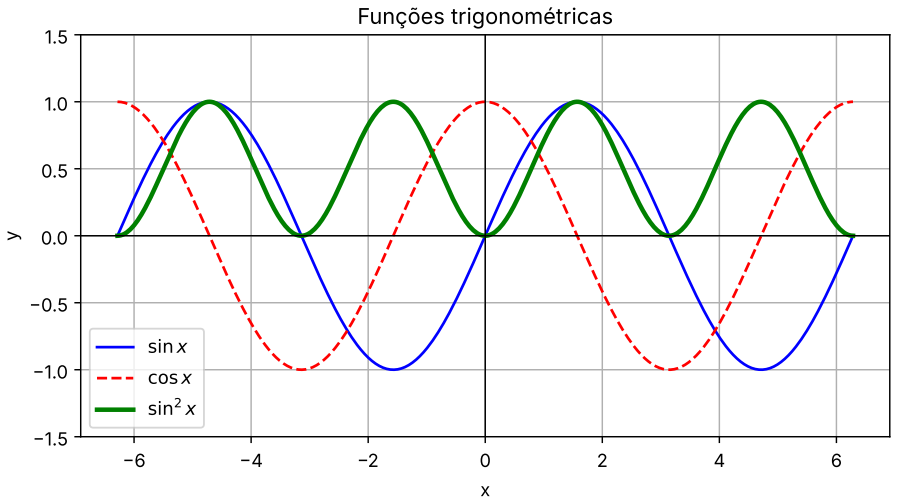
Exemplos: `"r--"` (vermelho tracejado), `"bo"` (só bolinhas azuis), `"k-o"` (linha preta com

bolinhas). Também se pode usar os parâmetros `color="purple"`, `linestyle="-"`, `linewidth=2`, `marker="o"`.

Comando	Efeito
`plt.plot(x, y, label="f")`, `plt.legend()`	legenda
`plt.title("...")`, `plt.xlabel("x")`, `plt.ylabel("y")`	título e eixos
`plt.xlim(a, b)`, `plt.ylim(c, d)`	janela de visualização
`plt.grid(True)`	grade
`plt.axhline(0)`, `plt.axvline(0)`	retas $y = 0$ e $x = 0$
`plt.axis("equal")`	mesma escala nos dois eixos
`plt.figure(figsize=(8, 4))`	tamanho da figura (em polegadas)

Textos entre `$...$` são escritos como fórmulas do LaTeX: `label=r"$\sin(x)$"` (o `r` antes das aspas evita problemas com a barra invertida).

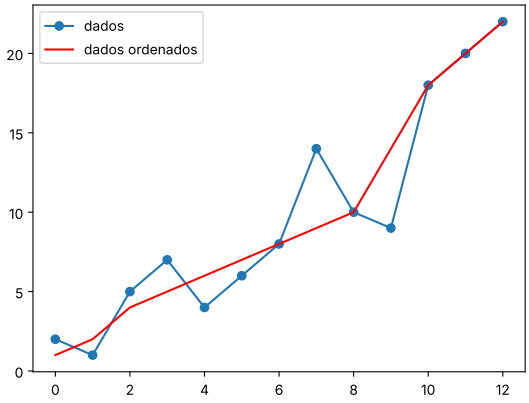
[3]:
```
x = np.linspace(-2*np.pi, 2*np.pi, 400)
plt.figure(figsize=(8, 4))
plt.plot(x, np.sin(x), "b-", label=r"$\sin x$")
plt.plot(x, np.cos(x), "r--", label=r"$\cos x$")
plt.plot(x, np.sin(x)**2, color="green", linewidth=2.5, label=r"$\sin^2 x$")
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(0, color="black", linewidth=0.8)
plt.title("Funções trigonométricas")
plt.xlabel("x")
plt.ylabel("y")
plt.ylim(-1.5, 1.5)
plt.grid(True)
plt.legend()
plt.show()
```



Dados *versus* posição. Se `plt.plot` receber apenas um *array*, os valores são colocados no eixo y e o eixo x é a posição $0, 1, 2, \ldots$ (exemplo do livro da SBM):

[4]:
```
a = np.array([2, 1, 5, 7, 4, 6, 8, 14, 10, 9, 18, 20, 22])
b = np.sort(a)
plt.plot(a, "o-", label="dados")
plt.plot(b, color="red", label="dados ordenados")
plt.legend()
```

```
plt.show()
```



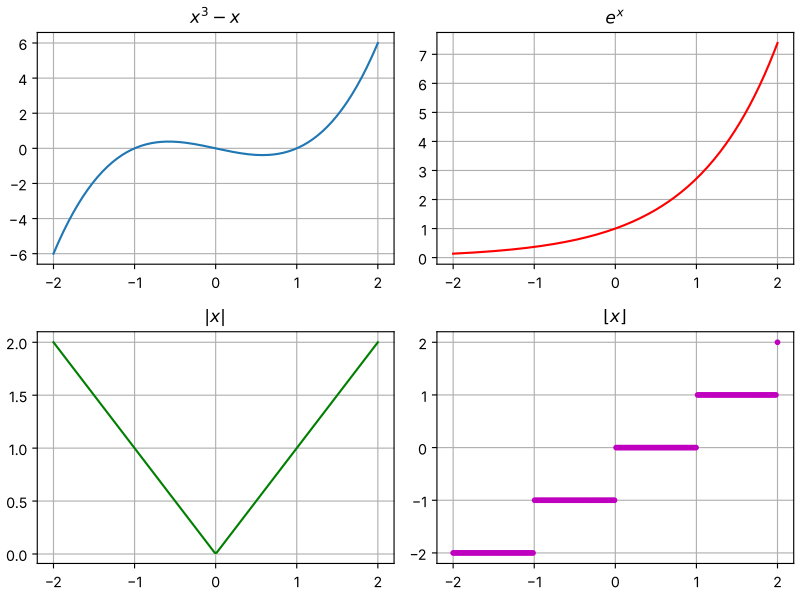
Para quem vem do Maxima

No Maxima: `plot2d([sin(x), cos(x)], [x, -2*%pi, 2*%pi], [y, -1.5, 1.5])`. No Matplotlib não se passa a *fórmula*, e sim os *pontos*; por isso é preciso criar o `x` com `linspace`. Na Aula 7 veremos o `sp.plot` do SymPy, que funciona como o `plot2d`.

6.3 Vários gráficos em uma figura

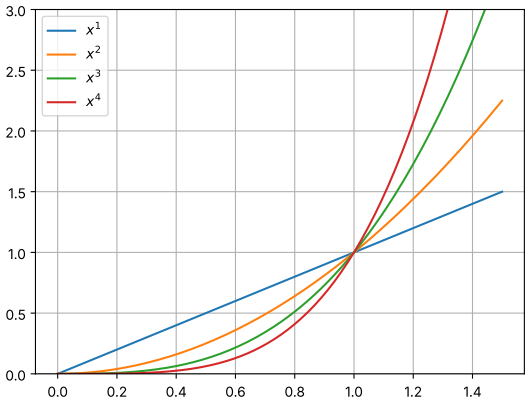
Para colocar gráficos lado a lado, usamos `fig, ax = plt.subplots(linhas, colunas)`. A variável `ax` passa a ser uma tabela de "eixos", e cada gráfico é desenhado com `ax[i, j].plot(...)`. Com os eixos os comandos de personalização têm o prefixo `set_`: `ax.set_title`, `ax.set_xlabel`, `ax.set_ylim`...

[5]:
```
x = np.linspace(-2, 2, 200)
fig, ax = plt.subplots(2, 2, figsize=(8, 6))
ax[0, 0].plot(x, x**3 - x)
ax[0, 0].set_title(r"$x^3 - x$")
ax[0, 1].plot(x, np.exp(x), "r")
ax[0, 1].set_title(r"$e^x$")
ax[1, 0].plot(x, np.abs(x), "g")
ax[1, 0].set_title(r"$|x|$")
ax[1, 1].plot(x, np.floor(x), "m.")
ax[1, 1].set_title(r"$\lfloor x \rfloor$")
for eixo in ax.flat:          # aplica a todos os quatro
    eixo.grid(True)
plt.tight_layout()            # evita sobreposição dos títulos
plt.show()
```



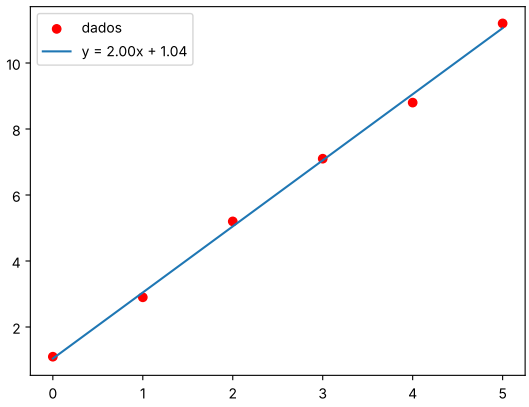
Famílias de curvas. Um `for` desenha várias curvas de uma vez, por exemplo $y = x^n$ para vários n:

[6]:
```
x = np.linspace(0, 1.5, 200)
for n in [1, 2, 3, 4]:
    plt.plot(x, x**n, label=f"$x^{n}$")
plt.ylim(0, 3)
plt.legend()
plt.grid(True)
plt.show()
```



Dados experimentais e reta ajustada. Pontos isolados são desenhados com `plt.scatter` (ou com `"o"`). Retomando os dados da Aula 5:

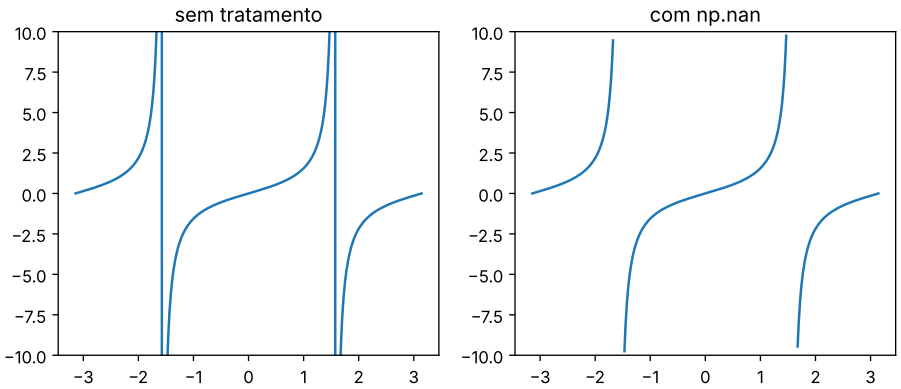
[7]:
```
xd = np.array([0, 1, 2, 3, 4, 5])
yd = np.array([1.1, 2.9, 5.2, 7.1, 8.8, 11.2])
a, b = np.polyfit(xd, yd, 1)
xs = np.linspace(0, 5, 2)
plt.scatter(xd, yd, color="red", label="dados")
plt.plot(xs, a*xs + b, label=f"y = {a:.2f}x + {b:.2f}")
plt.legend()
plt.show()
```



6.4 Descontinuidades e assíntotas

O Matplotlib liga *todos* os pontos consecutivos. Em uma assíntota vertical, isso produz uma reta "falsa" ligando $+\infty$ a $-\infty$. Uma solução é limitar o eixo y e trocar por `np.nan` ("não é número") os valores muito grandes: pontos `nan` não são ligados.

[8]:
```
x = np.linspace(-np.pi, np.pi, 1000)
y = np.tan(x)
fig, ax = plt.subplots(1, 2, figsize=(9, 3.5))
ax[0].plot(x, y)
ax[0].set_ylim(-10, 10)
ax[0].set_title("sem tratamento")
y[np.abs(y) > 10] = np.nan          # remove os valores enormes
ax[1].plot(x, y)
ax[1].set_ylim(-10, 10)
ax[1].set_title("com np.nan")
plt.show()
```

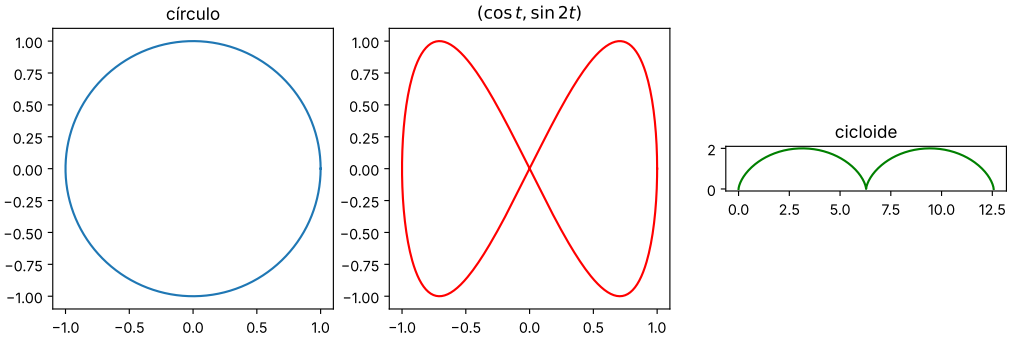


6.5 Curvas paramétricas e polares

Uma curva parametrizada $\gamma(t) = (x(t), y(t))$, $t \in [a, b]$, é desenhada da mesma forma: discretiza-se o *parâmetro* t e liga-se os pontos $(x(t_i), y(t_i))$. Use `plt.axis("equal")` para não deformar a figura.

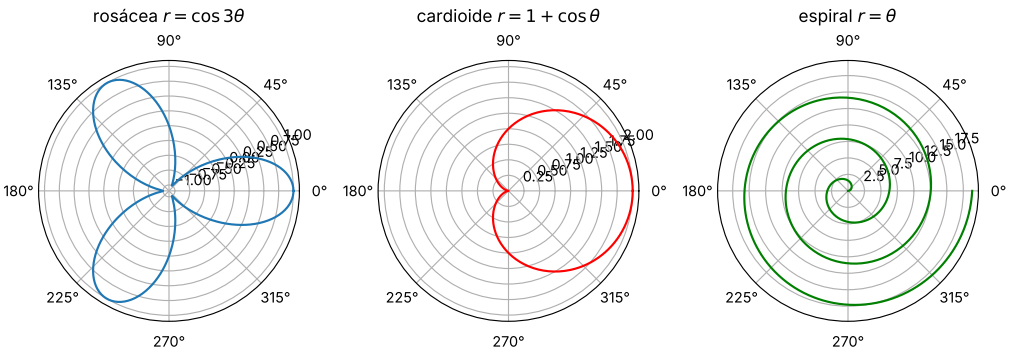
[9]:
```
t = np.linspace(0, 2*np.pi, 400)
fig, ax = plt.subplots(1, 3, figsize=(10, 3.4))
```

```
ax[0].plot(np.cos(t), np.sin(t))                    # círculo
ax[0].set_title("círculo")
ax[1].plot(np.cos(t), np.sin(2*t), "r")              # Lissajous
ax[1].set_title(r"$(\cos t, \sin 2t)$")
s = np.linspace(0, 4*np.pi, 400)
ax[2].plot(s - np.sin(s), 1 - np.cos(s), "g")        # cicloide
ax[2].set_title("cicloide")
for eixo in ax:
    eixo.set_aspect("equal")
plt.tight_layout()
plt.show()
```



Coordenadas polares. Uma curva $r = r(\theta)$ é a curva paramétrica $x = r(\theta)\cos\theta$, $y = r(\theta)\sin\theta$. Também é possível usar eixos polares com `subplot_kw={"projection": "polar"}`.

[10]:
```
th = np.linspace(0, 2*np.pi, 500)
fig, ax = plt.subplots(1, 3, figsize=(10, 3.4), subplot_kw={"projection":
    ↪ "polar"})
ax[0].plot(th, np.cos(3*th))
ax[0].set_title(r"rosácea $r=\cos 3\theta$")
ax[1].plot(th, 1 + np.cos(th), "r")
ax[1].set_title(r"cardioide $r=1+\cos\theta$")
th2 = np.linspace(0, 6*np.pi, 800)
ax[2].plot(th2, th2, "g")
ax[2].set_title(r"espiral $r=\theta$")
plt.tight_layout()
plt.show()
```



6.6 Funções de duas variáveis: curvas de nível

Para $z = f(x, y)$ precisamos de uma *malha* de pontos (x_i, y_j). O comando `np.meshgrid` faz o "produto cartesiano" de dois vetores, devolvendo as matrizes `X` e `Y` com as coordenadas de cada ponto da malha:

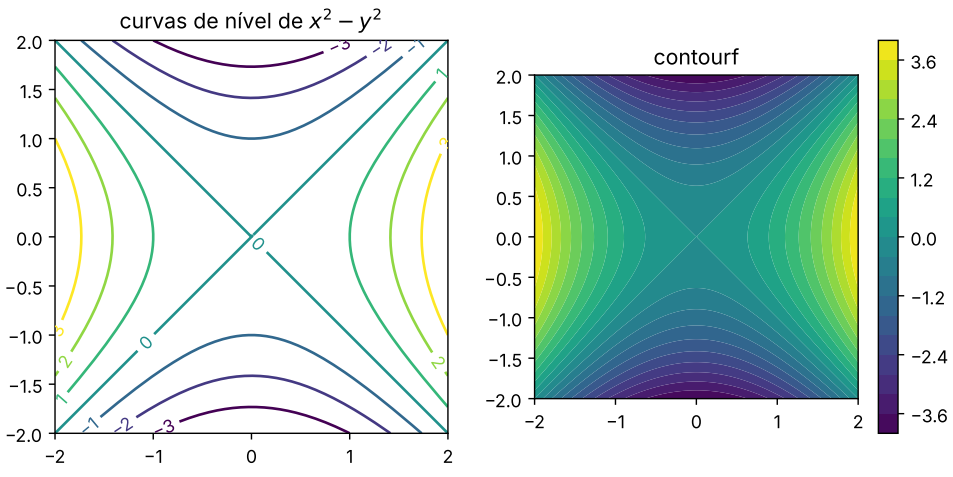
[11]:
```
x = np.array([0, 1, 2])
y = np.array([10, 20])
X, Y = np.meshgrid(x, y)
print(X)
print(Y)
print(X + Y)        # f(x, y) = x + y calculada em todos os pontos da malha
```

```
[[0 1 2]
 [0 1 2]]
[[10 10 10]
 [20 20 20]]
[[10 11 12]
 [20 21 22]]
```

Com a malha, `plt.contour(X, Y, Z, níveis)` desenha as **curvas de nível** $f(x, y) = c$ e `plt.contourf` pinta as regiões entre elas.

[12]:
```
x = np.linspace(-2, 2, 200)
y = np.linspace(-2, 2, 200)
X, Y = np.meshgrid(x, y)
Z = X**2 - Y**2                    # sela

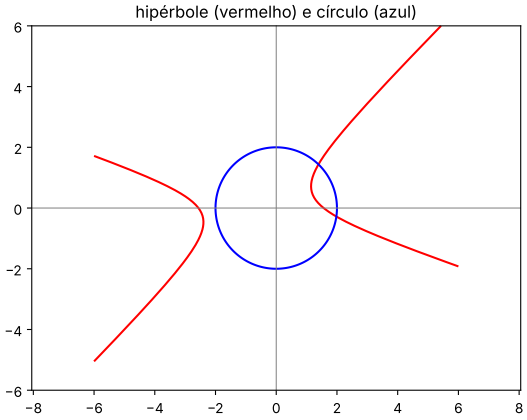
fig, ax = plt.subplots(1, 2, figsize=(9, 4))
curvas = ax[0].contour(X, Y, Z, levels=[-3, -2, -1, 0, 1, 2, 3])
ax[0].clabel(curvas)                # escreve os valores nas curvas
ax[0].set_title(r"curvas de nível de $x^2-y^2$")
preench = ax[1].contourf(X, Y, Z, levels=20, cmap="viridis")
fig.colorbar(preench, ax=ax[1])
ax[1].set_title("contourf")
for eixo in ax:
    eixo.set_aspect("equal")
plt.show()
```



Curvas implícitas. Para desenhar uma curva dada por uma equação $F(x, y) = 0$ (como uma cônica), basta desenhar a curva de nível 0 de F. Exemplo (tutorial do R. Miranda):

$x^2 + 2xy - 3y^2 + x + 2y - 4 = 0.$

[13]:
```
x = np.linspace(-6, 6, 400)
X, Y = np.meshgrid(x, x)
F = X**2 + 2*X*Y - 3*Y**2 + X + 2*Y - 4
plt.contour(X, Y, F, levels=[0], colors="red")
plt.contour(X, Y, X**2 + Y**2 - 4, levels=[0], colors="blue")   # círculo x^2
    ↪ + y^2 = 4
plt.axhline(0, color="gray", lw=0.6)
plt.axvline(0, color="gray", lw=0.6)
plt.axis("equal")
plt.title("hipérbole (vermelho) e círculo (azul)")
plt.show()
```



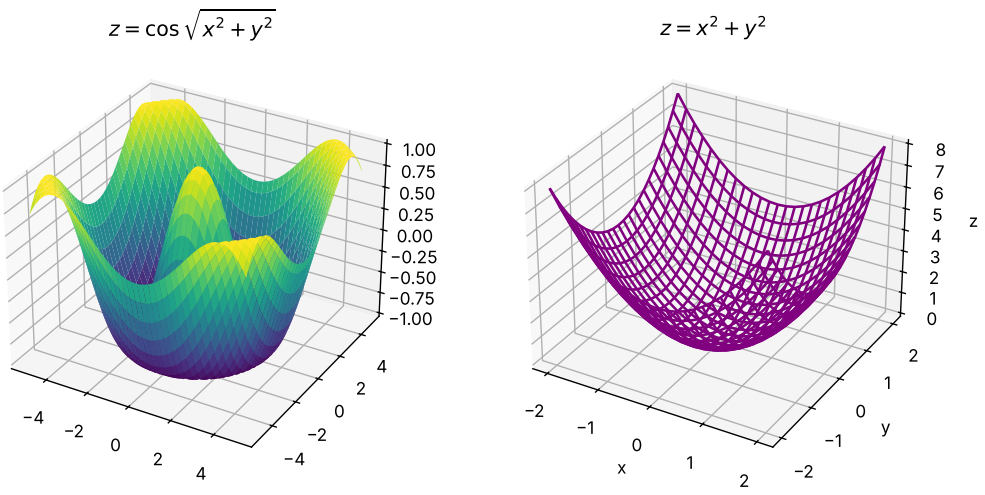
6.7 Gráficos em três dimensões

Para gráficos 3D, cria-se um eixo com `projection="3d"`. O gráfico de $z = f(x, y)$ é feito com `ax.plot_surface(X, Y, Z)` (superfície colorida) ou `ax.plot_wireframe(X, Y, Z)` (malha). Exemplo do livro da SBM, $z = \cos\sqrt{x^2 + y^2}$, e um paraboloide:

[14]:
```
x = np.linspace(-5, 5, 80)
X, Y = np.meshgrid(x, x)
Z = np.cos(np.sqrt(X**2 + Y**2))

fig = plt.figure(figsize=(10, 4.5))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_surface(X, Y, Z, cmap="viridis")
ax1.set_title(r"$z=\cos\sqrt{x^2+y^2}$")

ax2 = fig.add_subplot(1, 2, 2, projection="3d")
x = np.linspace(-2, 2, 25)
X, Y = np.meshgrid(x, x)
ax2.plot_wireframe(X, Y, X**2 + Y**2, color="purple")
ax2.set_xlabel("x"); ax2.set_ylabel("y"); ax2.set_zlabel("z")
ax2.set_title(r"$z=x^2+y^2$")
plt.show()
```



Dica

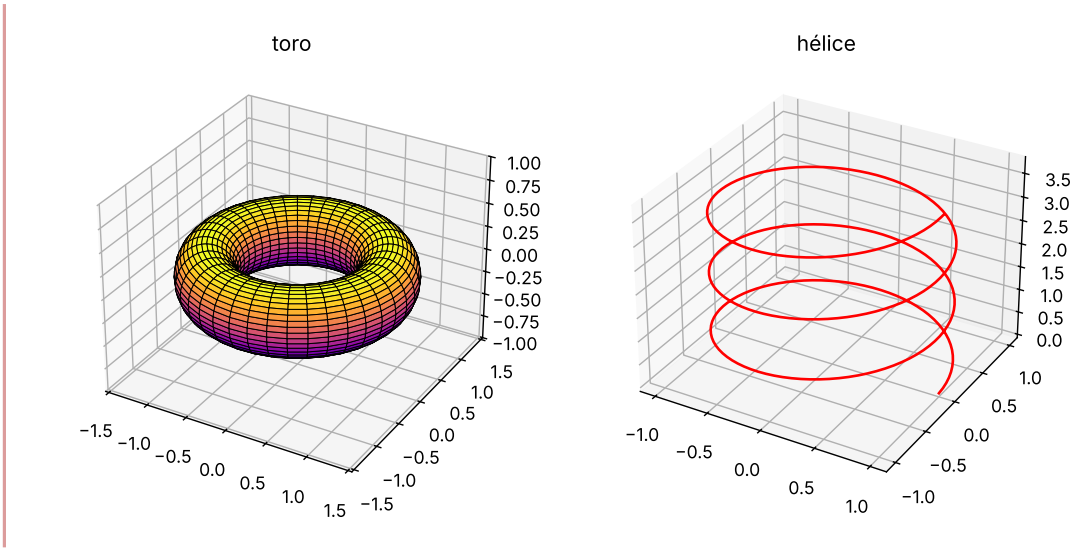
Para girar o gráfico 3D com o mouse no JupyterLab, execute `%matplotlib widget` no início do notebook (requer o pacote `ipympl`). Sem ele, pode-se escolher o ponto de vista com `ax.view_init(elev=30, azim=45)`.

Superfícies e curvas parametrizadas. Uma superfície parametrizada $(x(u,v), y(u,v), z(u,v))$ é feita com uma malha nos parâmetros (u, v). Abaixo, o toro (tutorial do R. Miranda) e a hélice $(\cos t, \sin t, t/5)$:

[15]:
```
u = np.linspace(0, 2*np.pi, 40)
U, V = np.meshgrid(u, u)
r, R = 0.35, 1
X = (R + r*np.cos(V)) * np.cos(U)
Y = (R + r*np.cos(V)) * np.sin(U)
Z = r * np.sin(V)

fig = plt.figure(figsize=(10, 4.5))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_surface(X, Y, Z, cmap="plasma", edgecolor="k", linewidth=0.2)
ax1.set_zlim(-1, 1)
ax1.set_title("toro")

ax2 = fig.add_subplot(1, 2, 2, projection="3d")
t = np.linspace(0, 6*np.pi, 500)
ax2.plot(np.cos(t), np.sin(t), t/5, "r")
ax2.set_title("hélice")
plt.show()
```



6.8 Salvando figuras

Para salvar a figura em um arquivo (para um relatório ou trabalho), use `plt.savefig` **antes** do `plt.show()`. O formato é escolhido pela extensão: `.png` (imagem), `.pdf` ou `.svg` (vetoriais, ideais para documentos LaTeX).

```
x = np.linspace(0, 2*np.pi, 200)
plt.plot(x, np.sin(x))
plt.savefig("seno.png", dpi=200)      # dpi: resolução
plt.savefig("seno.pdf")
plt.show()
```

O arquivo é gravado na mesma pasta do notebook e aparece no painel de arquivos do JupyterLab.

Nota ao professor

Esta aula é a mais "visual" e costuma render bem se os alunos alterarem os exemplos: trocar a função, o intervalo, o número de pontos. Uma sugestão de dinâmica: projetar a Figura do exemplo dos 4 pontos e perguntar o que está errado. Os exemplos 3D podem ser lentos em máquinas antigas; se necessário, reduza o número de pontos da malha.

6.9 Exercícios

Exercício 6.1 (função e derivada)

Em um mesmo gráfico, desenhe $f(x) = x^3 - 3x$ e $f'(x) = 3x^2 - 3$ em $[-2{,}5;\ 2{,}5]$, com cores e legendas diferentes, eixos e grade. Marque com bolinhas vermelhas os pontos críticos de f.

Exercício 6.2 (exponenciais e logaritmos)

Desenhe, em uma figura com 1 linha e 2 colunas: (a) 2^x, e^x e 10^x em $[-2, 2]$; (b) $\log_2 x$, $\ln x$ e $\log_{10} x$ em $(0, 10]$. Use `plt.axis("equal")` em um deles e compare com $y = x$ para ver a simetria entre e^x e $\ln x$.

Exercício 6.3 (família de parábolas)

Desenhe $y = x^2 + bx + 1$ para $b = -3, -2, \ldots, 3$ e marque o vértice de cada parábola. Que curva os vértices descrevem?

Exercício 6.4 (curvas paramétricas e polares)

Desenhe: (a) a astroide $(\cos^3 t, \sin^3 t)$; (b) a lemniscata $r^2 = \cos 2\theta$ (use a curva de nível 0 de $(x^2 + y^2)^2 - (x^2 - y^2)$); (c) a rosácea $r = \sin 4\theta$.

Exercício 6.5 (superfície e curvas de nível)

Para $f(x, y) = x\, e^{-x^2 - y^2}$, faça o gráfico 3D e o mapa de curvas de nível lado a lado. Pelo mapa, estime onde ficam o máximo e o mínimo de f.

Exercício 6.6 (extra: tangente e secantes)

Para $f(x) = \sqrt{x}$ e $a = 1$, desenhe o gráfico de f e as retas secantes por $(1, f(1))$ e $(1 + h, f(1 + h))$ para $h = 2, 1, 0{,}5, 0{,}1$. Desenhe também a reta tangente $y = 1 + \frac{1}{2}(x - 1)$. (Essa figura antecipa a Aula 8.)

CAPÍTULO 7

SymPy I: cálculo simbólico, álgebra e equações

Objetivos da aula

- entender a diferença entre cálculo numérico (`math`, NumPy) e simbólico (SymPy);
- criar símbolos e expressões; obter valores exatos e aproximados (`evalf`, `N`);
- manipular expressões: `expand`, `factor`, `simplify`, `collect`, `apart`, `subs`;
- resolver equações, sistemas e inequações com `solve`, `solveset`, `linsolve` e `nsolve`;
- calcular somatórios simbólicos e fazer gráficos com o próprio SymPy.

Roteiro sugerido

Tempo	Atividade
0–15 min	Simbólico *versus* numérico; `symbols`; valores exatos e `evalf`.
15–35 min	Manipulação de expressões: expandir, fatorar, simplificar, frações parciais, substituir.
35–60 min	Equações: `solve`, `Eq`, `solveset`, sistemas, `nsolve`; inequações.
60–70 min	Somatórios e números inteiros (primos, fatoração).
70–80 min	Gráficos com `sp.plot` e `sp.plot_implicit`.
80–90 min	Exercícios.

7.1 Cálculo simbólico

Até aqui, todas as contas foram **numéricas**: $\sqrt{2}$ era `1.4142135623730951`, um número aproximado. O **SymPy** faz cálculo **simbólico**, como o Maxima: trabalha com $\sqrt{2}$, π, frações e variáveis de forma exata.

[1]:
```
import math
import sympy as sp

print(math.sqrt(8), sp.sqrt(8))
print(1/3 + 1/6, sp.Rational(1, 3) + sp.Rational(1, 6))
print(sp.sqrt(2)**2, math.sqrt(2)**2)
```

```
2.8284271247461903 2*sqrt(2)
0.5 1/2
2 2.0000000000000004
```

Quando a última linha da célula é uma expressão do SymPy, o Jupyter a mostra formatada como fórmula matemática:

[2]:
```
sp.sqrt(8) + sp.pi/2 + sp.Rational(3, 4)
```

[2]:
$$\frac{3}{4} + \frac{\pi}{2} + 2\sqrt{2}$$

Atenção

`1/3` é calculado pelo Python *antes* de chegar ao SymPy, e vira o `float` `0.333...`. Para frações exatas use `sp.Rational(1, 3)` ou `sp.S(1)/3`. O mesmo vale para `x**(1/2)`: prefira `sp.sqrt(x)` ou `x**sp.Rational(1, 2)`.

Valores aproximados. O método `.evalf(n)` (ou a função `sp.N(expr, n)`) calcula o valor numérico com n algarismos significativos, quantos quisermos:

[3]:
```
print(sp.pi.evalf())
print(sp.N(sp.pi, 50))
print(sp.N(sp.E, 30), sp.sqrt(2).evalf(20))
```

```
3.14159265358979
3.1415926535897932384626433832795028841971693993751
2.7182818284590452353602874713 1.4142135623730950488
```

Matemática	SymPy	Maxima
π, e, i, ∞	`sp.pi`, `sp.E`, `sp.I`, `sp.oo`	`%pi`, `%e`, `%i`, `inf`
$\sqrt{x}$, e^x, $\ln x$	`sp.sqrt(x)`, `sp.exp(x)`, `sp.log(x)`	`sqrt(x)`, `exp(x)`, `log(x)`
$\sin x$, $\arctan x$	`sp.sin(x)`, `sp.atan(x)`	`sin(x)`, `atan(x)`
$\lvert x\rvert$, $n!$, $\binom{n}{k}$	`sp.Abs(x)`, `sp.factorial(n)`, `sp.binomial(n, k)`	`abs(x)`, `n!`, `binomial(n,k)`
valor numérico	`expr.evalf()`, `sp.N(expr, 30)`	`float(expr)`, `bfloat(expr)`

7.2 Símbolos e expressões

O Python não sabe o que é x. Antes de usar uma letra como variável matemática, é preciso declará-la como **símbolo**:

[4]:
```
x, y, z = sp.symbols('x y z')
expr = (x + 2*y)**2 - x
expr
```

[4]:
$$-x + (x + 2y)^2$$

[5]:
```
w + 1          # w não foi declarado
```

```
NameError: name 'w' is not defined
```

Símbolos podem ter *hipóteses*, que ajudam nas simplificações: `positive=True`, `real=True`, `integer=True`.

[6]:
```
a = sp.symbols('a')
b = sp.symbols('b', positive=True)
print(sp.sqrt(a**2), sp.sqrt(b**2))
```

```
sqrt(a**2) b
```

Como a pode ser negativo, $\sqrt{a^2} = |a|$ e o SymPy não pode simplificar para a; já para $b > 0$ vale $\sqrt{b^2} = b$.

Para quem vem do Maxima

No Maxima as letras já são símbolos e não precisam ser declaradas. Hipóteses são feitas com `assume(t > 0)`. No SymPy, com `sp.symbols('t', positive=True)`.

7.3 Manipulando expressões

SymPy	O que faz	Maxima
`sp.expand(e)`	expande produtos e potências	`expand`
`sp.factor(e)`	fatora	`factor`
`sp.simplify(e)`	tenta simplificar (escolhe a forma "mais simples")	`ratsimp`
`sp.collect(e, x)`	agrupa os termos por potências de x	`collectterms`
`sp.cancel(e)`	cancela fatores comuns de uma fração	`ratsimp`
`sp.apart(e, x)`	decompõe em frações parciais	`partfrac`
`sp.together(e)`	junta em uma só fração	`ratsimp`
`sp.trigsimp(e)`	simplifica expressões trigonométricas	`trigsimp`
`sp.expand_trig(e)`	expande expressões trigonométricas	`trigexpand`
`e.subs(x, valor)`	substitui x por um valor ou expressão	`subst`, `ev`

[7]:
```
sp.expand((x + 1)**5)
```

[7]:
$$x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1$$

[8]:
```
sp.factor(x**3 - 6*x**2 + 11*x - 6)
```

[8]:
$$(x - 3)(x - 2)(x - 1)$$

[9]:
```
sp.factor(x**4 - y**4), sp.factor(x**4 + 4)
```

[9]:
$$\left((x - y)(x + y)\left(x^2 + y^2\right),\ \left(x^2 - 2x + 2\right)\left(x^2 + 2x + 2\right)\right)$$

[10]:
```
f = (x**2 - 1) / (x**2 + 2*x + 1)
sp.simplify(f)
```

[10]:
$$\frac{x - 1}{x + 1}$$

[11]:
```
g = (4*x**2 + 3) / (x**3 - x)
sp.apart(g)                          # frações parciais
```

[11]:
$$\frac{7}{2(x + 1)} + \frac{7}{2(x - 1)} - \frac{3}{x}$$

[12]:
```
sp.together(1/x + 1/(x + 1))
```

[12]:
$$\frac{2x + 1}{x(x + 1)}$$

[13]:
```
p = sp.expand((x + y + 1)**2)
print(p)
sp.collect(p, x)                     # agrupa por potências de x
```

```
x**2 + 2*x*y + 2*x + y**2 + 2*y + 1
```

[13]: $x^2 + x\,(2y + 2) + y^2 + 2y + 1$

```
[14]: sp.simplify(sp.sin(x)**2 + sp.cos(x)**2), sp.expand_trig(sp.sin(2*x)),
          ↪ sp.trigsimp(2*sp.sin(x)*sp.cos(x))
```

[14]: $(1,\ 2\sin(x)\cos(x),\ \sin(2x))$

Substituição. O método `subs` troca um símbolo por um número ou por outra expressão. Para várias substituições, passa-se um dicionário.

```
[15]: F = x**2 + y**3 - 2
      print(F.subs(x, 1))
      print(F.subs({x: 1, y: 2}))
      print(F.subs(x, sp.sqrt(y)))
      print(F.subs(x, sp.pi).evalf(5))
```

```
y**3 - 1
7
y**3 + y - 2
y**3 + 7.8696
```

Funções. Uma função matemática pode ser guardada como expressão e avaliada com `subs`, ou definida com `def`/`lambda`. Para fazer gráficos com o Matplotlib ou calcular muitos valores rapidamente, `sp.lambdify` converte a expressão em uma função do NumPy.

```
[16]: f = x**3 - 2*x + 1
      print(f.subs(x, 2))
      def g(t):
          return t**3 - 2*t + 1
      print(g(sp.Rational(1, 2)), g(x + 1))
      import numpy as np
      f_num = sp.lambdify(x, f, "numpy")
      print(f_num(np.array([0, 1, 2, 3])))
```

```
5
1/8 -2*x + (x + 1)**3 - 1
[ 1  0  5 22]
```

Para quem vem do Maxima

`f(x) := x^3 - 2*x + 1;` e `f(2);` no Maxima correspondem a `def f(x): return x**3 - 2*x + 1` e `f(2)`. A substituição `ev(expr, x=2)` ou `subst(2, x, expr)` é `expr.subs(x, 2)`.

7.4 Equações

7.4.1 `solve`

O comando `sp.solve(expr, x)` resolve a equação expr $= 0$ na variável x e devolve a **lista** de soluções. Para equações da forma "lado esquerdo = lado direito", use `sp.Eq(esquerdo, direito)`.

[17]:
```
sp.solve(x**2 - 5*x + 6, x)
```

[17]:
$$[2,\ 3]$$

[18]:
```
sp.solve(sp.Eq(x**2, 2*x + 1), x)
```

[18]:
$$\left[1 - \sqrt{2},\ 1 + \sqrt{2}\right]$$

[19]:
```
sp.solve(x**2 + 1, x), sp.solve(x**3 - 1, x)
```

[19]:
$$\left([-i,\ i],\ \left[1,\ -\frac{1}{2} - \frac{\sqrt{3}i}{2},\ -\frac{1}{2} + \frac{\sqrt{3}i}{2}\right]\right)$$

Equações com parâmetros. Resolvendo $ax^2 + bx + c = 0$ "em geral", reencontramos a fórmula de Bhaskara:

[20]:
```
a, b, c = sp.symbols('a b c')
sp.solve(a*x**2 + b*x + c, x)
```

[20]:
$$\left[\frac{-b - \sqrt{-4ac + b^2}}{2a},\ \frac{-b + \sqrt{-4ac + b^2}}{2a}\right]$$

Também podemos isolar uma variável em uma fórmula. Por exemplo, isolar r em $V = \frac{4}{3}\pi r^3$:

[21]:
```
V, r = sp.symbols('V r', positive=True)
sp.solve(sp.Eq(V, sp.Rational(4, 3)*sp.pi*r**3), r)
```

[21]:
$$\left[\frac{\sqrt[3]{6}\sqrt[3]{V}}{2\sqrt[3]{\pi}}\right]$$

Usando as soluções. Como `solve` devolve uma lista, cada solução é acessada por índice:

[22]:
```
sol = sp.solve(x**2 - 3*x - 1, x)
for s in sol:
    print(s, "~", s.evalf(6), "  verificação:", sp.simplify((x**2 - 3*x -
    ↪ 1).subs(x, s)))
```

```
3/2 - sqrt(13)/2 ~ -0.302776   verificação: 0
3/2 + sqrt(13)/2 ~ 3.30278   verificação: 0
```

7.4.2 `solveset`: soluções em um conjunto

O `sp.solveset(equação, x, domínio)` devolve um *conjunto*, permite escolher o domínio (`sp.S.Reals`, `sp.S.Complexes`) e descreve corretamente conjuntos infinitos de soluções:

[23]:
```
sp.solveset(x**2 + 1, x, sp.S.Reals), sp.solveset(sp.sin(x), x, sp.S.Reals)
```

[23]:
$$(\emptyset,\ \{2n\pi \mid n \in \mathbb{Z}\} \cup \{2n\pi + \pi \mid n \in \mathbb{Z}\})$$

Isto é: $x^2 + 1 = 0$ não tem solução real e $\operatorname{sin} x = 0$ para $x = k\pi$, $k \in \mathbb{Z}$.

Inequações. Com `solveset` no domínio real também se resolvem inequações:

[24]:
```
sp.solveset(x**2 - 4 > 0, x, sp.S.Reals), sp.solveset((x - 1)/(x + 2) <= 0,
    ↪ x, sp.S.Reals)
```

[24]:
$$\left((-\infty, -2) \cup (2, \infty)\,,\ (-2, 1]\right)$$

7.4.3 Sistemas de equações

Para um sistema, passa-se uma lista de equações e uma lista de incógnitas. Com `dict=True` as soluções vêm como dicionários {variável: valor}.

[25]:
```
sp.solve([x + y - 2, x**2 + y**2 + x - 10], [x, y], dict=True)
```

[25]:
$$\left[\left\{x : \frac{3}{4} - \frac{\sqrt{57}}{4},\ y : \frac{5}{4} + \frac{\sqrt{57}}{4}\right\},\ \left\{x : \frac{3}{4} + \frac{\sqrt{57}}{4},\ y : \frac{5}{4} - \frac{\sqrt{57}}{4}\right\}\right]$$

Para sistemas **lineares** há o `sp.linsolve`, que também descreve sistemas indeterminados:

[26]:
```
eq1 = sp.Eq(x + y - 2*z, 1)
eq2 = sp.Eq(x + y - z, 3)
eq3 = sp.Eq(3*x + y - z, 3)
sp.linsolve([eq1, eq2, eq3], [x, y, z])
```

[26]:
$$\{(0,\ 5,\ 2)\}$$

[27]:
```
sp.linsolve([x + y + z - 1, x - y + 2*z - 3], [x, y, z])     # 2 equações, 3
    ↪ incógnitas
```

[27]:
$$\left\{\left(2 - \frac{3z}{2},\ \frac{z}{2} - 1,\ z\right)\right\}$$

A solução depende de z (variável livre): o sistema é possível e indeterminado, e o conjunto solução é uma reta no espaço.

7.4.4 Quando não há fórmula: `nsolve`

Pelo teorema de Abel–Ruffini (e pela teoria de Galois), não existe fórmula com radicais para as raízes de polinômios gerais de grau ≥ 5. Nesses casos, e em equações como $\cos x = x$, o SymPy não encontra a solução exata e precisamos de métodos **numéricos**. O `sp.nsolve(expr, x, chute)` procura uma solução perto de um valor inicial (usando o método de Newton):

[28]:
```
raizes = sp.solve(x**7 - x**6 + x**3 - 9, x)
print(len(raizes), "raízes")
raizes[0]
```

```
7 raízes
```

[28]:
$$\operatorname{CRootOf}\left(x^7 - x^6 + x^3 - 9, 0\right)$$

O SymPy sabe que há 7 raízes, mas a resposta apenas diz "a raiz de índice 0 do polinômio". Pedimos os valores numéricos:

[29]:
```
print(sp.nsolve(x**7 - x**6 + x**3 - 9, x, 1))
print(sp.real_roots(x**7 - x**6 + x**3 - 9))
print([r.evalf(6) for r in sp.Poly(x**7 - x**6 + x**3 - 9).nroots()])
```

```
print(sp.nsolve(sp.cos(x) - x, x, 0.5))
```

```
1.49827540261252
[CRootOf(x**7 - x**6 + x**3 - 9, 0)]
[1.49828, -1.10809 - 0.623891*I, -1.10809 + 0.623891*I, -0.221851 -
    ↪ 1.27147*I, -0.221851 + 1.27147*I, 1.0808 - 1.03041*I, 1.0808 +
    ↪ 1.03041*I]
0.739085133215161
```

O `nsolve` também resolve sistemas não lineares, com um chute para cada incógnita:

[30]:
```
sp.nsolve([x**5 + y**6 + x**3 - 1, x + y - 1], [x, y], [0.5, 0])
```

[30]:
$$\begin{bmatrix}0.837615759482675\\0.162384240517325\end{bmatrix}$$

Dica

Para escolher um bom chute, faça antes um gráfico (próxima seção): as soluções de $f(x) = 0$ são os pontos onde o gráfico corta o eixo x.

Para quem vem do Maxima

`solve`, `linsolve` e `realroots`/`allroots` do Maxima correspondem a `sp.solve`, `sp.linsolve` e `sp.real_roots`/`sp.Poly(p).nroots()`. O `find_root(cos(x) = x, x, 0, 1)` do Maxima corresponde a `sp.nsolve(sp.cos(x) - x, x, 0.5)`.

7.5 Somatórios e números inteiros

O SymPy calcula somatórios de forma *simbólica*, isto é, encontra fórmulas fechadas:

[31]:
```
k, n = sp.symbols('k n', integer=True, positive=True)
sp.factor(sp.summation(k**2, (k, 1, n)))
```

[31]:
$$\frac{n\,(n+1)\,(2n+1)}{6}$$

[32]:
```
sp.factor(sp.summation(k**3, (k, 1, n))), sp.summation(1/k**2, (k, 1,
    ↪ sp.oo)), sp.summation(sp.Rational(1, 2)**k, (k, 1, sp.oo))
```

[32]:
$$\left(\frac{n^2\,(n+1)^2}{4},\ \frac{\pi^2}{6},\ 1\right)$$

[33]:
```
print(sp.isprime(2**31 - 1), sp.factorint(360), sp.divisors(60))
print(sp.prime(100), list(sp.primerange(1, 30)))
```

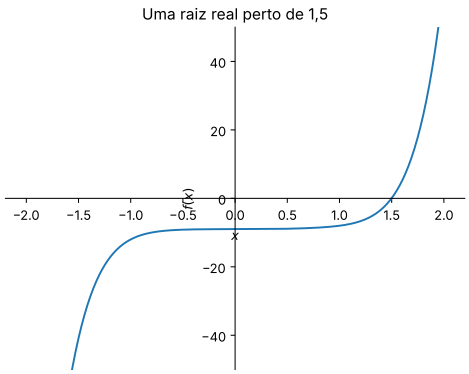
```
True {2: 3, 3: 2, 5: 1} [1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30, 60]
541 [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

O dicionário `{2: 3, 3: 2, 5: 1}` significa $360 = 2^3 \cdot 3^2 \cdot 5$, e `sp.prime(100)` é o centésimo primo. (O `primerange` gera os primos de um intervalo; o `list` serve para vê-los todos.)

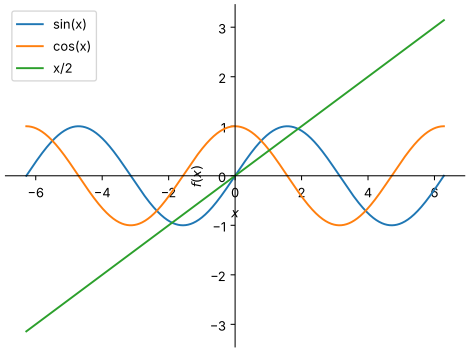
7.6 Gráficos com o SymPy

O SymPy tem seus próprios comandos de gráfico, que recebem diretamente a *expressão* (como o `plot2d` do Maxima). São práticos para gráficos rápidos; para gráficos elaborados, use o Matplotlib (Aula 6) com `sp.lambdify`.

[34]:
```
sp.plot(x**7 - x**6 + x**3 - 9, (x, -2, 2), ylim=(-50, 50), title="Uma raiz
    ↪ real perto de 1,5")
```

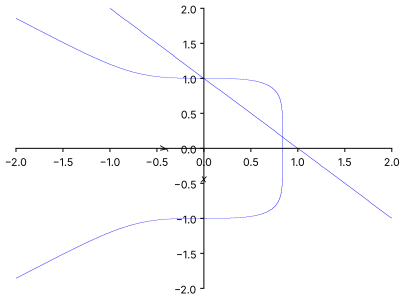


[35]:
```
sp.plot(sp.sin(x), sp.cos(x), x/2, (x, -2*sp.pi, 2*sp.pi), legend=True)
```



Curvas definidas implicitamente, como as do sistema resolvido acima com `nsolve`, são feitas com `sp.plot_implicit`:

[36]:
```
sp.plot_implicit(sp.Or(sp.Eq(x**5 + y**6 + x**3, 1), sp.Eq(x + y, 1)), (x,
    ↪ -2, 2), (y, -2, 2))
```



O ponto de encontro das duas curvas perto de (0,84; 0,16) é a solução encontrada pelo `nsolve`.

Observação

Os gráficos do SymPy também aceitam curvas paramétricas (`sp.plot_parametric`) e superfícies (`from sympy.plotting import plot3d`).

Nota ao professor

Como o material de Maxima da disciplina tem atividades de simplificação, fatoração e resolução de equações, sugere-se retomar aqui duas ou três delas, pedindo que os alunos as resolvam com o SymPy usando a tabela do Apêndice A. Um ponto de atenção recorrente: `1/2*x` no SymPy é `0.5*x` (float); use `x/2` ou `sp.Rational(1, 2)*x`.

7.7 Exercícios

Exercício 7.1 (exato *versus* aproximado)

Calcule de forma exata e com 30 algarismos: (a) $\sqrt{12}+\sqrt{27}$; (b) $\frac{1}{3}+\frac{2}{7}$; (c) $e^{\pi\sqrt{163}}$ (está muito perto de um inteiro!).

Exercício 7.2 (manipulação)

(a) Expanda $(x-2y)^6$. (b) Fatore x^6-1 e x^4-5x^2+4. (c) Simplifique $\frac{x^3-8}{x^2-4}$. (d) Decomponha $\frac{x+1}{x^2(x-1)}$ em frações parciais.

Exercício 7.3 (equações)

Resolva: (a) $x^3-6x^2+11x-6=0$; (b) $2^x=32$; (c) $\sqrt{x+3}=x-3$ (verifique as soluções!); (d) $x^4+x^2-12=0$ nos reais e nos complexos; (e) $|2x-1|<3$.

Exercício 7.4 (sistemas)

Resolva: (a) $\begin{cases} x+2y-z=4 \\ 2x-y+z=1 \\ x+y+z=6 \end{cases}$; (b) $\begin{cases} x^2+y^2=25 \\ x-y=1 \end{cases}$; (c) Encontre a,b,c tais que a parábola $y=ax^2+bx+c$ passe por $(1,2)$, $(2,3)$ e $(3,6)$.

Exercício 7.5 (numérico)

Faça o gráfico de $f(x) = e^{-x} - x^2 + 2$ e use `nsolve` para encontrar todas as raízes reais.

Exercício 7.6 (somatórios)

Encontre fórmulas fechadas para $\sum_{k=1}^{n}(2k-1)$, $\sum_{k=1}^{n} k(k+1)$ e $\sum_{k=0}^{n} 2^k$. Calcule $\sum_{k=1}^{\infty} \frac{1}{k(k+1)}$ e $\sum_{k=0}^{\infty} \frac{1}{k!}$.

CAPÍTULO 8

SymPy II: limites, derivadas e séries de Taylor

Objetivos da aula

- calcular limites (em um ponto, laterais e no infinito) com `sp.limit`;
- calcular derivadas pela definição e com `sp.diff`: ordem superior, derivadas parciais, derivação implícita;
- encontrar retas tangentes, pontos críticos, máximos e mínimos, e resolver problemas de otimização;
- obter polinômios de Taylor com `sp.series` e visualizar as aproximações.

Roteiro sugerido

Tempo	Atividade
0–20 min	Limites: `sp.limit`, limites laterais, no infinito, limites que não existem.
20–30 min	Derivada pela definição; `sp.diff`; ordem superior; derivadas parciais.
30–45 min	Reta tangente (com gráfico); derivação implícita.
45–65 min	Pontos críticos, máximos e mínimos; um problema de otimização.
65–80 min	Polinômios de Taylor e gráficos das aproximações.
80–90 min	Exercícios.

Em todas as células desta aula supomos executado:

[1]:
```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
x, h = sp.symbols('x h')
```

8.1 Limites

Sintaxe

```
sp.limit(expressão, variável, ponto)          # sem direção: pela direita (ver
    ↪ Atenção)
sp.limit(expressão, variável, ponto, '+-')    # bilateral
sp.limit(expressão, variável, ponto, '+')     # pela direita
sp.limit(expressão, variável, ponto, '-')     # pela esquerda
sp.limit(expressão, variável, sp.oo)          # x -> infinito
```

Motivação numérica. Uma tabela de valores sugere que $\frac{\sin x}{x} \to 1$ quando $x \to 0$, mas não prova nada; o SymPy calcula o limite *exato*.

[2]:
```
for xv in [0.1, 0.01, 0.001, 0.0001]:
    print(xv, np.sin(xv) / xv)
sp.limit(sp.sin(x)/x, x, 0)
```

```
0.1 0.9983341664682815
0.01 0.9999833334166665
0.001 0.9999998333333416
0.0001 0.9999999983333334
```

[2]:
$$1$$

[3]:
```
sp.limit((x**2 - 4)/(x - 2), x, 2), sp.limit((1 - sp.cos(x))/x**2, x, 0),
    ↪ sp.limit((sp.exp(x) - 1)/x, x, 0)
```

[3]:
$$\left(4,\ \frac{1}{2},\ 1\right)$$

Limites no infinito. O infinito é `sp.oo` (dois "o" minúsculos).

[4]:
```
n = sp.symbols('n')
sp.limit((1 + 1/n)**n, n, sp.oo), sp.limit((3*x**2 + x)/(2*x**2 - 5), x,
    ↪ sp.oo), sp.limit(x*sp.exp(-x), x, sp.oo)
```

[4]:
$$\left(e,\ \frac{3}{2},\ 0\right)$$

Limites laterais e limites que não existem.

[5]:
```
sp.limit(1/x, x, 0, '+'), sp.limit(1/x, x, 0, '-'), sp.limit(sp.Abs(x)/x, x,
    ↪ 0, '+'), sp.limit(sp.Abs(x)/x, x, 0, '-')
```

[5]:
$$(\infty,\ -\infty,\ 1,\ -1)$$

Atenção

Sem o quarto argumento, o `sp.limit` calcula o limite **pela direita** (o padrão é `'+'`). Para o limite bilateral use `'+-'`: se os limites laterais forem diferentes, o SymPy avisa com um erro.

[6]:
```
print(sp.limit(1/x, x, 0))          # padrão: pela direita!
print(sp.limit(x**2, x, 3, '+-'))
```

```
oo
9
```

[7]:
```
sp.limit(sp.Abs(x)/x, x, 0, '+-')
```

```
ValueError: The limit does not exist since left hand limit = -1 and right
    ↪ hand limit = 1
```

Funções definidas por partes e continuidade. O comando `sp.Piecewise` define funções por partes. Vamos verificar se $g(x) = \begin{cases} x^2 + 1, & x < 1 \\ 3x - 1, & x \geq 1 \end{cases}$ é contínua em $x = 1$:

[8]:
```
g = sp.Piecewise((x**2 + 1, x < 1), (3*x - 1, True))
sp.limit(g, x, 1, '-'), sp.limit(g, x, 1, '+'), g.subs(x, 1)
```

[8]:
$$(2,\ 2,\ 2)$$

Os limites laterais e o valor coincidem: g é contínua em $x = 1$.

Escrevendo o limite sem calcular. `sp.Limit` (com L maiúsculo) cria o limite "por extenso", útil para mostrar a pergunta e a resposta; `.doit()` o calcula.

[9]:
```
L = sp.Limit(sp.sin(3*x)/sp.tan(5*x), x, 0)
sp.Eq(L, L.doit())
```

[9]:
$$\lim_{x\to 0^+}\left(\frac{\sin(3x)}{\tan(5x)}\right) = \frac{3}{5}$$

Para quem vem do Maxima

`limit(sin(x)/x, x, 0)`, `limit(1/x, x, 0, plus)` e `limit(f, x, inf)` do Maxima correspondem a `sp.limit(sp.sin(x)/x, x, 0)`, `sp.limit(1/x, x, 0, '+')` e `sp.limit(f, x, sp.oo)`. Atenção: no Maxima, `limit` sem direção é bilateral; no SymPy, é pela direita.

8.2 Derivadas

8.2.1 Pela definição

A derivada é o limite $f'(x) = \lim_{h\to 0}\dfrac{f(x+h)-f(x)}{h}$. Com o SymPy:

[10]:
```
f = x**3
sp.limit((f.subs(x, x + h) - f)/h, h, 0)
```

[10]:
$$3x^2$$

[11]:
```
f = sp.sqrt(x)
quociente = (f.subs(x, x + h) - f)/h
print(quociente)
sp.limit(quociente, h, 0)
```

```
(-sqrt(x) + sqrt(h + x))/h
```

[11]:
$$\frac{1}{2\sqrt{x}}$$

8.2.2 O comando `diff`

Sintaxe

```
sp.diff(f, x)          # f'(x)
sp.diff(f, x, 2)       # f''(x)   (ou sp.diff(f, x, x))
sp.diff(f, x, y)       # derivada parcial mista: primeiro em x, depois em y
f.diff(x)              # forma alternativa
```

[12]:
```
sp.diff(x**2 + x, x), sp.diff(sp.exp(x**3 + x) + 1/x, x),
    ↪ sp.diff(sp.atan(x), x)
```

[12]:
$$\left(2x+1,\ \left(3x^2+1\right)e^{x^3+x}-\frac{1}{x^2},\ \frac{1}{x^2+1}\right)$$

[13]:
```
f = x**2 * sp.sin(x)
[sp.diff(f, x, k) for k in range(1, 4)]
```

[13]:
$$\left[x^2\cos(x)+2x\sin(x),\ -x^2\sin(x)+4x\cos(x)+2\sin(x),\ -x^2\cos(x)-6x\sin(x)+6\cos(x)\right]$$

Derivada em um ponto. Deriva-se primeiro e depois substitui-se o ponto:

[14]:
```
f = x**3 - 2*x
df = sp.diff(f, x)
df, df.subs(x, 2)
```

[14]:
$$\left(3x^2-2,\ 10\right)$$

Atenção

`sp.diff(f.subs(x, 2), x)` dá 0: substituindo antes, f vira um número, e a derivada de uma constante é zero.

Simplificando o resultado. As derivadas costumam vir "feias". Para melhorar a forma do resultado, use `sp.simplify` ou `sp.factor`:

[15]:
```
f = (x**2 - 1)/(x**2 + 1)
d = sp.diff(f, x)
d, sp.simplify(d)
```

[15]:
$$\left(-\frac{2x\left(x^2-1\right)}{\left(x^2+1\right)^2}+\frac{2x}{x^2+1},\ \frac{4x}{\left(x^2+1\right)^2}\right)$$

Derivadas parciais.

[16]:
```
y = sp.symbols('y')
F = sp.cos(x*y) + (3*x + y)/(y**2 + 1)
sp.diff(F, x), sp.diff(F, x, y)        # F_x e F_xy
```

[16]:
$$\left(-y\sin(xy)+\frac{3}{y^2+1},\ -\left(xy\cos(xy)+\frac{6y}{\left(y^2+1\right)^2}+\sin(xy)\right)\right)$$

Derivação implícita. Para uma curva $F(x,y)=0$, o comando `sp.idiff(F, y, x)` calcula $\frac{dy}{dx}$. Para o círculo $x^2+y^2=25$:

[17]:
```
dydx = sp.idiff(x**2 + y**2 - 25, y, x)
dydx, dydx.subs({x: 3, y: 4})
```

[17]:
$$\left(-\frac{x}{y},\ -\frac{3}{4}\right)$$

Para quem vem do Maxima

`diff(f, x)`, `diff(f, x, 2)` e `diff(f, x, 1, y, 1)` correspondem a `sp.diff(f, x)`, `sp.diff(f, x, 2)` e `sp.diff(f, x, y)`. A derivada em um ponto, `at(diff(f,x), x=2)` ou `ev(diff(f,x), x=2)`, é `sp.diff(f, x).subs(x, 2)`.

8.3 Reta tangente

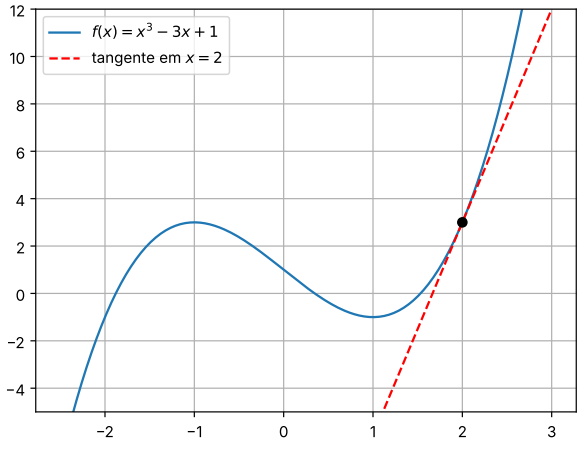
A reta tangente ao gráfico de f em $x = a$ é $y = f(a) + f'(a)(x - a)$. Vamos escrever uma função que a calcula e fazer o gráfico com o Matplotlib, convertendo as expressões com `sp.lambdify`.

[18]:
```
def tangente(f, a):
    return sp.expand(f.subs(x, a) + sp.diff(f, x).subs(x, a)*(x - a))

f = x**3 - 3*x + 1
T = tangente(f, 2)
print("reta tangente em x = 2:  y =", T)

xs = np.linspace(-2.5, 3, 300)
plt.plot(xs, sp.lambdify(x, f)(xs), label=r"$f(x)=x^3-3x+1$")
plt.plot(xs, sp.lambdify(x, T)(xs), "r--", label="tangente em $x=2$")
plt.plot(2, f.subs(x, 2), "ko")
plt.ylim(-5, 12)
plt.legend()
plt.grid(True)
plt.show()
```

```
reta tangente em x = 2:  y = 9*x - 15
```



8.4 Máximos e mínimos

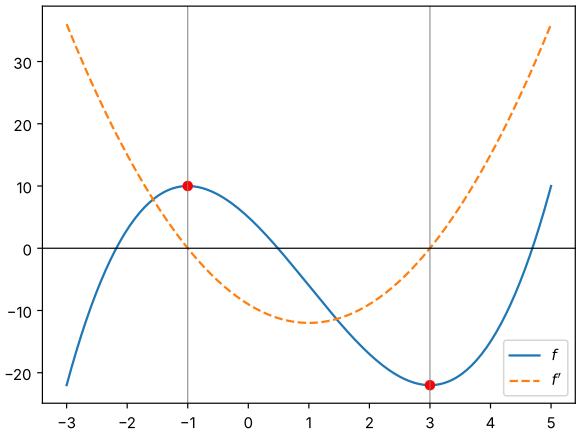
Os pontos críticos de f são as soluções de $f'(x) = 0$. Pelo teste da segunda derivada, se $f''(c) > 0$ há um mínimo local em c, e se $f''(c) < 0$, um máximo local.

[19]:
```
f = x**3 - 3*x**2 - 9*x + 5
df = sp.diff(f, x)
d2f = sp.diff(f, x, 2)
criticos = sp.solve(df, x)
```

```
print("f'(x) =", df, "   pontos críticos:", criticos)
for c in criticos:
    tipo = "mínimo local" if d2f.subs(x, c) > 0 else "máximo local"
    print(f"x = {c}: f''(x) = {d2f.subs(x, c)},  f(x) = {f.subs(x, c)}  ->
    ↪ {tipo}")
```

```
f'(x) = 3*x**2 - 6*x - 9    pontos críticos: [-1, 3]
x = -1: f''(x) = -12,  f(x) = 10  ->  máximo local
x = 3: f''(x) = 12,  f(x) = -22  ->  mínimo local
```

[20]:
```
xs = np.linspace(-3, 5, 300)
plt.plot(xs, sp.lambdify(x, f)(xs), label="$f$")
plt.plot(xs, sp.lambdify(x, df)(xs), "--", label="$f'$")
for c in criticos:
    plt.plot(c, f.subs(x, c), "ro")
    plt.axvline(c, color="gray", lw=0.6)
plt.axhline(0, color="black", lw=0.8)
plt.legend()
plt.show()
```



Observe que f' se anula exatamente nos pontos de máximo e de mínimo de f, que $f' > 0$ onde f cresce e $f' < 0$ onde f decresce.

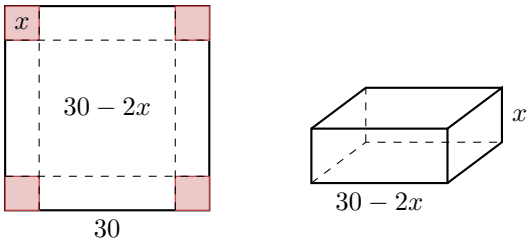
Extremos absolutos em um intervalo fechado. Em $[a, b]$, compara-se o valor de f nos pontos críticos de (a, b) e nos extremos a e b:

[21]:
```
a, b = -2, 6
candidatos = [a, b] + [c for c in criticos if a < c < b]
valores = {c: f.subs(x, c) for c in candidatos}
print(valores)
print("máximo absoluto:", max(valores.values()), "  mínimo absoluto:",
    ↪ min(valores.values()))
```

```
{-2: 3, 6: 59, -1: 10, 3: -22}
máximo absoluto: 59   mínimo absoluto: -22
```

Problema de otimização. De uma folha quadrada de papelão de lado 30 cm, cortam-se quadrados de lado x nos cantos e dobram-se as abas para formar uma caixa sem tampa. Qual x dá o maior volume?

O volume é $V(x) = x(30 - 2x)^2$, com $0 < x < 15$:



[22]:
```
V = x*(30 - 2*x)**2
crit = sp.solve(sp.diff(V, x), x)
print("pontos críticos:", crit)
print("V''(5) =", sp.diff(V, x, 2).subs(x, 5), "  V(5) =", V.subs(x, 5))
```

```
pontos críticos: [5, 15]
V''(5) = -120   V(5) = 2000
```

O ponto $x = 15$ está fora do domínio (volume nulo); em $x = 5\,\text{cm}$ a segunda derivada é negativa, e o volume máximo é $2000\ \text{cm}^3$.

8.5 Polinômios de Taylor

O polinômio de Taylor de ordem n de f em torno de x_0 é

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(x_0)}{k!}(x - x_0)^k.$$

O comando `sp.series(f, x, x0, n)` devolve a expansão até a ordem $n-1$ mais o termo de erro $O\big((x-x_0)^n\big)$; `.removeO()` retira o termo de erro, deixando o polinômio.

[23]:
```
sp.series(sp.exp(x), x, 0, 6)
```

[23]:
$$1 + x + \frac{x^2}{2} + \frac{x^3}{6} + \frac{x^4}{24} + \frac{x^5}{120} + O\left(x^6\right)$$

[24]:
```
sp.series(sp.cos(x), x, 0, 10).removeO()
```

[24]:
$$\frac{x^8}{40320} - \frac{x^6}{720} + \frac{x^4}{24} - \frac{x^2}{2} + 1$$

[25]:
```
sp.series(sp.log(x), x, 1, 5)          # em torno de x0 = 1
```

[25]:
$$-1 - \frac{(x-1)^2}{2} + \frac{(x-1)^3}{3} - \frac{(x-1)^4}{4} + x + O\left((x-1)^5; x \to 1\right)$$

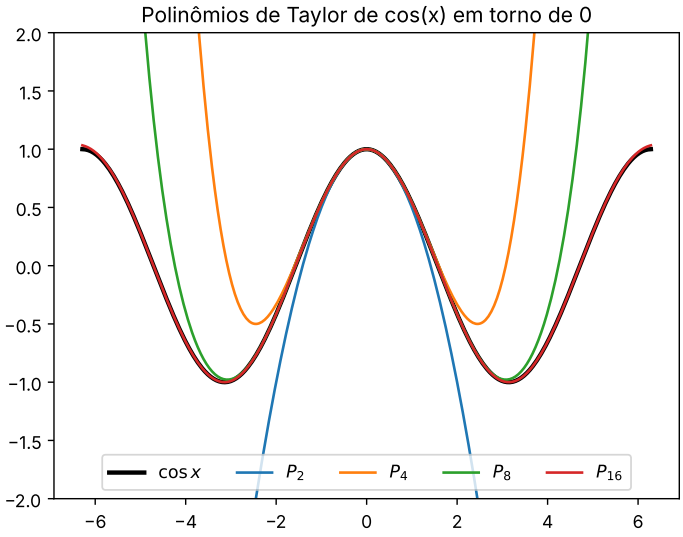
Aproximando valores. Como no tutorial do R. Miranda, usamos o polinômio de Taylor de e^x para aproximar e^2 por um número racional:

[26]:
```
P = sp.series(sp.exp(x), x, 0, 10).removeO()
aprox = P.subs(x, 2)
print(aprox, "~", aprox.evalf(10), "   e^2 =", sp.exp(2).evalf(10))
```

```
20947/2835 ~ 7.388712522    e^2 = 7.389056099
```

Visualizando as aproximações. Quanto maior a ordem, maior o intervalo em que P_n se parece com f:

[27]:
```
xs = np.linspace(-2*np.pi, 2*np.pi, 400)
plt.plot(xs, np.cos(xs), "k", lw=2.5, label=r"$\cos x$")
for n in [2, 4, 8, 16]:
    Pn = sp.series(sp.cos(x), x, 0, n + 1).removeO()
    plt.plot(xs, sp.lambdify(x, Pn)(xs), label=f"$P_{{{n}}}$")
plt.ylim(-2, 2)
plt.legend(loc="lower center", ncol=5)
plt.title("Polinômios de Taylor de cos(x) em torno de 0")
plt.show()
```



Observação

No f-string acima, `f"$P_{{{n}}}$"` produz o texto `$P_{16}$`: as chaves duplas `{{` e `}}` viram chaves literais, e `{n}` é substituído pelo valor de n.

Para quem vem do Maxima

`taylor(cos(x), x, 0, 8)` do Maxima corresponde a `sp.series(sp.cos(x), x, 0, 9)`: no SymPy o último argumento é a ordem do termo de erro, uma unidade acima do grau do polinômio.

Nota ao professor

Um erro comum é reutilizar o nome `f` ou `x` para coisas diferentes (por exemplo, `x` como *array* do NumPy e depois como símbolo). Nos códigos desta aula o *array* é sempre `xs`, e `x` é sempre o símbolo. Vale chamar a atenção para isso. Se sobrar tempo, o método de Newton $x_{k+1} = x_k - f(x_k)/f'(x_k)$ é uma boa ponte com a Aula 3 (o método babilônico é o caso $f(x) = x^2 - a$).

8.6 Exercícios

Exercício 8.1 (limites)

Calcule: (a) $\lim_{x\to 0}\dfrac{\tan x - x}{x^3}$; (b) $\lim_{x\to\infty}\left(1+\dfrac{2}{x}\right)^x$; (c) $\lim_{x\to 0^+} x\ln x$; (d) $\lim_{x\to\infty}\left(\sqrt{x^2+x}-x\right)$; (e) $\lim_{x\to 3}\dfrac{\sqrt{x+1}-2}{x-3}$.

Exercício 8.2 (continuidade)

Encontre o valor de k que torna contínua em $x=2$ a função $f(x)=\begin{cases} kx^2, & x\le 2 \\ 2x+k, & x>2 \end{cases}$. (Dica: resolva com `sp.solve` a equação "limite à esquerda = limite à direita".)

Exercício 8.3 (derivadas)

Derive e simplifique: (a) x^x; (b) $\ln\left(\frac{1+x}{1-x}\right)$; (c) $e^{-x^2}\cos(3x)$; (d) $\arcsin(\sqrt{x})$. Calcule também a derivada de (c) em $x=0$.

Exercício 8.4 (tangente e normal)

Encontre as retas tangente e normal ao gráfico de $f(x)=\frac{x}{1+x^2}$ em $x=1$ e faça um gráfico com a curva e as duas retas (use `plt.axis("equal")` para que a normal pareça perpendicular).

Exercício 8.5 (estudo de função)

Para $f(x)=x^4-4x^3+10$: encontre pontos críticos, intervalos de crescimento, extremos locais, pontos de inflexão (onde f'' muda de sinal) e faça o gráfico de f, f' e f''.

Exercício 8.6 (otimização)

Uma lata cilíndrica deve ter volume de 1 litro ($1000\,\text{cm}^3$). Que raio e altura minimizam a área total de material $A=2\pi r^2+2\pi rh$? (Use $h=\frac{1000}{\pi r^2}$.)

Exercício 8.7 (extra: Taylor e erro)

Encontre o polinômio de Taylor de ordem 5 de $\sin x$ em torno de 0, use-o para aproximar $\sin(0{,}5)$ e compare o erro com a estimativa $\frac{|x|^7}{7!}$.

CAPÍTULO 9

SymPy III: integrais e aplicações

Objetivos da aula

- calcular integrais indefinidas, definidas e impróprias com `sp.integrate`;
- reconhecer quando não há primitiva elementar e calcular o valor numérico da integral;
- calcular áreas entre curvas e volumes de sólidos de revolução;
- visualizar somas de Riemann e obter a integral como limite dessas somas;
- calcular integrais duplas em retângulos e em regiões mais gerais.

Roteiro sugerido

Tempo	Atividade
0–20 min	Integrais indefinidas e definidas; verificação por derivação; o objeto `sp.Integral`.
20–30 min	Integrais impróprias; integrais sem primitiva elementar e valores numéricos.
30–45 min	Somas de Riemann (gráfico e limite).
45–65 min	Área entre curvas; volume de sólidos de revolução.
65–75 min	Integrais duplas.
75–90 min	Exercícios.

[1]:
```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
x, y = sp.symbols('x y')
```

9.1 Integrais indefinidas e definidas

Sintaxe

```
sp.integrate(f, x)                  # primitiva (sem a constante C)
sp.integrate(f, (x, a, b))          # integral definida de a até b
sp.Integral(f, (x, a, b))           # integral "por extenso" (não calcula); .doit()
    ↪ calcula
```

[2]:
```
sp.integrate(x**2, x), sp.integrate(x*sp.exp(x), x),
    ↪ sp.integrate(sp.exp(x)*sp.cos(x), x)
```

[2]:
$$\left(\frac{x^3}{3},\ (x-1)\,e^x,\ \frac{e^x\sin(x)}{2}+\frac{e^x\cos(x)}{2}\right)$$

Atenção

O SymPy **não escreve a constante de integração**: a resposta é uma primitiva; a família de todas as primitivas é $F(x)+C$.

[3]:
```
sp.integrate(1/(x**2 - 1), x), sp.integrate(sp.sin(x)**3, x),
    ↪ sp.integrate(1/sp.sqrt(1 - x**2), x)
```

[3]:
$$\left(\frac{\log(x-1)}{2} - \frac{\log(x+1)}{2},\ \frac{\cos^3(x)}{3} - \cos(x),\ \operatorname{asin}(x)\right)$$

Observação

O SymPy escreve log para o logaritmo natural ln. Também omite o módulo: $\int \frac{dx}{x}$ aparece como $\log(x)$, e não $\ln|x|$.

Conferindo por derivação. Uma forma de verificar uma primitiva é derivá-la:

[4]:
```
f = x**2 * sp.sin(x)
F = sp.integrate(f, x)
print(F)
sp.simplify(sp.diff(F, x) - f)
```

```
-x**2*cos(x) + 2*x*sin(x) + 2*cos(x)
```

[4]:
$$0$$

Integrais definidas. Pelo Teorema Fundamental do Cálculo, $\int_a^b f(x)\,dx = F(b) - F(a)$. O SymPy faz isso diretamente:

[5]:
```
sp.integrate(x**3 + x, (x, 0, 1)), sp.integrate(sp.sin(x), (x, 0, sp.pi)),
    ↪ sp.integrate(sp.log(x), (x, 1, sp.E))
```

[5]:
$$\left(\frac{3}{4},\ 2,\ 1\right)$$

[6]:
```
I = sp.Integral(x * sp.sqrt(1 + x**2), (x, 0, 2))
sp.Eq(I, I.doit())
```

[6]:
$$\int_0^2 x\sqrt{x^2+1}\,dx = -\frac{1}{3} + \frac{5\sqrt{5}}{3}$$

Cuidado com descontinuidades. O tutorial do R. Miranda propõe "enganar" o SymPy com $\int_0^3 \frac{dx}{x-2}$, cujo integrando tem uma assíntota vertical em $x = 2$, dentro do intervalo. Aplicar a fórmula $F(3) - F(0) = \ln|1| - \ln|-2|$ sem pensar daria $-\ln 2$, o que está **errado**: a integral imprópria diverge.

[7]:
```
sp.integrate(1/(x - 2), (x, 0, 3))
```

[7]:
$$\text{NaN}$$

Para quem vem do Maxima

`integrate(f, x)` e `integrate(f, x, a, b)` do Maxima correspondem a `sp.integrate(f, x)` e `sp.integrate(f, (x, a, b))` (repare nos parênteses em volta de `(x, a, b)`). O Maxima às vezes pergunta "Is x positive, negative or zero?"; no

SymPy declaramos as hipóteses nos símbolos (`sp.symbols('a', positive=True)`).

9.2 Integrais impróprias e valores numéricos

O infinito é `sp.oo`. Integrais impróprias divergentes resultam em `oo`.

[8]:
```
sp.integrate(sp.exp(-x**2), (x, -sp.oo, sp.oo)), sp.integrate(1/x**6, (x, 1,
    ↪ sp.oo)), sp.integrate(1/x, (x, 1, sp.oo))
```

[8]:
$$\left(\sqrt{\pi},\ \frac{1}{5},\ \infty\right)$$

[9]:
```
p = sp.symbols('p', positive=True)
sp.integrate(1/x**p, (x, 1, sp.oo))
```

[9]:
$$\begin{cases} \frac{1}{p-1} & \text{for } p > 1 \\ \int_1^\infty x^{-p}\,dx & \text{otherwise} \end{cases}$$

O SymPy responde "por casos": a integral vale $\frac{1}{p-1}$ se $p > 1$ (e diverge caso contrário).

Funções sem primitiva elementar. Algumas funções simples, como e^{-x^2} e $\frac{\sin x}{x}$, não têm primitiva que se escreva com funções elementares. O SymPy responde com funções especiais (a função erro erf, o seno integral Si) ou devolve a integral sem calcular. Em todos os casos, o valor numérico de uma integral definida pode ser obtido com `evalf`:

[10]:
```
sp.integrate(sp.exp(-x**2), x), sp.integrate(sp.sin(x)/x, x)
```

[10]:
$$\left(\frac{\sqrt{\pi}\,\mathrm{erf}\,(x)}{2},\ \mathrm{Si}\,(x)\right)$$

[11]:
```
J = sp.Integral(sp.sqrt(1 + x**4), (x, 0, 1))
print(J.doit())
print(J.evalf(15))
```

```
gamma(1/4)*hyper((-1/2, 1/4), (5/4,), -1)/(4*gamma(5/4))
1.08942941322482
```

Para $\int_0^1 \sqrt{1+x^4}\,dx$ o SymPy encontrou uma resposta em termos das funções gama (Γ) e hipergeométrica, pouco úteis na prática; o valor numérico, por outro lado, é imediato.

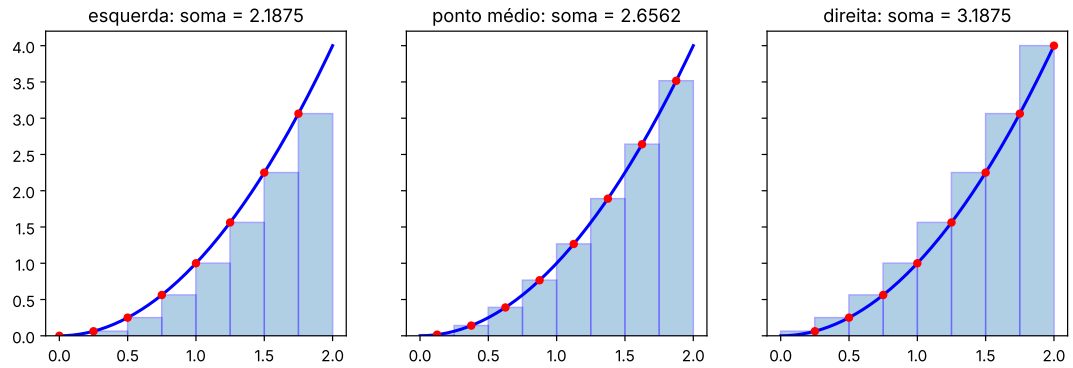
9.3 Somas de Riemann

Dividindo $[a, b]$ em n partes de largura $\Delta x = \frac{b-a}{n}$ e escolhendo um ponto x_i^* em cada parte, a soma de Riemann $\sum_{i=1}^{n} f(x_i^*)\,\Delta x$ aproxima $\int_a^b f$. O código abaixo, adaptado do tutorial do R. Miranda (que por sua vez se baseia no *Mathematical Python*, de P. Walls), desenha as somas pela esquerda, pelo ponto médio e pela direita.

[12]:
```
def f(t):
    return t**2

a, b, N = 0, 2, 8
dx = (b - a) / N
xs = np.linspace(a, b, 200)
esq = np.linspace(a, b - dx, N)          # extremos esquerdos
pontos = {"esquerda": esq, "ponto médio": esq + dx/2, "direita": esq + dx}

fig, ax = plt.subplots(1, 3, figsize=(12, 3.6), sharey=True)
for eixo, (nome, xi) in zip(ax, pontos.items()):
    eixo.plot(xs, f(xs), "b", lw=2)
    eixo.bar(esq, f(xi), width=dx, align="edge", alpha=0.35, edgecolor="b")
    eixo.plot(xi, f(xi), "r.", ms=9)
    soma = np.sum(f(xi) * dx)
    eixo.set_title(f"{nome}: soma = {soma:.4f}")
plt.show()
print("valor exato:", sp.integrate(x**2, (x, 0, 2)))
```




```
valor exato: 8/3
```

Aumentando N, as três somas se aproximam de $\frac{8}{3} \approx 2{,}6667$. O SymPy permite fazer a conta com n genérico e calcular o limite, exatamente como na definição da integral:

[13]:
```
n, i = sp.symbols('n i', integer=True, positive=True)
dx = sp.Rational(2) / n
Sn = sp.summation(((i*dx)**2) * dx, (i, 1, n))       # soma pela direita
print(sp.factor(Sn))
sp.limit(Sn, n, sp.oo)
```

```
4*(n + 1)*(2*n + 1)/(3*n**2)
```

[13]:
$$\frac{8}{3}$$

9.4 Área entre curvas

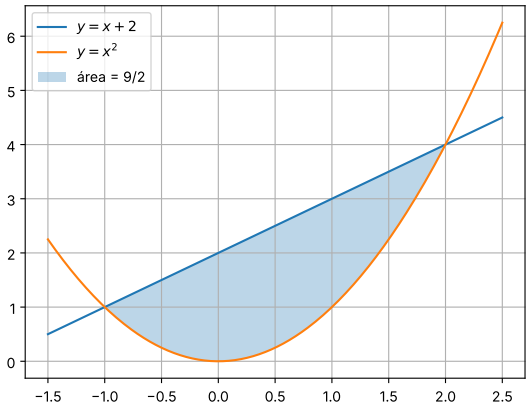
A área da região entre os gráficos de f e g, para $x \in [a, b]$, é $\int_a^b |f(x) - g(x)|\,dx$. Os limites de integração costumam ser os pontos de interseção, obtidos com `solve`. Exemplo: a região entre $y = x^2$ e $y = x + 2$.

[14]:
```
f = x + 2
g = x**2
pts = sp.solve(sp.Eq(f, g), x)
print("interseções em x =", pts)
area = sp.integrate(f - g, (x, pts[0], pts[1]))
area
```

```
interseções em x = [-1, 2]
```

[14]:
$$\frac{9}{2}$$

[15]:
```
xs = np.linspace(-1.5, 2.5, 300)
F, G = sp.lambdify(x, f), sp.lambdify(x, g)
plt.plot(xs, F(xs), label="$y=x+2$")
plt.plot(xs, G(xs), label="$y=x^2$")
xr = np.linspace(-1, 2, 200)
plt.fill_between(xr, G(xr), F(xr), alpha=0.3, label="área = 9/2")
plt.legend()
plt.grid(True)
plt.show()
```



Atenção

Se as curvas se cruzam dentro do intervalo, $\int_a^b (f-g)$ *não* é a área: as partes em que $g > f$ entram com sinal negativo. Divida o intervalo nos pontos de interseção ou integre `sp.Abs(f - g)`.

[16]:
```
# área entre sen(x) e cos(x) em [0, pi]: as curvas se cruzam em pi/4
errado = sp.integrate(sp.sin(x) - sp.cos(x), (x, 0, sp.pi))
certo = sp.integrate(sp.cos(x) - sp.sin(x), (x, 0, sp.pi/4)) +
    ↪ sp.integrate(sp.sin(x) - sp.cos(x), (x, sp.pi/4, sp.pi))
errado, certo
```

[16]:
$$\left(2,\ 2\sqrt{2}\right)$$

9.5 Volumes de sólidos de revolução

Girando em torno do eixo x a região sob o gráfico de $f \geq 0$, $x \in [a, b]$, obtém-se um sólido cujas seções transversais são discos de raio $f(x)$. Seu volume é

$$V = \pi \int_a^b f(x)^2 \, dx \qquad \text{(método dos discos)}.$$

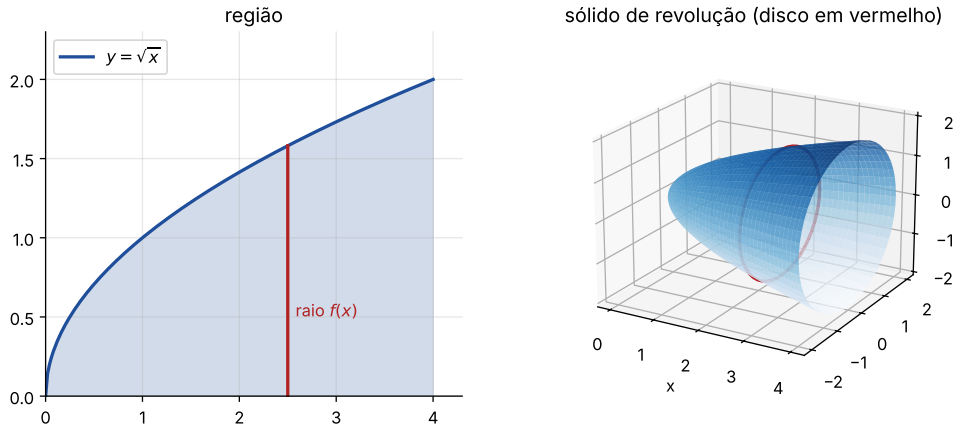


Figura 9.1: Região sob $y = \sqrt{x}$, $0 \leq x \leq 4$, e o sólido obtido ao girá-la em torno do eixo x (um paraboloide).

[17]:
```
V = sp.pi * sp.integrate(sp.sqrt(x)**2, (x, 0, 4))
V
```

[17]: 8π

Com um raio simbólico, deduzimos o volume da esfera girando o semicírculo $y = \sqrt{r^2 - x^2}$:

[18]:
```
r = sp.symbols('r', positive=True)
sp.pi * sp.integrate(sp.sqrt(r**2 - x**2)**2, (x, -r, r))
```

[18]: $\dfrac{4\pi r^3}{3}$

Comprimento de arco. O comprimento do gráfico de f entre a e b é $\int_a^b \sqrt{1 + f'(x)^2}\, dx$. Para $y = \frac{2}{3}x^{3/2}$, $0 \leq x \leq 3$:

[19]:
```
f = sp.Rational(2, 3) * x**sp.Rational(3, 2)
sp.integrate(sp.sqrt(1 + sp.diff(f, x)**2), (x, 0, 3))
```

[19]: $\dfrac{14}{3}$

9.6 Integrais duplas

Integrais iteradas são feitas indicando várias variáveis, **da mais interna para a mais externa**: `sp.integrate(f, (y, c, d), (x, a, b))` calcula $\int_a^b \left(\int_c^d f \, dy \right) dx$.

[20]:
```
sp.integrate(x*y**2, (y, 0, 2), (x, 0, 3))          # retângulo [0,3] x [0,2]
```

[20]:
$$12$$

Em uma região do tipo $\{(x,y) : a \le x \le b,\ g_1(x) \le y \le g_2(x)\}$, os limites da integral interna dependem de x. Por exemplo, a região entre $y = x^2$ e $y = x$:

[21]:
```
sp.integrate(x + y, (y, x**2, x), (x, 0, 1))
```

[21]:
$$\frac{3}{20}$$

A área de uma região é a integral dupla de 1. Em coordenadas polares, $dA = r\,dr\,d\theta$: a área do círculo de raio R é

[22]:
```
rho, theta, R = sp.symbols('rho theta R', positive=True)
sp.integrate(rho, (rho, 0, R), (theta, 0, 2*sp.pi))
```

[22]:
$$\pi R^2$$

Nota ao professor

A comparação "errado *versus* certo" na área entre seno e cosseno costuma gerar uma boa discussão. Se a turma ainda não viu integrais duplas, a última seção pode ser omitida ou apenas mostrada. Uma atividade alternativa é retomar a regra dos trapézios da Aula 4 e compará-la com o valor exato do SymPy.

9.7 Exercícios

Exercício 9.1 (primitivas)

Calcule e confira derivando: (a) $\int x^2 e^{-x}\,dx$; (b) $\int \frac{x+3}{x^2+x-2}\,dx$; (c) $\int \sec^2 x\,dx$; (d) $\int x \ln x\,dx$; (e) $\int \frac{dx}{1+4x^2}$.

Exercício 9.2 (definidas e impróprias)

Calcule: (a) $\int_0^{\pi/2} \sin^2 x\,dx$; (b) $\int_1^e \frac{\ln x}{x}\,dx$; (c) $\int_0^\infty x e^{-x}\,dx$; (d) $\int_0^1 \frac{dx}{\sqrt{x}}$; (e) $\int_0^1 \frac{dx}{x}$.

Exercício 9.3 (área)

Calcule a área da região limitada por $y = x^3 - x$ e $y = 3x$ e faça um gráfico da região.

Exercício 9.4 (Riemann)

Para $f(x) = \sqrt{x}$ em $[0,4]$, calcule numericamente as somas pela esquerda, direita e ponto médio para $N = 10, 100, 1000$ e compare com o valor exato $\frac{16}{3}$. Qual converge mais rápido?

Exercício 9.5 (volume)

Calcule o volume do sólido obtido girando em torno do eixo x a região sob $y = e^{-x}$, $x \ge 0$. Calcule também o volume do cone obtido girando $y = \frac{R}{h}x$, $0 \le x \le h$, e confira que é $\frac{1}{3}\pi R^2 h$.

Exercício 9.6 (extra: valor médio e integral dupla)

(a) O valor médio de f em $[a, b]$ é $\frac{1}{b-a}\int_a^b f$. Calcule o valor médio de $\operatorname{sin} x$ em $[0, \pi]$. (b) Calcule $\iint_D xy\, dA$, em que D é o triângulo de vértices $(0, 0)$, $(1, 0)$ e $(1, 1)$.

CAPÍTULO 10

SymPy IV: matrizes simbólicas e equações diferenciais

Objetivos da aula

- trabalhar com matrizes exatas no SymPy: determinante, inversa, escalonamento (`rref`), posto, núcleo, autovalores, autovetores e diagonalização;
- discutir sistemas lineares com parâmetros;
- resolver equações diferenciais ordinárias com `sp.dsolve`, com e sem condições iniciais;
- visualizar campos de direções e soluções; conhecer a solução numérica de sistemas (*extra*).

Roteiro sugerido

Tempo	Atividade
0–25 min	`sp.Matrix`: operações exatas, `det`, `inv`, `rref`, `rank`, `nullspace`.
25–40 min	Autovalores, autovetores e diagonalização; sistemas com parâmetros.
40–65 min	EDOs com `dsolve`: 1ª e 2ª ordem, condições iniciais, verificação.
65–75 min	Campo de direções; gráfico de soluções.
75–90 min	Atividade de encerramento do curso.

[1]:
```
import sympy as sp
import numpy as np
import matplotlib.pyplot as plt
x, y, z, t = sp.symbols('x y z t')
```

10.1 Matrizes no SymPy

Na Aula 5 usamos o NumPy, que faz contas *aproximadas*. O `sp.Matrix` faz contas *exatas*, com frações, raízes e símbolos, como a função `matrix` do Maxima.

[2]:
```
A = sp.Matrix([[1, 2], [-3, 1]])
B = sp.Matrix([[2], [4]])
C = sp.Matrix([[2, 5], [4, 10]])
A, B, C
```

[2]:
$$\left(\begin{bmatrix}1 & 2\\ -3 & 1\end{bmatrix}, \begin{bmatrix}2\\ 4\end{bmatrix}, \begin{bmatrix}2 & 5\\ 4 & 10\end{bmatrix}\right)$$

Atenção

No SymPy, `A * B` é o **produto de matrizes** (no NumPy era elemento a elemento) e `A**n` é a potência de matrizes. `A @ B` também funciona nos dois.

Matemática	SymPy	Maxima
AB, A^n, A^T	`A*B`, `A**n`, `A.T`	`A.B`, `A^^n`, `transpose(A)`
$\det A$, A^{-1}	`A.det()`, `A.inv()` ou `A**-1`	`determinant(A)`, `invert(A)`
forma escalonada reduzida	`A.rref()`	`echelon(A)` (escalonada)
posto, núcleo	`A.rank()`, `A.nullspace()`	`rank(A)`, `nullspace(A)`
autovalores, autovetores	`A.eigenvals()`, `A.eigenvects()`	`eigenvalues(A)`, `eigenvectors(A)`
polinômio característico	`A.charpoly(x)`	`charpoly(A, x)`
identidade, nula	`sp.eye(n)`, `sp.zeros(m, n)`	`ident(n)`, `zeromatrix(m,n)`

[3]:
```
A*B, A*C - C*A, A.det(), A.inv()
```

[3]:
$$\left(\begin{bmatrix}10\\-2\end{bmatrix},\ \begin{bmatrix}23 & 16\\24 & -23\end{bmatrix},\ 7,\ \begin{bmatrix}\frac{1}{7} & -\frac{2}{7}\\\frac{3}{7} & \frac{1}{7}\end{bmatrix}\right)$$

Compare com a Aula 5: o determinante é exatamente 7 e a inversa tem frações exatas.

Matrizes com símbolos. Podemos obter fórmulas gerais:

[4]:
```
a, b, c, d = sp.symbols('a b c d')
M = sp.Matrix([[a, b], [c, d]])
M.det(), M.inv()
```

[4]:
$$\left(ad - bc,\ \begin{bmatrix}\frac{d}{ad-bc} & -\frac{b}{ad-bc}\\-\frac{c}{ad-bc} & \frac{a}{ad-bc}\end{bmatrix}\right)$$

[5]:
```
k = sp.symbols('k')
K = sp.Matrix([[1, 2, 3], [0, k, 1], [2, 4, k]])
dK = sp.factor(K.det())
print("det =", dK, "   singular para k =", sp.solve(dK, k))
```

```
det = k*(k - 6)    singular para k = [0, 6]
```

Escalonamento. O método `rref()` devolve a forma escalonada reduzida e as colunas dos pivôs. Aplicado à matriz ampliada de um sistema, resolve o sistema "à mão", como em Álgebra Linear:

[6]:
```
Ab = sp.Matrix([[1, 1, -2, 1],
                [1, 1, -1, 3],
                [3, 1, -1, 3]])       # sistema x+y-2z=1, x+y-z=3, 3x+y-z=3
Ab.rref()
```

[6]:
$$\left(\begin{bmatrix}1 & 0 & 0 & 0\\0 & 1 & 0 & 5\\0 & 0 & 1 & 2\end{bmatrix},\ (0,\ 1,\ 2)\right)$$

A última coluna dá a solução $x = 0$, $y = 5$, $z = 2$ (a mesma das Aulas 5 e 7).

Sistemas com parâmetro. Para que valores de k o sistema $\begin{cases} x + 2y + 3z = 1 \\ ky + z = 2 \\ 2x + 4y + kz = 3 \end{cases}$ tem solução única? Pelo cálculo acima, $\det K = k(k-6)$: há solução única para $k \neq 0$ e $k \neq 6$. Nos casos $k = 0$ e $k = 6$, comparamos os postos:

[7]:
```
bvec = sp.Matrix([1, 2, 3])
for kv in [0, 6]:
    Kk = K.subs(k, kv)
    ampliada = Kk.row_join(bvec)
    print(f"k = {kv}: posto(K) = {Kk.rank()}, posto(ampliada) =
    ↪ {ampliada.rank()}")
sp.simplify(K.LUsolve(bvec))          # solução para k diferente de 0 e 6
```

```
k = 0: posto(K) = 2, posto(ampliada) = 3
k = 6: posto(K) = 2, posto(ampliada) = 3
```

[7]:
$$\begin{bmatrix} \frac{k^2-13k+26}{k(k-6)} \\ \frac{2k-13}{k(k-6)} \\ \frac{1}{k-6} \end{bmatrix}$$

Para $k = 0$ e $k = 6$ os postos são diferentes: o sistema é impossível.

Núcleo, imagem e autovetores.

[8]:
```
C.nullspace(), C.columnspace(), C.rank()
```

[8]:
$$\left(\left[\begin{bmatrix} -\frac{5}{2} \\ 1 \end{bmatrix} \right], \left[\begin{bmatrix} 2 \\ 4 \end{bmatrix} \right], 1 \right)$$

[9]:
```
M = sp.Matrix([[3, 0, 0], [0, 3, 0], [0, 1, 1]])
print(M.charpoly(x).as_expr(), "  autovalores:", M.eigenvals())
M.eigenvects()
```

```
x**3 - 7*x**2 + 15*x - 9   autovalores: {3: 2, 1: 1}
```

[9]:
$$\left[\left(1,\ 1,\ \left[\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right] \right), \left(3,\ 2,\ \left[\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 2 \\ 1 \end{bmatrix} \right] \right) \right]$$

A saída de `eigenvals()` é um dicionário {autovalor: multiplicidade}; a de `eigenvects()` é uma lista de triplas (autovalor, multiplicidade, lista de autovetores). Como há 3 autovetores independentes, M é diagonalizável:

[10]:
```
P, D = M.diagonalize()
P, D, P*D*P.inv() == M
```

[10]:
$$\left(\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 2 \\ 1 & 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 3 \end{bmatrix}, \text{True} \right)$$

Aplicação: reconhecimento de cônicas. A parte quadrática de $ax^2+bxy+cy^2+dx+ey+f = 0$ é $\begin{pmatrix} x & y \end{pmatrix} Q \begin{pmatrix} x \\ y \end{pmatrix}$ com $Q = \begin{pmatrix} a & b/2 \\ b/2 & c \end{pmatrix}$. Os sinais dos autovalores de Q dizem o tipo da cônica (mesmo sinal: elipse; sinais opostos: hipérbole; um nulo: parábola, salvo casos degenerados). Para a cônica $x^2 + 2xy - 3y^2 + x + 2y - 4 = 0$ desenhada na Aula 6:

[11]:
```
Q = sp.Matrix([[1, sp.Rational(2, 2)], [sp.Rational(2, 2), -3]])
Q.eigenvals()
```

[11]:
$$\left\{-1+\sqrt{5}:1,\ -\sqrt{5}-1:1\right\}$$

Os autovalores $-1 \pm \sqrt{5}$ têm sinais opostos: é uma hipérbole, como no gráfico.

10.2 Equações diferenciais ordinárias

Uma equação diferencial envolve uma função desconhecida e suas derivadas. No SymPy, a função desconhecida é declarada com `sp.Function`, a derivada $y'(t)$ é escrita `y(t).diff(t)` e a equação é resolvida com `sp.dsolve`.

Sintaxe

```
y = sp.Function('y')
edo = sp.Eq(y(t).diff(t), ...)              # ou uma expressão igualada a zero
sp.dsolve(edo, y(t))                        # solução geral
sp.dsolve(edo, y(t), ics={y(0): 1})         # problema de valor inicial
```

10.2.1 Primeira ordem

Crescimento exponencial. $y' = ky$:

[12]:
```
y = sp.Function('y')
k = sp.symbols('k')
sp.dsolve(sp.Eq(y(t).diff(t), k*y(t)), y(t))
```

[12]:
$$y(t) = C_1 e^{kt}$$

A constante arbitrária aparece como C_1. Com uma condição inicial, ela é determinada:

[13]:
```
sol = sp.dsolve(sp.Eq(y(t).diff(t), k*y(t)), y(t), ics={y(0): 100})
sol
```

[13]:
$$y(t) = 100e^{kt}$$

O lado direito da solução, `sol.rhs`, é uma expressão comum, que pode ser avaliada, derivada ou desenhada:

[14]:
```
print(sol.rhs.subs({k: sp.log(2)/3, t: 6}))     # se a população dobra a
    ↪ cada 3 horas
```

```
400
```

Equação linear de primeira ordem. $y' + 2y = e^t$, $y(0) = 1$:

[15]:
```
edo = sp.Eq(y(t).diff(t) + 2*y(t), sp.exp(t))
sp.dsolve(edo, y(t), ics={y(0): 1})
```

[15]:
$$y(t) = \frac{e^t}{3} + \frac{2e^{-2t}}{3}$$

Equação separável e equação logística. $y' = t\,y^2$ e $P' = rP\left(1 - \frac{P}{K}\right)$:

[16]:
```
sp.dsolve(sp.Eq(y(t).diff(t), t*y(t)**2), y(t))
```

[16]:
$$y(t) = -\frac{2}{C_1 + t^2}$$

[17]:
```
P = sp.Function('P')
r, K = sp.symbols('r K', positive=True)
log_sol = sp.dsolve(sp.Eq(P(t).diff(t), r*P(t)*(1 - P(t)/K)), P(t),
    ↪ ics={P(0): 10})
sp.simplify(log_sol)
```

[17]:
$$P(t) = \frac{10Ke^{rt}}{K + 10e^{rt} - 10}$$

Verificando a solução. O comando `sp.checkodesol` substitui a solução na equação; a resposta `(True, 0)` confirma que ela está correta.

[18]:
```
sp.checkodesol(edo, sp.dsolve(edo, y(t)))
```

[18]:
$$(\text{True},\ 0)$$

10.2.2 Segunda ordem

A segunda derivada é `y(t).diff(t, 2)`. Equação $g'' - 2g' + g = 0$ (raiz dupla da equação característica):

[19]:
```
g = sp.Function('g')
sp.dsolve(g(t).diff(t, 2) - 2*g(t).diff(t) + g(t), g(t))
```

[19]:
$$g(t) = (C_1 + C_2 t)\, e^{t}$$

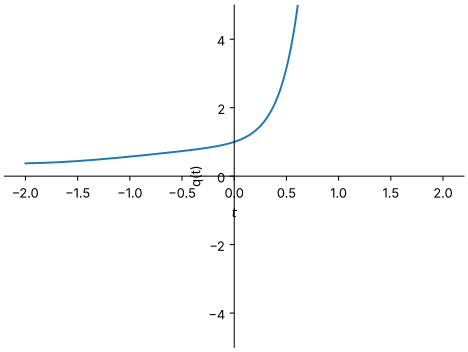
Oscilador harmônico. $x'' + \omega^2 x = 0$ com $x(0) = 1$, $x'(0) = 0$. A condição sobre a derivada é escrita `x(t).diff(t).subs(t, 0)`:

[20]:
```
X = sp.Function('X')
w = sp.symbols('omega', positive=True)
sp.dsolve(X(t).diff(t, 2) + w**2*X(t), X(t), ics={X(0): 1,
    ↪ X(t).diff(t).subs(t, 0): 0})
```

[20]:
$$X(t) = \cos(\omega t)$$

Uma equação não homogênea (tutorial do R. Miranda): $q'' - 6q' + 2q = t\cos t$, com $q(0) = 1$, $q'(0) = 1$. A solução é longa, mas o gráfico é imediato com `sp.plot`:

[21]:
```
q = sp.Function('q')
edo2 = q(t).diff(t, 2) - 6*q(t).diff(t) + 2*q(t) - t*sp.cos(t)
solucao = sp.dsolve(edo2, q(t), ics={q(0): 1, q(t).diff(t).subs(t, 0): 1})
sp.plot(solucao.rhs, (t, -2, 2), ylim=(-5, 5), ylabel="q(t)")
```



Para quem vem do Maxima

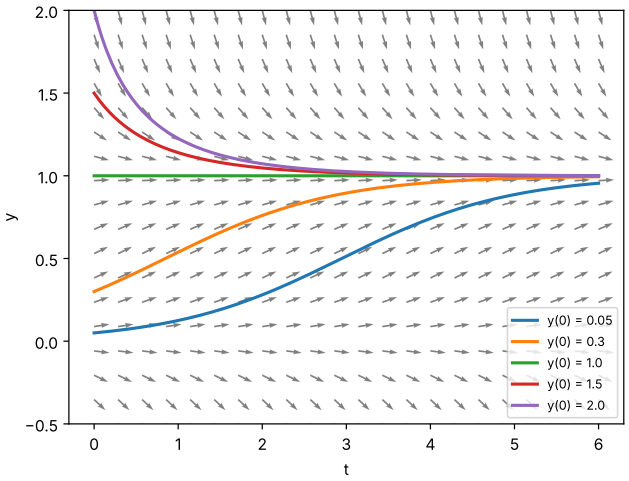
No Maxima: `ode2('diff(y,t) = k*y, y, t)` e, para condições iniciais, `ic1` e `ic2` (ou `desolve` com `atvalue`). No SymPy tudo é feito com `sp.dsolve(..., ics={...})`. Note que no Maxima a derivada da incógnita é escrita com apóstrofo, `'diff(y,t)`; no SymPy é `y(t).diff(t)`.

10.2.3 Campo de direções

Para $y' = F(t, y)$, em cada ponto (t, y) desenhamos um pequeno segmento com inclinação $F(t, y)$. As soluções são curvas tangentes a esses segmentos. O comando `plt.quiver` desenha setas em uma malha. Exemplo: $y' = y(1 - y)$ (logística com $r = K = 1$), com algumas soluções.

[22]:
```
F = lambda T, Y: Y*(1 - Y)
T, Y = np.meshgrid(np.linspace(0, 6, 22), np.linspace(-0.5, 2, 18))
U, V = np.ones_like(T), F(T, Y)
N = np.sqrt(U**2 + V**2)                          # normaliza as setas
plt.quiver(T, Y, U/N, V/N, angles="xy", color="gray", width=0.003)

ts = np.linspace(0, 6, 200)
for y0 in [0.05, 0.3, 1.0, 1.5, 2.0]:
    sol = sp.dsolve(sp.Eq(y(t).diff(t), y(t)*(1 - y(t))), y(t), ics={y(0):
    ↪ y0})
    plt.plot(ts, sp.lambdify(t, sol.rhs)(ts) * np.ones_like(ts), lw=2,
    ↪ label=f"y(0) = {y0}")
plt.ylim(-0.5, 2)
plt.xlabel("t"); plt.ylabel("y")
plt.legend(loc="lower right", fontsize=8)
plt.show()
```



As soluções com $y(0) > 0$ tendem a $y = 1$ (equilíbrio estável); $y = 0$ é um equilíbrio instável.

10.2.4 Extra: sistemas e soluções numéricas

O `dsolve` encontra soluções *exatas*, que nem sempre existem. Para sistemas não lineares, como o famoso sistema de Lorenz

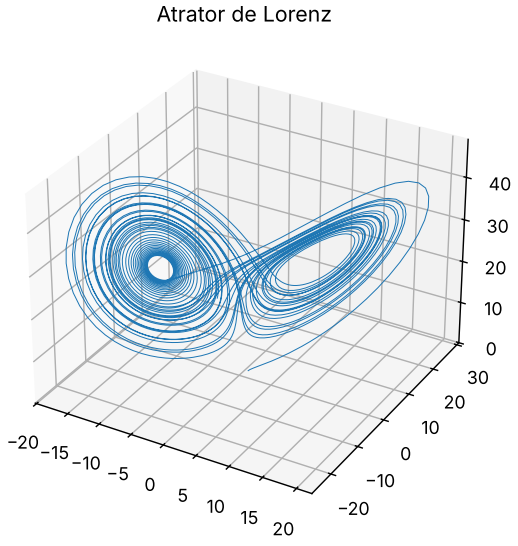
$$\dot{x} = \sigma(y - x), \qquad \dot{y} = x(\rho - z) - y, \qquad \dot{z} = xy - \beta z,$$

usa-se um método numérico, como o `solve_ivp` da biblioteca SciPy (tutorial do R. Miranda):

[23]:
```
from scipy.integrate import solve_ivp

def lorenz(t, u, sigma=10, rho=28, beta=8/3):
    x, y, z = u
    return [sigma*(y - x), x*(rho - z) - y, x*y - beta*z]

sol = solve_ivp(lorenz, (0, 40), [1, 1, 1], max_step=0.01)
ax = plt.figure(figsize=(6, 5)).add_subplot(projection="3d")
ax.plot(sol.y[0], sol.y[1], sol.y[2], lw=0.5)
ax.set_title("Atrator de Lorenz")
plt.show()
```



10.3 Atividade de encerramento

Sugestão para a parte final da aula (ou como trabalho): em duplas, cada grupo escolhe **um** dos temas abaixo e produz um notebook com texto explicativo (células Markdown), código e pelo menos um gráfico.

1. **Estudo completo de uma função**: domínio, limites, assíntotas, derivadas, extremos, inflexões e gráfico (Aulas 6 e 8).

2. **Aproximações de π**: série de Leibniz, Monte Carlo, área do círculo por somas de Riemann e por trapézios; comparar velocidades de convergência (Aulas 3, 4 e 9).

3. **Métodos para equações**: bissecção, método babilônico/Newton e `nsolve`; comparar número de passos (Aulas 3 e 7).

4. **Cônicas**: dada $ax^2 + bxy + cy^2 + dx + ey + f = 0$, classificar pelos autovalores e desenhar (Aulas 6 e 10).

5. **Modelos populacionais**: crescimento exponencial e logístico; ajuste de parâmetros a dados reais (por exemplo, população brasileira nos censos) com `np.polyfit` e `dsolve` (Aulas 5 e 10).

Nota ao professor

Esta aula concentra muito conteúdo; se necessário, a parte de matrizes pode ser apenas apresentada (os alunos já viram NumPy na Aula 5) e o foco ficar nas EDOs. O exemplo de Lorenz requer a SciPy; se ela não estiver instalada nas máquinas, mostre-o projetado. A atividade de encerramento pode substituir uma avaliação ou servir como revisão.

10.4 Exercícios

Exercício 10.1 (matrizes)

Para $A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix}$, calcule de forma exata: $\det A$, A^{-1}, o polinômio característico, os autovalores e os autovetores. Verifique que A é diagonalizável.

Exercício 10.2 (sistema com parâmetro)

Discuta, em função de a, o sistema $\begin{cases} x + y + z = 1 \\ x + ay + z = 2 \\ x + y + az = 3 \end{cases}$: para que valores há solução única? Para os demais valores, o sistema é impossível ou indeterminado?

Exercício 10.3 (EDOs de primeira ordem)

Resolva e verifique com `checkodesol`: (a) $y' = -2ty$, $y(0) = 1$; (b) $y' + y = t$; (c) $y' = \frac{y}{t} + t$.

Exercício 10.4 (lei do resfriamento)

Pela lei de Newton, a temperatura de um café satisfaz $T' = -k(T - 25)$. Se $T(0) = 90$ e $T(5) = 70$ (minutos, °C), encontre k e o instante em que o café chega a 40 °C. Faça o gráfico de $T(t)$.

Exercício 10.5 (segunda ordem)

Resolva $y'' + y' + y = 0$ com $y(0) = 1$, $y'(0) = 0$ (oscilador amortecido) e faça o gráfico para $t \in [0, 15]$. Compare com $y'' + y = 0$ com as mesmas condições iniciais.

Exercício 10.6 (extra: campo de direções)

Desenhe o campo de direções de $y' = t - y$ em $[-2, 3] \times [-2, 3]$ e algumas soluções obtidas com `dsolve`. Que reta é solução da equação?

APÊNDICE A

Do Maxima para o Python

Esta tabela ajuda a adaptar as atividades feitas com o Maxima. Supõe-se que foram executados `import sympy as sp`, `import numpy as np`, `import matplotlib.pyplot as plt` e `x, y, t = sp.symbols('x y t')`.

Tarefa	Maxima	Python (SymPy/NumPy)
Básico		
atribuição	`a: 5;`	`a = 5`
definir função	`f(x) := x^2 + 1;`	`def f(x): return x**2 + 1` ou `f = x**2 + 1` (expressão)
avaliar	`f(2); ev(e, x=2); subst(2, x, e);`	`f(2)` ou `e.subs(x, 2)`
potência	`x^2` ou `x**2`	`x**2` (**nunca** `x^2`)
constantes	`%pi`, `%e`, `%i`, `inf`	`sp.pi`, `sp.E`, `sp.I`, `sp.oo`
valor numérico	`float(e); bfloat(e), fpprec:50;`	`e.evalf()` `sp.N(e, 50)`
fração exata	`1/3`	`sp.Rational(1, 3)` (`1/3` é `float`)
último resultado	`%`	`_` (no Jupyter)
listas	`L: [1, 2, 3]; L[1];`	`L = [1, 2, 3]`; `L[0]` (começa em 0)
gerar lista	`makelist(k^2, k, 1, 10);`	`[k**2 for k in range(1, 11)]`
soma de lista	`apply("+", L); lsum(k, k, L);`	`sum(L)`
Álgebra		
expandir	`expand(e);`	`sp.expand(e)`
fatorar	`factor(e);`	`sp.factor(e)`
simplificar	`ratsimp(e); fullratsimp(e);`	`sp.simplify(e)`, `sp.cancel(e)`
trigonometria	`trigsimp`, `trigexpand`, `trigreduce`	`sp.trigsimp`, `sp.expand_trig`, `sp.fu`
frações parciais	`partfrac(e, x);`	`sp.apart(e, x)`
agrupar	`collectterms(e, x);`	`sp.collect(e, x)`
numerador, denominador	`num(e)`, `denom(e)`	`sp.numer(e)`, `sp.denom(e)`
divisão de polinômios	`divide(p, q, x);`	`sp.div(p, q, x)`
hipóteses	`assume(a > 0);`	`a = sp.symbols('a', positive=True)`
Equações		
resolver	`solve(x^2 = 4, x);`	`sp.solve(sp.Eq(x**2, 4), x)` ou `sp.solve(x**2 - 4, x)`
sistema	`solve([eq1, eq2], [x, y]);`	`sp.solve([eq1, eq2], [x, y], dict=True)`
sistema linear	`linsolve([eq1, eq2], [x, y]);`	`sp.linsolve([eq1, eq2], [x, y])`

Tarefa	Maxima	Python (SymPy/NumPy)
raízes numéricas	`allroots(p); realroots(p);`	`sp.Poly(p, x).nroots()`, `sp.real_roots(p)`, `np.roots(coef)`
raiz numérica	`find_root(f, x, a, b);`	`sp.nsolve(f, x, chute)`
inequação	`fourier_elim` / `solve_rat_ineq`	`sp.solveset(x**2 > 4, x, sp.S.Reals)`
lado direito	`rhs(eq);`	`eq.rhs`
Cálculo		
limite	`limit(f, x, 0);`	`sp.limit(f, x, 0, '+-')` (sem `'+-'`: pela direita)
limite lateral	`limit(f, x, 0, plus); minus`	`sp.limit(f, x, 0, '+')`, `'-'`
derivada	`diff(f, x); diff(f, x, 2);`	`sp.diff(f, x)`, `sp.diff(f, x, 2)`
derivada parcial mista	`diff(f, x, 1, y, 1);`	`sp.diff(f, x, y)`
derivada em ponto	`at(diff(f,x), x=a);`	`sp.diff(f, x).subs(x, a)`
primitiva	`integrate(f, x);`	`sp.integrate(f, x)`
integral definida	`integrate(f, x, a, b);`	`sp.integrate(f, (x, a, b))`
integral numérica	`quad_qags(f, x, a, b);`	`sp.Integral(f, (x, a, b)).evalf()`
somatório	`sum(k^2, k, 1, n), simpsum;`	`sp.summation(k**2, (k, 1, n))`
Taylor	`taylor(f, x, 0, 5);`	`sp.series(f, x, 0, 6)`
EDO	`ode2('diff(y,x) = x*y, y, x);`	`sp.dsolve(sp.Eq(y(x).diff(x), x*y(x)), y(x))`
condição inicial	`ic1(sol, x=0, y=1); ic2(...)`	`sp.dsolve(..., ics={y(0): 1})`
Matrizes		
criar	`A: matrix([1,2],[3,4]);`	`A = sp.Matrix([[1, 2], [3, 4]])` ou `np.array(...)`
elemento	`A[1,2]`	`A[0, 1]` (começa em 0)
produto	`A . B`	`A * B` (SymPy), `A @ B` (NumPy e SymPy)
transposta, inversa	`transpose(A)`, `invert(A)`, `A^^-1`	`A.T`, `A.inv()`
determinante	`determinant(A);`	`A.det()` (SymPy), `np.linalg.det(A)`
escalonar	`echelon(A);`	`A.rref()`
posto, núcleo	`rank(A)`, `nullspace(A)`	`A.rank()`, `A.nullspace()`
autovalores	`eigenvalues(A); eigenvectors(A);`	`A.eigenvals()`, `A.eigenvects()`
identidade	`ident(n)`	`sp.eye(n)`, `np.eye(n)`
Gráficos		
função	`plot2d(f, [x, a, b]);`	`sp.plot(f, (x, a, b))` ou Matplotlib
várias funções	`plot2d([f, g], [x, a, b]);`	`sp.plot(f, g, (x, a, b))`
paramétrica	`plot2d([parametric, cos(t), sin(t), [t,0,2*%pi]]);`	`sp.plot_parametric(sp.cos(t), sp.sin(t), (t, 0, 2*sp.pi))`
implícita	`implicit_plot(x^2+y^2=1, [x,-2,2], [y,-2,2]);`	`sp.plot_implicit(sp.Eq(x**2 + y**2, 1))`
superfície	`plot3d(f, [x,a,b], [y,c,d]);`	`from sympy.plotting import plot3d`; `plot3d(f, (x, a, b), (y, c, d))`
pontos	`plot2d([discrete, xs, ys]);`	`plt.plot(xs, ys, "o")`

Diferenças que mais causam erros

1. **Índices começam em 0** no Python e em 1 no Maxima.
2. **Símbolos precisam ser declarados** no SymPy: `x = sp.symbols('x')`.
3. **Frações**: `1/3` é um número decimal. Use `sp.Rational(1, 3)` quando quiser exatidão.
4. **Potência**: `**`; o `^` não é potência.
5. **Integral definida**: os limites vão em uma tupla, `(x, a, b)`.
6. **Limite sem direção**: no SymPy é pela direita; use `'+-'` para o bilateral.
7. **Prefixos**: `sp.sin` (simbólico), `np.sin` (vetores numéricos) e `math.sin` (um número) são funções diferentes; use cada uma no seu contexto.
8. **Igualdade**: `=` atribui, `==` compara, `sp.Eq(a, b)` escreve a equação $a = b$.

APÊNDICE B

Referência rápida

Python básico (Aulas 1 e 2)

```
# entrada e saída
n = int(input("n = "))
x = float(input("x = "))
print("resultado:", x, n)
print(f"{x:.3f}  {x:e}")

# operações
a + b, a - b, a * b, a / b
a // b      # quociente inteiro
a % b       # resto
a ** b      # potência
abs(a), round(a, 2)

# comparação e lógica
==  !=  <  <=  >  >=
and   or   not

# decisão
if cond:
    ...
elif outra:
    ...
else:
    ...
```

```
# repetição
for i in range(1, n + 1):
    ...
while cond:
    ...
break, continue

# listas
L = [3, 1, 2]
L[0], L[-1], L[1:3]
L.append(5); L.insert(0, 9)
len(L), sum(L), max(L)
sorted(L), L.copy()
[k**2 for k in range(10)]

# tuplas e dicionários
P = (1, 2);  x, y = P
D = {"a": 1};  D["a"]

# funções
def f(x, a=1):
    return a * x**2
g = lambda x: x + 1
```

Módulo `math` e NumPy (Aulas 3 a 5)

```
import math
math.sqrt(x), math.exp(x)
math.log(x), math.log(x, b)
math.sin(x), math.radians(g)
math.pi, math.e
math.floor(x), math.ceil(x)
math.factorial(n)
math.comb(n, k), math.gcd(a, b)
math.isqrt(n)
```

```
import numpy as np
v = np.array([1, 2, 3])
np.arange(a, b, h)
np.linspace(a, b, n)
v * 2, v + v, v @ v
np.sin(v), np.exp(v)
v[v > 0], np.sum(v), v.mean()
np.cumsum(v), np.argmax(v)
```

```
A = np.array([[1, 2], [3, 4]])
A.shape, A[0, 1], A[:, 0], A.T, A @ B
np.linalg.det(A), np.linalg.inv(A), np.linalg.solve(A, b), np.linalg.eig(A)
np.roots([1, -3, 2]), np.polyval(p, x), np.polyfit(x, y, grau)
```

Matplotlib (Aula 6)

```
import matplotlib.pyplot as plt
x = np.linspace(a, b, 400)
plt.plot(x, f(x), "r--", label="f")      # cor/estilo, legenda
plt.title("..."); plt.xlabel("x"); plt.ylabel("y")
plt.xlim(a, b); plt.ylim(c, d); plt.grid(True); plt.legend(); plt.axis("equal")
plt.savefig("figura.png", dpi=200); plt.show()

fig, ax = plt.subplots(1, 2, figsize=(10, 4))   # vários gráficos: ax[0].plot(...)
X, Y = np.meshgrid(x, y); plt.contour(X, Y, Z, levels=[0])   # curvas de nível
ax = plt.figure().add_subplot(projection="3d"); ax.plot_surface(X, Y, Z)
```

SymPy (Aulas 7 a 10)

```
import sympy as sp
x, y = sp.symbols('x y')                        # declarar símbolos
sp.Rational(1, 3), sp.sqrt(2), sp.pi, sp.E, sp.I, sp.oo
e.evalf(20), sp.N(e, 20), e.subs(x, 2), e.subs({x: 1, y: 2})
sp.expand(e), sp.factor(e), sp.simplify(e), sp.apart(e), sp.collect(e, x)
sp.solve(e, x), sp.solve(sp.Eq(a, b), x), sp.solveset(e, x, sp.S.Reals)
sp.linsolve([e1, e2], [x, y]), sp.nsolve(e, x, chute)
sp.limit(f, x, a, '+-'), sp.limit(f, x, sp.oo), sp.diff(f, x, n)
sp.integrate(f, x), sp.integrate(f, (x, a, b)), sp.summation(f, (k, 1, n))
sp.series(f, x, x0, n).removeO()
M = sp.Matrix([[1, 2], [3, 4]]); M.det(), M.inv(), M.rref(), M.eigenvects()
y = sp.Function('y'); sp.dsolve(sp.Eq(y(x).diff(x), y(x)), y(x), ics={y(0): 1})
sp.plot(f, (x, a, b)); sp.lambdify(x, f, "numpy")
```

Mensagens de erro mais comuns

Erro	Causa provável
`SyntaxError`	faltam dois-pontos, parênteses ou aspas; uso de `^`; vírgula decimal
`IndentationError`	recuo faltando ou inconsistente após `if`, `for`, `def`...
`NameError: name 'x' is not defined`	variável/símbolo não criado, célula anterior não executada, erro de digitação
`TypeError`	operação entre tipos incompatíveis (texto + número; `math.sin` de um *array*)
`ValueError`	valor inválido: `int("3.5")`, `math.sqrt(-1)`, limite que não existe
`IndexError`	índice fora da lista (lembre que começa em 0)
`ZeroDivisionError`	divisão por zero
`ModuleNotFoundError`	biblioteca não instalada: `%pip install nome`

APÊNDICE C

Soluções dos exercícios

As soluções abaixo são *uma* forma de resolver cada exercício; outras soluções corretas são possíveis e devem ser valorizadas. Todas as saídas mostradas foram obtidas executando os códigos.

[1]:
```
import math
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt
```

Aula 1

Exercício 1.1.

[2]:
```
C = float(input("Temperatura em °C: "))
F = 9/5*C + 32
print(f"{C} °C = {F:.1f} °F")
```

```
Temperatura em °C: 36.5
36.5 °C = 97.7 °F
```

Exercício 1.2.

[3]:
```
for ano in [1900, 2000, 2024, 2025]:
    bissexto = (ano % 4 == 0 and ano % 100 != 0) or ano % 400 == 0
    print(ano, "é bissexto" if bissexto else "não é bissexto")
```

```
1900 não é bissexto
2000 é bissexto
2024 é bissexto
2025 não é bissexto
```

Exercício 1.3.

[4]:
```
s = 0
for k in range(1, 100, 2):
    s += k
print("(a)", s)

s = 0
for k in range(1, 21):
    s += 1/k**2
print("(b)", s, "  pi^2/6 =", math.pi**2/6)

n = 10
s = 0
for k in range(1, n + 1):
    s += k**3
print("(c)", s, (n*(n + 1)//2)**2)
```

```
(a) 2500
(b) 1.5961632439130233   pi^2/6 = 1.6449340668482264
(c) 3025 3025
```

Exercício 1.4.

[5]:
```
soma, k = 0, 0
while soma <= 5000:
    k += 1
    soma += k
print("último número somado:", k, "  soma:", soma)
```

```
último número somado: 100   soma: 5050
```

Exercício 1.5.

[6]:
```
a1 = float(input("a1 = "))
r = float(input("razão = "))
n = int(input("n = "))
soma = 0
for k in range(n):
    termo = a1 + k*r
    print(termo, end=" ")
    soma += termo
print("\nsoma =", soma, "  fórmula =", n*(a1 + termo)/2)
```

```
a1 = 2
razão = 3
n = 6
2.0 5.0 8.0 11.0 14.0 17.0
soma = 57.0   fórmula = 57.0
```

Exercício 1.6.

[7]:
```
for n in [6, 27]:
    passos = 0
    seq = [n]
    while n != 1:
        n = n // 2 if n % 2 == 0 else 3*n + 1
        seq.append(n)
        passos += 1
    print(passos, "passos; maior valor:", max(seq), "; início:", seq[:8])
```

```
8 passos; maior valor: 16 ; início: [6, 3, 10, 5, 16, 8, 4, 2]
111 passos; maior valor: 9232 ; início: [27, 82, 41, 124, 62, 31, 94, 47]
```

Exercício 1.7.

[8]:
```
n = int(input("n = "))
qtd = 0
for d in range(1, n + 1):
    if n % d == 0:
        print(d, end=" ")
        qtd += 1
print()
print(n, "é primo" if qtd == 2 else "não é primo")
```

```
n = 28
1 2 4 7 14 28
28 não é primo
```

Aula 2

Exercício 2.1.

[9]:
```
L = [3, 8, -2, 7, 0, 5, -6, 1]
print(L[2], L[-1])
print([v for v in L if v > 0])
print([v**2 for v in L])
print(sum(L)/len(L), sorted(L, reverse=True))
```

```
-2 1
[3, 8, 7, 5, 1]
[9, 64, 4, 49, 0, 25, 36, 1]
2.0 [8, 7, 5, 3, 1, 0, -2, -6]
```

Exercício 2.2.

[10]:
```
def soma_quadrados(n):
    return sum([k**2 for k in range(1, n + 1)])

print(all(soma_quadrados(n) == n*(n + 1)*(2*n + 1)//6 for n in range(1, 21)))
```

```
True
```

Exercício 2.3.

[11]:
```
def media(L):
    return sum(L) / len(L)

def desvio(L):
    m = media(L)
    return math.sqrt(sum([(v - m)**2 for v in L]) / len(L))

notas = [9.5, 7.5, 10, 8.0, 6.5]
print(media(notas), desvio(notas))
```

```
8.3 1.2884098726725126
```

Exercício 2.4.

[12]:
```
def eh_primo(n):
    if n < 2:
        return False
    d = 2
    while d*d <= n:
        if n % d == 0:
            return False
        d += 1
    return True

print([(p, p + 2) for p in range(2, 100) if eh_primo(p) and eh_primo(p + 2)])
```

```
[(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
```

Exercício 2.5.

[13]:
```
def mdc(a, b):
    while b != 0:
        a, b = b, a % b
    return a

def mmc(a, b):
    return a*b // mdc(a, b)

m = 1
for k in range(2, 11):
    m = mmc(m, k)
print(mdc(252, 198), mmc(4, 6), m)
```

```
18 12 2520
```

Exercício 2.6.

[14]:
```
def avalia(coef, x):
    return sum([a * x**k for k, a in enumerate(coef)])

def deriva(coef):
    return [k * coef[k] for k in range(1, len(coef))]

p = [1, -3, 0, 2]           # 1 - 3x + 2x^3
print(avalia(p, 2), deriva(p), avalia(deriva(p), 2))
```

```
11 [-3, 0, 6] 21
```

Aula 3

Exercício 3.1.

[15]:
```
def distancia(P, Q):
    return math.hypot(Q[0] - P[0], Q[1] - P[1])

A, B, C = (0, 0), (4, 0), (1, 3)
print(distancia(A, B) + distancia(B, C) + distancia(C, A))
```

```
11.404918347287664
```

Exercício 3.2.

[16]:
```
def angulo(h, m):
    dif = abs((30*h + 0.5*m) - 6*m) % 360
    return min(dif, 360 - dif)

print(angulo(3, 0), angulo(9, 45), angulo(12, 30))
```

```
90.0 22.5 165.0
```

Exercício 3.3.

[17]:
```
for n in [5, 10, 15]:
    s = sum([1/math.factorial(k) for k in range(n + 1)])
    print(n, s, abs(s - math.e))
for n in [10, 1000, 100000]:
    s = 4*sum([(-1)**k/(2*k + 1) for k in range(n + 1)])
    print(n, s, abs(s - math.pi))
```

```
5 2.716666666666667 0.0016151617923783057
10 2.7182818011463845 2.7312660577649694e-08
15 2.7182818284589945 5.062616992290714e-14
10 3.232315809405593 0.09072315581579993
1000 3.1425916543395434 0.000999000749750234
100000 3.141602653489794 9.999900000945416e-06
```

A série de e converge muito mais rápido: o erro cai como $\frac{1}{(n+1)!}$, enquanto na de Leibniz cai como $\frac{1}{n}$.

Exercício 3.4.

[18]:
```
def seno(x, n):
    return sum([(-1)**k * x**(2*k + 1) / math.factorial(2*k + 1) for k in
    ↪ range(n + 1)])

for x in [1, 10]:
    print(x, seno(x, 5), seno(x, 20), math.sin(x))
```

```
1 0.841470984648068 0.8414709848078965 0.8414709848078965
10 -1056.9392336059002 -0.5440211107317661 -0.5440211108893698
```

Exercício 3.5.

[19]:
```
def raiz_cubica(x, tol=1e-12):
    y = x if abs(x) > 1 else 1.0
    while True:
        novo = (2*y + x/y**2) / 3
        if abs(novo - y) < tol:
            return novo
        y = novo

print(raiz_cubica(27), raiz_cubica(2), math.cbrt(2), raiz_cubica(-8))
```

```
3.0 1.2599210498948732 1.2599210498948734 -2.0
```

Exercício 3.6.

[20]:
```
def bisseccao(f, a, b, tol=1e-10):
    while (b - a)/2 > tol:
        m = (a + b)/2
        if f(a)*f(m) <= 0:
            b = m
        else:
            a = m
    return (a + b)/2

print(bisseccao(lambda x: x**2 - 2, 1, 2), math.sqrt(2))
print(bisseccao(lambda x: math.exp(-x) - x, 0, 1))
print(bisseccao(lambda x: x**7 + x - 1, 0, 1))
```

```
1.4142135623260401 1.4142135623730951
0.5671432903618552
0.7965443541179411
```

Aula 4

Exercício 4.1.

[21]:
```
print(np.arange(1, 40, 2))
print(np.linspace(-1, 1, 11))
print(2**np.arange(11))
print(1/np.arange(1, 11))
```

```
[ 1  3  5  7  9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39]
[-1.  -0.8 -0.6 -0.4 -0.2  0.   0.2  0.4  0.6  0.8  1. ]
[   1    2    4    8   16   32   64  128  256  512 1024]
[1.         0.5        0.33333333 0.25       0.2        0.16666667
 0.14285714 0.125      0.11111111 0.1       ]
```

Exercício 4.2.

[22]:
```
x = np.linspace(0, np.pi, 7)
for xi, s, c in zip(x, np.sin(x), np.cos(x)):
    print(f"{xi:6.3f} {s:8.4f} {c:8.4f} {s**2 + c**2:6.2f}")
```

```
 0.000   0.0000   1.0000   1.00
 0.524   0.5000   0.8660   1.00
 1.047   0.8660   0.5000   1.00
 1.571   1.0000   0.0000   1.00
 2.094   0.8660  -0.5000   1.00
 2.618   0.5000  -0.8660   1.00
 3.142   0.0000  -1.0000   1.00
```

Exercício 4.3.

[23]:
```
h = np.array([1.72, 1.65, 1.80, 1.58, 1.75, 1.69, 1.83, 1.61])
print(h.mean(), h.std(), np.sum(h > 1.70), h[h > 1.70].mean())
```

```
1.70375 0.08275528684017716 4 1.775
```

Exercício 4.4.

[24]:
```
k = np.arange(1, 100001)
H = np.cumsum(1/k)
n = np.argmax(H > 10) + 1          # primeiro índice em que passa de 10
print(n, H[n-1])
N = 10**6
print(np.sum(1/np.arange(1, N + 1)) - np.log(N))
```

```
12367 10.000043008275778
0.5772161649014524
```

Exercício 4.5.

[25]:
```
def simpson(f, a, b, n):
    x = np.linspace(a, b, n + 1)
```

```
    y = f(x)
    h = (b - a)/n
    return h/3 * (y[0] + 4*np.sum(y[1:-1:2]) + 2*np.sum(y[2:-1:2]) + y[-1])

def trapezios(f, a, b, n):
    x = np.linspace(a, b, n + 1)
    y = f(x)
    return (b - a)/n * (np.sum(y) - (y[0] + y[-1])/2)

g = lambda x: 4/(1 + x**2)
for n in [10, 20, 40]:
    print(n, abs(trapezios(g, 0, 1, n) - np.pi), abs(simpson(g, 0, 1, n) -
    ↪ np.pi))
```

```
10 0.0016666646826344333 3.9650577932093256e-08
20 0.0004166663566441997 6.200080449048073e-10
40 0.00010416666618251469 9.688250202088966e-12
```

Exercício 4.6.

[26]:
```
rng = np.random.default_rng(0)
u = rng.random(100000)
print(np.mean(np.exp(-u**2)), trapezios(lambda x: np.exp(-x**2), 0, 1, 100))
```

```
0.7471060128813966 0.7468180014679697
```

Aula 5

Exercício 5.1.

[27]:
```
A = np.array([[1, 2, 0], [0, 1, 3], [4, 0, 1]])
B = np.array([[2, 0, 1], [1, 1, 0], [0, 3, 1]])
print(A + B)
print(A @ B)
print(B @ A)
print(A.T @ B)
print(np.linalg.det(A @ B), np.linalg.det(A) * np.linalg.det(B))
print(np.array_equal((A @ B).T, B.T @ A.T))
```

```
[[3 2 1]
 [1 2 3]
 [4 3 2]]
[[ 4  2  1]
 [ 1 10  3]
 [ 8  3  5]]
[[ 6  4  1]
 [ 1  3  3]
 [ 4  3 10]]
[[ 2 12  5]
 [ 5  1  2]
 [ 3  6  1]]
125.00000000000004 124.99999999999996
True
```

Exercício 5.2. (Usa a função gauss da Aula 5.)

[28]:
```
def gauss(A, b):
    M = np.column_stack([A, b]).astype(float)
    n = len(b)
    for i in range(n - 1):
        p = i + np.argmax(np.abs(M[i:, i]))
        M[[i, p]] = M[[p, i]]
        for j in range(i + 1, n):
            M[j] = M[j] - M[j, i]/M[i, i] * M[i]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (M[i, -1] - M[i, i+1:n] @ x[i+1:]) / M[i, i]
    return x

A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])
b = np.array([8, -11, -3])
print(np.linalg.solve(A, b), gauss(A, b))
```

```
[ 2.  3. -1.] [ 2.  3. -1.]
```

Exercício 5.3.

[29]:
```
for n in [4, 8, 12]:
    i = np.arange(1, n + 1).reshape(n, 1)
    H = 1/(i + i.T - 1)
    x = np.ones(n)
    y = np.linalg.solve(H, H @ x)
    print(n, f"erro máximo = {np.max(np.abs(y - x)):.1e}", f"cond =
    ↪ {np.linalg.cond(H):.1e}")
```

```
4 erro máximo = 5.4e-14 cond = 1.6e+04
8 erro máximo = 1.3e-07 cond = 1.5e+10
12 erro máximo = 2.8e-01 cond = 1.6e+16
```

A matriz de Hilbert é mal condicionada: pequenos erros de arredondamento são ampliados por um fator da ordem do número de condição, e para $n = 12$ a solução perde quase todos os algarismos corretos.

Exercício 5.4.

[30]:
```
A = np.array([[4, 1], [2, 3]])
lam, V = np.linalg.eig(A)
lam = lam.real
V = V.real
for k in range(2):
    print(lam[k], np.allclose(A @ V[:, k], lam[k] * V[:, k]))
print(lam.sum(), np.trace(A), lam.prod(), np.linalg.det(A))
```

```
5.0 True
2.0 True
7.0 7 10.0 10.000000000000002
```

Exercício 5.5.

[31]:
```
for p in [[1, 0, -5, 0, 4], [1, 0, -2, -5]]:
    r = np.roots(p)
    print(r, np.round(np.abs(np.polyval(p, r)), 10))
```

```
[-2.  2. -1.  1.] [0. 0. 0. 0.]
[ 2.09455148+0.j         -1.04727574+1.13593989j -1.04727574-1.13593989j]
    ↪ [0. 0. 0.]
```

Exercício 5.6.

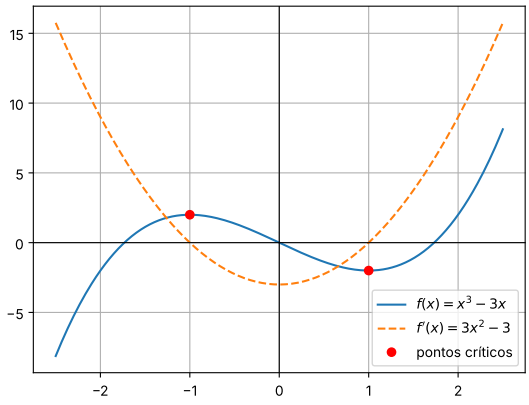
[32]:
```
t = np.array([0, 0.5, 1, 1.5, 2, 2.5])
h = np.array([1.0, 7.9, 12.4, 14.6, 14.3, 11.7])
a, b, c = np.polyfit(t, h, 2)
print(f"h(t) = {a:.3f} t^2 + {b:.3f} t + {c:.3f}")
print(f"g ~ {-2*a:.2f} m/s^2,  altura máxima em t = {-b/(2*a):.2f} s")
```

```
h(t) = -4.764 t^2 + 16.191 t + 0.996
g ~ 9.53 m/s^2,  altura máxima em t = 1.70 s
```

Aula 6

Exercício 6.1.

[33]:
```
x = np.linspace(-2.5, 2.5, 300)
plt.plot(x, x**3 - 3*x, label="$f(x)=x^3-3x$")
plt.plot(x, 3*x**2 - 3, "--", label="$f'(x)=3x^2-3$")
plt.plot([-1, 1], [2, -2], "ro", label="pontos críticos")
plt.axhline(0, color="k", lw=0.8); plt.axvline(0, color="k", lw=0.8)
plt.grid(True); plt.legend()
plt.show()
```



Exercício 6.2.

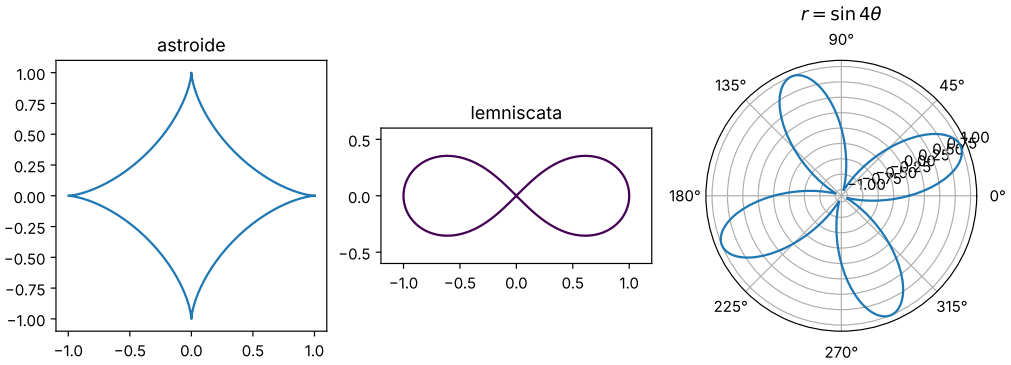
[34]:
```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
x = np.linspace(-2, 2, 200)
for base, nome in [(2, "$2^x$"), (np.e, "$e^x$"), (10, "$10^x$")]:
    ax[0].plot(x, base**x, label=nome)
ax[0].set_ylim(0, 8); ax[0].legend()
x = np.linspace(0.05, 10, 300)
ax[1].plot(x, np.log2(x), label=r"$\log_2 x$")
ax[1].plot(x, np.log(x), label=r"$\ln x$")
ax[1].plot(x, np.log10(x), label=r"$\log_{10} x$")
ax[1].legend()
plt.show()
```

Exercício 6.3. O vértice de $y = x^2 + bx + 1$ é $\left(-\frac{b}{2}, 1 - \frac{b^2}{4}\right)$; escrevendo $x_v = -\frac{b}{2}$, temos $y_v = 1 - x_v^2$: os vértices estão sobre a parábola $y = 1 - x^2$.

[35]:
```
x = np.linspace(-4, 4, 300)
for b in range(-3, 4):
    plt.plot(x, x**2 + b*x + 1, lw=1)
    plt.plot(-b/2, 1 - b**2/4, "ko")
plt.plot(x, 1 - x**2, "r--", label="$y = 1 - x^2$")
plt.ylim(-3, 6); plt.legend()
plt.show()
```

Exercício 6.4.

[36]:
```
t = np.linspace(0, 2*np.pi, 400)
fig = plt.figure(figsize=(11, 3.5))
ax1 = fig.add_subplot(1, 3, 1)
ax1.plot(np.cos(t)**3, np.sin(t)**3); ax1.set_aspect("equal");
    ↪ ax1.set_title("astroide")
ax2 = fig.add_subplot(1, 3, 2)
X, Y = np.meshgrid(np.linspace(-1.2, 1.2, 400), np.linspace(-0.6, 0.6, 300))
ax2.contour(X, Y, (X**2 + Y**2)**2 - (X**2 - Y**2), levels=[0])
ax2.set_aspect("equal"); ax2.set_title("lemniscata")
ax3 = fig.add_subplot(1, 3, 3, projection="polar")
ax3.plot(t, np.sin(4*t)); ax3.set_title(r"$r = \sin 4\theta$")
plt.show()
```



Exercício 6.5. O máximo fica perto de $\left(\frac{1}{\sqrt{2}}, 0\right)$ e o mínimo perto de $\left(-\frac{1}{\sqrt{2}}, 0\right)$ (confirme na Aula 8 igualando as derivadas parciais a zero).

[37]:
```
x = np.linspace(-2, 2, 100)
X, Y = np.meshgrid(x, x)
Z = X*np.exp(-X**2 - Y**2)
fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.plot_surface(X, Y, Z, cmap="coolwarm")
ax2 = fig.add_subplot(1, 2, 2)
c = ax2.contour(X, Y, Z, levels=15, cmap="coolwarm")
ax2.clabel(c, fontsize=7); ax2.set_aspect("equal")
plt.show()
```

Exercício 6.6.

[38]:
```
f = np.sqrt
x = np.linspace(0, 4, 200)
plt.plot(x, f(x), "k", lw=2, label=r"$\sqrt{x}$")
for hh in [2, 1, 0.5, 0.1]:
    m = (f(1 + hh) - f(1)) / hh
    plt.plot(x, f(1) + m*(x - 1), lw=1, label=f"secante h = {hh}")
plt.plot(x, 1 + (x - 1)/2, "r--", lw=2, label="tangente")
plt.ylim(0, 2.5); plt.legend(fontsize=8)
plt.show()
```

Aula 7

[39]:
```
x, y, a, b, c = sp.symbols('x y a b c')
```

Exercício 7.1.

[40]:
```
for e in [sp.sqrt(12) + sp.sqrt(27), sp.Rational(1, 3) + sp.Rational(2, 7),
    ↪ sp.exp(sp.pi*sp.sqrt(163))]:
    print(e, "=", sp.N(e, 30))
```

```
5*sqrt(3) = 8.66025403784438646763723170753
13/21 = 0.619047619047619047619047619048
exp(sqrt(163)*pi) = 262537412640768743.999999999999
```

Exercício 7.2.

[41]:
```
print(sp.expand((x - 2*y)**6))
print(sp.factor(x**6 - 1), "  |  ", sp.factor(x**4 - 5*x**2 + 4))
print(sp.cancel((x**3 - 8)/(x**2 - 4)))     # simplify não reduz esta
    ↪ fração; cancel sim
print(sp.apart((x + 1)/(x**2*(x - 1))))
```

```
x**6 - 12*x**5*y + 60*x**4*y**2 - 160*x**3*y**3 + 240*x**2*y**4 - 192*x*y**5
    ↪ + 64*y**6
(x - 1)*(x + 1)*(x**2 - x + 1)*(x**2 + x + 1)   |   (x - 2)*(x - 1)*(x +
    ↪ 1)*(x + 2)
(x**2 + 2*x + 4)/(x + 2)
2/(x - 1) - 2/x - 1/x**2
```

Exercício 7.3. Em (c), elevar ao quadrado dá $x^2 - 7x + 6 = 0$, com raízes 1 e 6; mas $x = 1$ não satisfaz a equação original (é uma raiz "estranha"). O `solve` já descarta essa raiz.

[42]:
```
print(sp.solve(x**3 - 6*x**2 + 11*x - 6, x))
print(sp.solve(2**x - 32, x))
print(sp.solve(sp.Eq(sp.sqrt(x + 3), x - 3), x))
print(sp.solveset(x**4 + x**2 - 12, x, sp.S.Reals), sp.solveset(x**4 + x**2
    ↪ - 12, x, sp.S.Complexes))
print(sp.solveset(sp.Abs(2*x - 1) < 3, x, sp.S.Reals))
```

```
[1, 2, 3]
[5]
[6]
{-sqrt(3), sqrt(3)} {-sqrt(3), sqrt(3), -2*I, 2*I}
Interval.open(-1, 2)
```

Exercício 7.4.

[43]:
```
z = sp.symbols('z')
print(sp.linsolve([x + 2*y - z - 4, 2*x - y + z - 1, x + y + z - 6], [x, y,
    ↪ z]))
print(sp.solve([x**2 + y**2 - 25, x - y - 1], [x, y], dict=True))
p = a*x**2 + b*x + c
print(sp.solve([p.subs(x, 1) - 2, p.subs(x, 2) - 3, p.subs(x, 3) - 6], [a,
    ↪ b, c]))
```

```
{(5/7, 20/7, 17/7)}
[{x: -3, y: -4}, {x: 4, y: 3}]
{a: 1, b: -2, c: 3}
```

Exercício 7.5. O gráfico (`sp.plot(sp.exp(-x) - x**2 + 2, (x, -3, 3))`) mostra duas raízes, perto de -1 e de 1,5.

[44]:
```
f = sp.exp(-x) - x**2 + 2
print(sp.nsolve(f, x, -1), sp.nsolve(f, x, 1.5))
```

```
1.49164419466824 1.49164419466824
```

Exercício 7.6.

[45]:
```
k, n = sp.symbols('k n', integer=True, positive=True)
print(sp.factor(sp.summation(2*k - 1, (k, 1, n))), "|",
    ↪ sp.factor(sp.summation(k*(k + 1), (k, 1, n))),
      "|", sp.summation(2**k, (k, 0, n)))
print(sp.summation(1/(k*(k + 1)), (k, 1, sp.oo)),
    ↪ sp.summation(1/sp.factorial(k), (k, 0, sp.oo)))
```

```
n**2 | n*(n + 1)*(n + 2)/3 | 2**(n + 1) - 1
1 E
```

Aula 8

Exercício 8.1.

[46]:
```
print(sp.limit((sp.tan(x) - x)/x**3, x, 0, '+-'),
      sp.limit((1 + 2/x)**x, x, sp.oo),
      sp.limit(x*sp.log(x), x, 0, '+'),
      sp.limit(sp.sqrt(x**2 + x) - x, x, sp.oo),
      sp.limit((sp.sqrt(x + 1) - 2)/(x - 3), x, 3, '+-'))
```

```
1/3 exp(2) 0 1/2 1/4
```

Exercício 8.2.

[47]:
```
kk = sp.symbols('k')
esq = sp.limit(kk*x**2, x, 2, '-')
dir_ = sp.limit(2*x + kk, x, 2, '+')
print(esq, dir_, sp.solve(sp.Eq(esq, dir_), kk))
```

```
4*k k + 4 [4/3]
```

Exercício 8.3.

[48]:
```
for f in [x**x, sp.log((1 + x)/(1 - x)), sp.exp(-x**2)*sp.cos(3*x),
    ↪ sp.asin(sp.sqrt(x))]:
    print(sp.simplify(sp.diff(f, x)))
print(sp.diff(sp.exp(-x**2)*sp.cos(3*x), x).subs(x, 0))
```

```
x**x*(log(x) + 1)
-2/(x**2 - 1)
-(2*x*cos(3*x) + 3*sin(3*x))*exp(-x**2)
1/(2*sqrt(x)*sqrt(1 - x))
0
```

Exercício 8.4.

[49]:
```
f = x/(1 + x**2)
m = sp.diff(f, x).subs(x, 1)
y0 = f.subs(x, 1)
print("m =", m, "  tangente: y =", y0 + m*(x - 1), "  normal: x = 1" if m ==
    ↪ 0 else "")
```

```
m = 0   tangente: y = 1/2   normal: x = 1
```

Como $f'(1) = 0$, a tangente é horizontal, $y = \frac{1}{2}$, e a normal é a reta vertical $x = 1$.

Exercício 8.5.

[50]:
```
f = x**4 - 4*x**3 + 10
df, d2f = sp.diff(f, x), sp.diff(f, x, 2)
print("críticos:", sp.solve(df, x), "  inflexão (candidatos):",
    ↪ sp.solve(d2f, x))
for c in sp.solve(df, x):
    print(c, f.subs(x, c), d2f.subs(x, c))
print(sp.solveset(df > 0, x, sp.S.Reals))
```

```
críticos: [0, 3]   inflexão (candidatos): [0, 2]
0 10 0
3 -17 36
Interval.open(3, oo)
```

f decresce em $(-\infty, 3)$ e cresce em $(3, \infty)$; mínimo em $x = 3$ ($f = -17$); em $x = 0$ há ponto crítico sem extremo (é ponto de inflexão com tangente horizontal); as inflexões estão em $x = 0$ e $x = 2$.

Exercício 8.6.

[51]:
```
r = sp.symbols('r', positive=True)
A = 2*sp.pi*r**2 + 2*sp.pi*r*1000/(sp.pi*r**2)
rc = sp.solve(sp.diff(A, r), r)[0]
h = 1000/(sp.pi*rc**2)
print(rc, "~", rc.evalf(4), "cm;  h ~", h.evalf(4), "cm;  h/r =",
    ↪ sp.simplify(h/rc))
```

```
5*2**(2/3)/pi**(1/3) ~ 5.419 cm;  h ~ 10.84 cm;  h/r = 2
```

A lata ótima tem altura igual ao diâmetro ($h = 2r$).

Exercício 8.7.

[52]:
```
P5 = sp.series(sp.sin(x), x, 0, 6).removeO()
aprox = P5.subs(x, sp.Rational(1, 2))
erro = abs(sp.sin(sp.Rational(1, 2)) - aprox).evalf()
print(P5, "|", aprox.evalf(12), "| erro:", erro, "| estimativa:",
    ↪ (sp.Rational(1, 2)**7/sp.factorial(7)).evalf())
```

```
x**5/120 - x**3/6 + x | 0.479427083333 | erro: 1.54472913033306e-6 |
    ↪ estimativa: 1.55009920634921e-6
```

Aula 9

Exercício 9.1.

[53]:
```
for f in [x**2*sp.exp(-x), (x + 3)/(x**2 + x - 2), sp.sec(x)**2,
    ↪ x*sp.log(x), 1/(1 + 4*x**2)]:
    F = sp.integrate(f, x)
    print(F, "   ok" if sp.simplify(sp.diff(F, x) - f) == 0 else "
    ↪ conferir")
```

```
(-x**2 - 2*x - 2)*exp(-x)    ok
4*log(x - 1)/3 - log(x + 2)/3    ok
sin(x)/cos(x)    ok
x**2*log(x)/2 - x**2/4    ok
atan(2*x)/2    ok
```

Exercício 9.2.

[54]:
```
print(sp.integrate(sp.sin(x)**2, (x, 0, sp.pi/2)), sp.integrate(sp.log(x)/x,
    ↪ (x, 1, sp.E)),
      sp.integrate(x*sp.exp(-x), (x, 0, sp.oo)), sp.integrate(1/sp.sqrt(x),
    ↪ (x, 0, 1)),
      sp.integrate(1/x, (x, 0, 1)))
```

```
pi/4 1/2 1 2 oo
```

Exercício 9.3. As curvas se cruzam em $x = -2, 0, 2$; por simetria, a área é o dobro da área em $[0, 2]$.

[55]:
```
f, g = 3*x, x**3 - x
print(sp.solve(f - g, x))
area = sp.integrate(g - f, (x, -2, 0)) + sp.integrate(f - g, (x, 0, 2))
print(area, sp.integrate(sp.Abs(f - g), (x, -2, 2)))
```

```
[-2, 0, 2]
8 8
```

Exercício 9.4.

[56]:
```
for N in [10, 100, 1000]:
    dx = 4/N
    xe = np.linspace(0, 4 - dx, N)
    somas = [np.sum(np.sqrt(xi))*dx for xi in (xe, xe + dx/2, xe + dx)]
    print(N, ["%.6f" % abs(s - 16/3) for s in somas])
```

```
10 ['0.449259', '0.013737', '0.350741']
100 ['0.041630', '0.000470', '0.038370']
1000 ['0.004052', '0.000015', '0.003948']
```

O ponto médio tem o menor erro.

Exercício 9.5.

[57]:
```
R, hh = sp.symbols('R h', positive=True)
print(sp.pi*sp.integrate(sp.exp(-x)**2, (x, 0, sp.oo)),
    ↪ sp.pi*sp.integrate((R/hh*x)**2, (x, 0, hh)))
```

```
pi/2 pi*R**2*h/3
```

Exercício 9.6.

[58]:
```
print(sp.integrate(sp.sin(x), (x, 0, sp.pi))/sp.pi, sp.integrate(x*y, (y, 0,
    ↪ x), (x, 0, 1)))
```

```
2/pi 1/8
```

Aula 10

Exercício 10.1.

[59]:
```
A = sp.Matrix([[2, -1, 0], [-1, 2, -1], [0, -1, 2]])
print(A.det(), A.inv())
print(sp.factor(A.charpoly(x).as_expr()), A.eigenvals())
print(A.is_diagonalizable())
```

```
4 Matrix([[3/4, 1/2, 1/4], [1/2, 1, 1/2], [1/4, 1/2, 3/4]])
(x - 2)*(x**2 - 4*x + 2) {2: 1, 2 - sqrt(2): 1, sqrt(2) + 2: 1}
True
```

Exercício 10.2.

[60]:
```
a = sp.symbols('a')
M = sp.Matrix([[1, 1, 1], [1, a, 1], [1, 1, a]])
bv = sp.Matrix([1, 2, 3])
print(sp.factor(M.det()))
print(M.subs(a, 1).rank(), M.subs(a, 1).row_join(bv).rank())
print(sp.simplify(M.LUsolve(bv)).T)
```

```
(a - 1)**2
1 2
Matrix([[(a - 4)/(a - 1), 1/(a - 1), 2/(a - 1)]])
```

Solução única para $a \neq 1$; para $a = 1$ os postos são 1 e 2: sistema impossível.

Exercício 10.3.

[61]:
```
t = sp.symbols('t')
yf = sp.Function('y')
edos = [(sp.Eq(yf(t).diff(t), -2*t*yf(t)), {yf(0): 1}),
        (sp.Eq(yf(t).diff(t) + yf(t), t), None),
        (sp.Eq(yf(t).diff(t), yf(t)/t + t), None)]
for edo, ci in edos:
```

```
    sol = sp.dsolve(edo, yf(t), ics=ci)
    print(sol, sp.checkodesol(edo, sol))
```

```
Eq(y(t), exp(-t**2)) (True, 0)
Eq(y(t), C1*exp(-t) + t - 1) (True, 0)
Eq(y(t), t*(C1 + t)) (True, 0)
```

Exercício 10.4.

[62]:
```
T = sp.Function('T')
k = sp.symbols('k', positive=True)
sol = sp.dsolve(sp.Eq(T(t).diff(t), -k*(T(t) - 25)), T(t), ics={T(0):
    ↪ 90}).rhs
kv = sp.solve(sp.Eq(sol.subs(t, 5), 70), k)[0]
t40 = sp.solve(sp.Eq(sol.subs(k, kv), 40), t)[0]
print(sol, "|  k =", kv, "~", kv.evalf(4), "|  t =", t40.evalf(4), "min")
```

```
25 + 65*exp(-k*t) |  k = log(13**(1/5)*3**(3/5)/3) ~ 0.07355 |  t = 19.94 min
```

Exercício 10.5.

[63]:
```
amort = sp.dsolve(yf(t).diff(t, 2) + yf(t).diff(t) + yf(t), yf(t),
    ↪ ics={yf(0): 1, yf(t).diff(t).subs(t, 0): 0})
livre = sp.dsolve(yf(t).diff(t, 2) + yf(t), yf(t), ics={yf(0): 1,
    ↪ yf(t).diff(t).subs(t, 0): 0})
print(amort)
print(livre)
```

```
Eq(y(t), (sqrt(3)*sin(sqrt(3)*t/2)/3 + cos(sqrt(3)*t/2))*exp(-t/2))
Eq(y(t), cos(t))
```

O gráfico pode ser feito com `sp.plot(amort.rhs, livre.rhs, (t, 0, 15))`: a solução amortecida oscila com amplitude decrescente, a outra mantém amplitude 1.

Exercício 10.6. A reta $y = t - 1$ é solução: $y' = 1$ e $t - y = 1$.

[64]:
```
sol = sp.dsolve(sp.Eq(yf(t).diff(t), t - yf(t)), yf(t))
print(sol)
```

```
Eq(y(t), C1*exp(-t) + t - 1)
```

Referências

[1] SOUZA, A. L. L. *Python para Matemáticos*. V Colóquio de Matemática da Região Nordeste. Rio de Janeiro: SBM, 2023. (Capítulos 3 a 6: base das Aulas 1 a 6.)

[2] MARTINS, R. M. *Tutorial rápido de Python para Matemáticos*. IMECC/Unicamp, 2022. Disponível em https://rmiranda99.github.io/tutorial-math-python/. (Base das Aulas 3 e 7 a 10.)

[3] WALLS, P. *Mathematical Python*. University of British Columbia. https://personal.math.ubc.ca/~pwalls/math-python/.

[4] Documentação oficial do Python (em português): https://docs.python.org/pt-br/3/.

[5] Documentação do módulo `math`: https://docs.python.org/pt-br/3/library/math.html.

[6] Documentação do NumPy: https://numpy.org/doc/stable/.

[7] Documentação do Matplotlib: https://matplotlib.org/stable/.

[8] Documentação do SymPy: https://docs.sympy.org/latest/.

[9] Projeto Jupyter / JupyterLab: https://jupyter.org/. Versão no navegador: https://jupyter.org/try-jupyter.

[10] LYNCH, S. *Dynamical Systems with Applications using Python*. Birkhäuser, 2018.